

POLITECNICO DI TORINO
MASTER OF SCIENCE IN COMPUTER ENGINEERING
Department of Control and Computer Engineering (DAUIN)
Database and Data Mining Group (DBDMG)

Large Language Models for Software Engineering

Theoretical foundations, methodologies, and practical architectures

Comprehensive master's course study guide

Authored for graduate students, researchers, and practitioners

Course Instructors:

Prof. Flavio Giobergia
Prof. Riccardo Coppola

Politecnico di Torino

Academic Year:

2025 / 2026

Target Program:

M.Sc. Computer Engineering

Contents

Preface	iii
Notation and conventions	v
I Foundations of large language models	1
1 Introduction to language models	2
1.1 Chapter overview and learning outcomes	2
1.2 Probabilistic language modeling and the chain rule	3
1.3 Markov assumptions and n -gram language models	5
1.4 Transition count matrix construction walkthrough	7
1.5 Autoregressive estimation and generation	8
1.6 Fundamental limitations of statistical n -gram models	10
1.7 Evaluation metrics: cross-entropy and perplexity	12
1.8 Historical evolution of language models	16
2 Deep learning foundations for NLP	30
2.1 Chapter overview and learning outcomes	30
2.2 From biological neurons to the artificial perceptron	31
2.3 Activation functions and the proof of linear stacking collapse	33
2.4 Universal approximation theorem	38
2.5 Architectural task formulations and software engineering case studies	40
2.6 Loss function formulations and optimization landscapes	41
2.7 Computational graphs and exact adjoint backpropagation	44
3 Distributed word representations and Word2Vec	47
3.1 The representation problem: why continuous embeddings are mandatory	47
3.2 The baseline approach: local one-hot encodings and why they fail	48
3.3 Distributed representations and the distributional hypothesis	52
3.4 The embedding training loop and semantic geometry evolution	53
3.5 Word2Vec continuous bag-of-words (CBOW)	56
3.6 Word2Vec continuous skip-gram and the softmax bottleneck	58
3.7 Scaling softmax: hierarchical softmax and negative sampling	60
3.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier	65
3.9 FastText: subword character n -gram embeddings	67
3.10 Emergent vector space geometry: clustering, metrics and relational analogies	68
4 Recurrent neural networks and sequence modeling	74
4.1 Chapter overview and learning outcomes	74
4.2 Sequential data and constraints of feed-forward neural networks	75
4.3 The recurrent neural network cell and weight sharing	76

4.4	Backpropagation through time and the proof of vanishing gradients	77
4.5	Case correction demonstration: Sherlock Holmes training dynamics	80
4.6	Gated architectures: LSTM and GRU	84
4.7	Sequence-to-sequence (Seq2Seq) and encoder-decoder pipelines	86
4.8	Persistent bottlenecks of gated RNNs and the road to transformers	88

Preface

Pedagogical vision

The emergence of Large Language Models (LLMs) represents a fundamental paradigm shift in computer science and software engineering. Moving beyond rigid syntax-directed rules, modern software engineering increasingly leverages deep probabilistic models capable of synthesizing code, repairing complex bugs, formalizing requirements, and orchestrating multi-agent systems.

This study guide accompanies the Master of Science course *Large Language Models for Software Engineering* at Politecnico di Torino, taught by Prof. Flavio Giobergia and Prof. Riccardo Coppola. The curriculum is intentionally structured into two complementary pillars:

1. **Part I: Foundations of large language models** — Traces the mathematical and architectural lineage of language modeling, beginning with n -gram Markov chains, proceeding through feedforward networks, distributed word embeddings (Word2Vec, FastText), recurrent networks and vanishing gradient proofs, and culminating in the self-attention Transformer architecture, pre-training scaling laws, instruction tuning (RLHF, DPO), and parameter-efficient fine-tuning (LoRA, QLoRA).
2. **Part II: Large language models for software engineering** — Bridges foundational model capabilities with the Software Development Life Cycle (SDLC), exploring prompt engineering patterns, autonomous agent frameworks (ReAct, multi-agent collaboration), automated test generation, software maintenance and refactoring, rigorous evaluation methodologies (Pass@ k , CodeBLEU), and security, intellectual property, and ethical implications.

Visual callout structure

To maximize learning effectiveness and exam readiness, the guide embeds three specialized pedagogical environments:

Exam insights: Recurrent exam patterns and pitfalls

Highlighted in terracotta/amber, these boxes isolate specific questions frequently asked in oral and written examinations, point out common mathematical misconceptions (e.g., mixing logarithm bases when computing perplexity), and underscore key grading criteria.

Intuition & analogy: Geometric and conceptual analogies

Highlighted in royal purple, these boxes provide conceptual, visual, and intuitive interpretations of complex mathematical machinery (e.g., attention as a soft differentiable lookup table, high-dimensional vector space geometry).

External deep dive: Seminal research and authoritative multimedia

Highlighted in pine teal, these boxes contextualize textbook theory with seminal academic papers [1–3] and breakdowns by leading researchers and educators.

Examination framework and course organization

As detailed by the instructors in the introductory lecture, assessment in this course is divided into two distinct components:

1. Written Examination (18 Points):

- Duration: 75 minutes.
- Format: Typically around 12 questions (combining closed-ended multiple-choice items and open-ended technical/mathematical derivations).
- Scope: Comprehensive coverage across all foundational and applied software engineering topics.
- Threshold: A minimum score of 10/18 is strictly required to achieve sufficiency.

2. Practical Course Project (14 Points):

- Implementation: Development of an applied LLM-for-software-engineering system using Python and PyTorch.
- Modality: Undertaken individually or in small teams.
- Scoring: Total points across the written test (18) and project (14) sum to 32, with scores exceeding 30 earning *30 e Lode* (honors).

3. Lecture and Lab Schedule:

- Monday: 08:30 – 11:30 (Room 8) — Core theoretical foundations.
- Wednesday: 11:30 – 13:00 (Room R1) — Practical workshops, interactive coding, and PyTorch laboratories.

How to use this study guide

Each chapter is fully self-contained and begins with precise learning objectives, followed by exhaustive mathematical derivations, algorithmic walkthroughs, and practical software engineering implications. Students are encouraged to independently verify all mathematical derivations and implement the core algorithms to develop deep mechanical empathy for large language models.

Notation and conventions

Throughout this study guide, we adopt consistent mathematical, algorithmic, and typographic conventions aligned with standard deep learning and software engineering literature.

Linear algebra and tensor operations

Symbol	Type	Description
x, y, z	Scalar	Real-valued scalars or scalar components ($x \in \mathbb{R}$).
$\mathbf{x}, \mathbf{y}, \mathbf{h}$	Vector	Column vectors ($\mathbf{x} \in \mathbb{R}^d$). Lowercase bold font.
$\mathbf{x}^\top$	Transpose	Transpose of vector $\mathbf{x}$ (row vector $\mathbf{x}^\top \in \mathbb{R}^{1 \times d}$).
$\mathbf{A}, \mathbf{W}, \mathbf{X}$	Matrix	Real matrices ($\mathbf{A} \in \mathbb{R}^{m \times n}$). Uppercase bold font.
$\mathbf{A}_{i,j}$ or a_{ij}	Scalar	Entry at row i and column j of matrix $\mathbf{A}$.
$\mathbf{I}$ or $\mathbf{I}_d$	Matrix	Identity matrix of dimension $d \times d$.
$\ \mathbf{x}\ _2$ or $\ \mathbf{x}\ $	Scalar	Euclidean (L_2) norm of vector $\mathbf{x}$, $\sqrt{\sum_i x_i^2}$.
$\ \mathbf{A}\ _F$	Scalar	Frobenius norm of matrix $\mathbf{A}$, $\sqrt{\sum_{i,j} A_{i,j}^2}$.
$\langle \mathbf{u}, \mathbf{v} \rangle$ or $\mathbf{u}^\top \mathbf{v}$	Scalar	Standard dot (inner) product between vectors $\mathbf{u}$ and $\mathbf{v}$.
$\odot$	Operator	Element-wise (Hadamard) product between tensors of identical shape.
$\otimes$	Operator	Outer product or Kronecker tensor product.

Table 1: Linear algebra and tensor notation summary.

Probability and information theory

Neural networks, transformers, and software engineering

Symbol	Type	Description
$\mathbb{P}(A)$	Scalar	Probability of event A ($\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$).
$\mathbb{P}(X = x)$	Scalar	Probability mass function for discrete random variable X .
$\mathbb{P}(Y X)$	Distribution	Conditional probability distribution of Y given X .
$\mathbb{E}_{x \sim P}[f(x)]$	Scalar	Expected value of function $f(x)$ under distribution P .
$\text{Var}(X)$	Scalar	Variance of random variable X , $\mathbb{E}[(X - \mathbb{E}[X])^2]$.
$\mathcal{H}(P)$	Scalar	Shannon entropy of distribution P , $-\sum_x P(x) \log P(x)$.
$\mathcal{H}(P, Q)$	Scalar	Cross-entropy between target distribution P and model Q .
$D_{\text{KL}}(P \parallel Q)$	Scalar	Kullback–Leibler (KL) divergence from Q to P .
$\text{PPL}(W)$	Scalar	Perplexity of word sequence W under a language model.

Table 2: Probability and information theoretic notation.

Symbol	Type	Description
$\mathcal{V}$	Set	Discrete vocabulary of tokens; $ \mathcal{V} = V$ denotes vocabulary size.
T or T	Integer	Sequence length (number of input or output tokens).
d_{model}	Integer	Hidden embedding dimensionality of the Transformer model.
d_{ff}	Integer	Dimensionality of the intermediate Feed-Forward Network layer.
d_k, d_v	Integer	Dimensionality of query/key and value projections per head.
h	Integer	Number of parallel attention heads ($h \cdot d_k = d_{\text{model}}$).
$\sigma(z)$	Function	Standard logistic sigmoid function: $\sigma(z) = \frac{1}{1+e^{-z}}$.
$\text{softmax}(\mathbf{z})$	Function	Softmax normalized exponential vector operator.
$\text{ReLU}(z)$	Function	Rectified Linear Unit activation: $\max(0, z)$.
$\text{GELU}(z)$	Function	Gaussian Error Linear Unit activation.
$\mathcal{L}$ or $\mathcal{L}$	Scalar	Scalar objective / loss function minimized during optimization.
$\text{Pass}@k$	Metric	Probability that at least one of k generated code samples passes unit tests.
CodeBLEU	Metric	Multi-aspect code evaluation metric combining BLEU, AST, and data flow.

Table 3: Model architecture parameters and SE metrics.

Part I

Foundations of large language models

Introduction to language models

1.1 Chapter overview and learning outcomes

Language modeling represents the fundamental probabilistic bedrock of contemporary Natural Language Processing (NLP) and modern AI-assisted Software Engineering. At its core, a language model formalizes the intuitive mechanism by which humans evaluate the fluency, plausibility, and likelihood of word sequences. This chapter establishes the mathematical foundations of language modeling, beginning with joint probability distributions over discrete token vocabularies, developing the unconditional decomposition provided by the probability chain rule, and examining the classic Markovian simplifying assumptions that yield statistical n -gram models.

We explore empirical transition counting matrices, trace step-by-step autoregressive text generation, analyze the three crippling pathologies of classical counting models (combinatorial sparsity V^n , finite context horizon, and semantic blindness), and derive the rigorous information-theoretic metrics used to benchmark language models: Cross-Entropy and Perplexity. Finally, we trace the evolutionary arc from Claude Shannon's 1948 foundational communication theory to Bengio's seminal 2003 neural probabilistic model, concluding with modern compute-optimal scaling laws.

Upon completing this chapter, you will be able to:

- Decompose joint probability distributions over token sequences using the probabilistic chain rule without introducing independence assumptions.
- Formalize the Markov assumption of order $n - 1$ and construct empirical n -gram transition matrices from raw corpora.
- Compute exact joint probabilities for novel sentences from estimated conditional distributions.
- Walk through the mechanical execution of autoregressive sampling, observing degenerate looping behaviors and termination dynamics.
- Quantify the three classic n -gram failure modes: parameter explosion (V^n), context horizon truncation, and orthogonal semantic isolation.
- Derive Cross-Entropy and Perplexity mathematically, mastering the base-2 versus base- e logarithmic conversions critical for academic examinations.
- Situate modern decoder-only transformers within the historical trajectory spanning Shannon (1948), Potamianos and Jelinek (1998), Bengio et al. (2003), Radford et al. (2019), and Hoffmann et al. (2022).

1.2 Probabilistic language modeling and the chain rule

Let $\mathcal{V} = \{w_1, w_2, \dots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ discrete tokens (e.g., words, sub-words, or punctuation symbols). An arbitrary natural language sentence or document of length N is represented as an ordered sequence of discrete random variables $W = (w_1, w_2, \dots, w_N) \in \mathcal{V}^N$.

Definition 1.1: Statistical language model

A *statistical language model* is a formal probability distribution $\mathbb{P}$ defined over the set of all possible discrete sequences $\mathcal{S} = \bigcup_{N=1}^{\infty} \mathcal{V}^N$. For any specific sequence $W = (w_1, \dots, w_N)$, the language model assigns a scalar probability $\mathbb{P}(W) \in [0, 1]$ satisfying the Kolmogorov probability axioms:

$$\mathbb{P}(W) \geq 0 \quad \forall W \in \mathcal{S}, \quad \text{and} \quad \sum_{S \in \mathcal{S}} \mathbb{P}(S) = 1. \quad (1.1)$$

The model maps the syntactic and semantic conventions of human language into real values on the unit interval:

- Fluent, grammatically coherent sentences receive high probability mass:

$$\mathbb{P}(\text{"this", "is", "a", "reasonable", "sentence"}) = 0.1$$

- Syntactically valid but semantically unusual or rare sentences receive noticeably lower probability:

$$\mathbb{P}(\text{"this", "is", "a", "purple", "sentence"}) = 0.01$$

- Ungrammatical, corrupted, or disjoint sequences receive negligible probability:

$$\mathbb{P}(\text{"this", "are", "a", "reasonable", "sentence"}) = 0.001$$

Evaluating the joint probability $\mathbb{P}(w_1, \dots, w_N)$ directly over the combinatorial space $\mathcal{V}^N$ is intractable because observing every conceivable multi-word sequence in a training dataset is impossible. To structure this estimation problem, we invoke the fundamental *chain rule of probability*.

Definition 1.2: Chain rule decomposition of language models

For any sequence of random variables $(w_1, w_2, \dots, w_N)$, the joint probability factors identically and unconditionally into an ordered product of conditional next-token probabilities:

$$\begin{aligned} \mathbb{P}(w_1, w_2, \dots, w_N) &= \mathbb{P}(w_1) \cdot \mathbb{P}(w_2 | w_1) \cdot \mathbb{P}(w_3 | w_1, w_2) \cdots \mathbb{P}(w_N | w_1, \dots, w_{N-1}) \\ &= \prod_{t=1}^N \mathbb{P}(w_t | w_1, \dots, w_{t-1}) = \prod_{t=1}^N \mathbb{P}(w_t | w_{<t}), \end{aligned} \quad (1.2)$$

where $w_{<t} = (w_1, \dots, w_{t-1})$ represents the prefix history preceding timestep t . For the initial token $t = 1$, the conditioning prefix is the empty sequence $\emptyset$, giving $\mathbb{P}(w_1 | w_{<1}) = \mathbb{P}(w_1)$.

Applying Equation (1.2) to our running example yields the concrete factorization:

$$\begin{aligned} \mathbb{P}(\text{this, is, a, reasonable, sentence}) &= \mathbb{P}(\text{this}) \\ &\cdot \mathbb{P}(\text{is} \mid \text{this}) \\ &\cdot \mathbb{P}(\text{a} \mid \text{this, is}) \\ &\cdot \mathbb{P}(\text{reasonable} \mid \text{this, is, a}) \\ &\cdot \mathbb{P}(\text{sentence} \mid \text{this, is, a, reasonable}). \end{aligned}$$

Reframing language modeling as conditional prediction. This mathematical decomposition marks a pivotal conceptual shift in Natural Language Processing. Instead of attempting to score complete, static documents simultaneously, the modeling focus shifts entirely to sequential, conditional next-word estimation:

Given history $w_{<t}$, estimate the categorical probability distribution over all words in $\mathcal{V}$: $\mathbb{P}(w_t = w \mid w_{<t})$.

This formulation aligns directly with the classic psychological *cloze test* (fill-in-the-blank):

$$\text{“This is a reasonable _____”} \implies \mathbb{P}(\text{sentence} \mid \text{This is a reasonable}) = ?$$

Every modern autoregressive language model, including GPT-4 and Claude, relies fundamentally on this conditional next-token formulation.

💡 Intuition & analogy: The core thesis: next-word prediction as full language modeling

In the opening lecture at Politecnico di Torino, Prof. Giobergia crystallized the essence of modern language modeling:

“If we can predict the next word, then we have estimated this conditional probability. And then we can plug this into the chain rule, and estimate the probability of any arbitrary sentence. This is what a language model does: nothing more, nothing less. If you understand this idea, you understand the core mechanism driving the entire course, from 1980s Markov chains to modern trillion-parameter LLMs.”

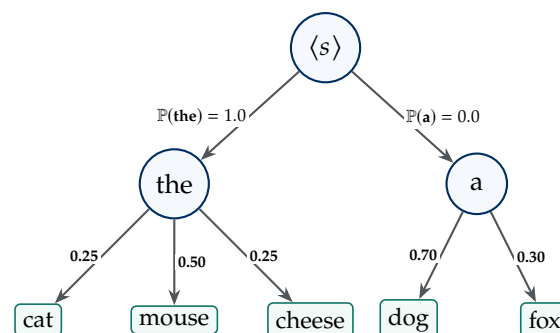


Figure 1.1: Visualizing autoregressive sequence generation as path traversal across a probability branching tree. At the root symbol $\langle s \rangle$, outgoing edges partition total probability mass across vocabulary tokens. At each node $w_{<t}$, children represent the conditional distribution $\mathbb{P}(w_t \mid w_{<t})$. The law of total probability guarantees that branch probabilities sum to unity at each decision point.

💡 Intuition & analogy: The branching probability tree

Sequence generation can be visualized geometrically as walking down a directed decision tree (Figure 1.1). At the root node $\langle s \rangle$, the model confronts the entire vocabulary $\mathcal{V}$. Choosing a token w_1 traverses an edge with probability $\mathbb{P}(w_1)$ and lands at a new conditioning state. From this state, another set of branches opens, weighted by $\mathbb{P}(w_2 | w_1)$. The joint probability of an entire sentence is simply the product of edge weights along that unique trajectory. Generating text consists of sampling a path from the root to a designated terminal token $\langle /s \rangle$.

1.3 Markov assumptions and n -gram language models

Although Equation (1.2) is mathematically exact and preserves all syntactic and semantic dependencies, computing $\mathbb{P}(w_t | w_{<t})$ directly in empirical applications presents an insurmountable combinatorial bottleneck. As the sequence index t grows, the conditioning history $w_{<t} = (w_1, \dots, w_{t-1})$ expands monotonically. Because each constituent token is drawn from a vocabulary $\mathcal{V}$ of size $V = |\mathcal{V}|$, the space of potential prefix histories preceding token t is the Cartesian product space $\mathcal{V}^{t-1}$, whose cardinality is:

$$|\mathcal{H}_t| = |\mathcal{V}|^{t-1} = V^{t-1}. \quad (1.3)$$

Even for a modest vocabulary of $V = 10,000$ words, tracking histories of length $t = 4$ requires estimating probabilities over $10,000^3 = 10^{12}$ distinct conditioning states. In any finite training corpus, almost every sequence of length greater than three occurs exactly zero times.

To make estimation mathematically tractable, the Russian mathematician Andrey Markov formulated the principle of conditional independence that now bears his name. In his landmark 1948 treatise on information theory, Claude Shannon [4] demonstrated that modeling natural language as a discrete Markov process produces increasingly coherent synthetic English as the conditioning order increases. Modern pedagogical treatments, notably by Jurafsky and Martin [5], formalize this simplification as the foundation of classical statistical natural language processing.

Definition 1.3: Markov assumption of order $n - 1$

An n -gram language model assumes that the conditional probability of the current word w_t depends exclusively on the immediately preceding $n - 1$ words, rendering it conditionally independent of all earlier discourse history:

$$\mathbb{P}(w_t | w_1, w_2, \dots, w_{t-1}) \approx \mathbb{P}(w_t | w_{t-n+1}, \dots, w_{t-1}). \quad (1.4)$$

The integer $n \geq 1$ dictates the order of the model:

- **Unigram Model ($n = 1$, Order 0):** Completely memoryless. Each word is treated as statistically independent of all past and future tokens:

$$\mathbb{P}(w_1, \dots, w_N) \approx \prod_{t=1}^N \mathbb{P}(w_t).$$

- **Bigram Model ($n = 2$, Order 1):** First-order Markov chain. Conditioning depends

exclusively on the single preceding token w_{t-1} :

$$\mathbb{P}(w_1, \dots, w_N) \approx \prod_{t=1}^N \mathbb{P}(w_t \mid w_{t-1}).$$

- **Trigram Model ($n = 3$, Order 2):** Second-order Markov chain. Conditioning depends on the preceding pair of tokens (w_{t-2}, w_{t-1}):

$$\mathbb{P}(w_1, \dots, w_N) \approx \prod_{t=1}^N \mathbb{P}(w_t \mid w_{t-2}, w_{t-1}).$$

1.3.1 Combinatorial state-space reduction

The primary mathematical virtue of the Markov assumption is that it truncates an exponentially growing state space into a fixed, stationary cardinality. Under an n -gram model, the conditioning context is always restricted to $n - 1$ words, regardless of the absolute sequence position t :

$$|\mathcal{H}^{(n)}| = |\mathcal{V}|^{n-1} = V^{n-1}. \quad (1.5)$$

Consequently, specifying the complete transition probability distribution across the entire vocabulary requires an $(n - 1)$ -dimensional conditioning tensor containing:

$$P_{\text{params}} = V^{n-1} \times (V - 1) \approx V^n \quad (1.6)$$

free parameters. Table 1.1 contrasts the unconstrained historical state space with fixed n -gram models for realistic vocabulary sizes.

Table 1.1: State-space cardinality $|\mathcal{H}|$ as a function of sequence position t versus fixed n -gram Markov orders for vocabularies $V = 10,000$ and $V = 50,000$.

Model / Order	Effective Context	Formula	States ($V = 10^4$)	States ($V = 5 \times 10^4$)
Unconstrained ($t = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Unconstrained ($t = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
Unconstrained ($t = 5$)	4 previous tokens	V^4	1.0×10^{16}	6.25×10^{18}
Unconstrained ($t = 10$)	9 previous tokens	V^9	1.0×10^{36}	1.95×10^{42}
Unconstrained ($t = 20$)	19 previous tokens	V^{19}	1.0×10^{76}	1.90×10^{89}
Unigram ($n = 1$)	0 tokens (memoryless)	$V^0 = 1$	1	1
Bigram ($n = 2$)	1 previous token	V^1	1.0×10^4	5.0×10^4
Trigram ($n = 3$)	2 previous tokens	V^2	1.0×10^8	2.5×10^9
4-gram ($n = 4$)	3 previous tokens	V^3	1.0×10^{12}	1.25×10^{14}

While the unconstrained state space exceeds the total number of atoms in the observable universe ($\approx 10^{80}$) by step $t = 20$, a bigram model maintains a manageable state space of 50,000 rows. However, as derived in Section 1.6, this drastic dimensional reduction comes at the heavy cost of complete amnesia for any context preceding the immediate $n - 1$ tokens.

1.3.2 Maximum likelihood estimation (MLE)

Under Maximum Likelihood Estimation, we estimate the conditional transition probabilities directly by counting occurrences of exact token sequences in a training corpus [5]. For a generic

n -gram:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-n+1}, \dots, w_{t-1}) = \frac{C(w_{t-n+1}, \dots, w_{t-1}, w_t)}{C(w_{t-n+1}, \dots, w_{t-1})}, \quad (1.7)$$

where $C(\cdot)$ denotes the raw frequency of that exact sequence in the training text. For a bigram model ($n = 2$), Equation (1.7) specializes to:

$$\mathbb{P}_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})} = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)}. \quad (1.8)$$

Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w \in \mathcal{V}} \mathbb{P}_{\text{MLE}}(w \mid w_{t-1}) = 1$.

1.4 Transition count matrix construction walkthrough

To understand exactly how statistical n -gram language models operate, we trace the step-by-step extraction of bigram frequencies and transition probabilities from the authoritative lecture corpus:

1. Sentence 1: *"The cat chased the mouse happily"*
2. Sentence 2: *"The mouse ate the cheese"*

Vocabulary extraction. Extracting all unique tokens (case-normalized with standard initial capitalization for clarity) yields a vocabulary of size $|\mathcal{V}| = 7$:

$$\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}.$$

Sequential bigram decomposition. We iterate through both training sentences token by token, recording each observed transition ($w_{t-1} \rightarrow w_t$):

- From Sentence 1 (*"The cat chased the mouse happily"*):
 1. (The, cat) $\implies C(\text{The, Cat}) \leftarrow 1$
 2. (cat, chased) $\implies C(\text{Cat, Chased}) \leftarrow 1$
 3. (chased, the) $\implies C(\text{Chased, The}) \leftarrow 1$
 4. (the, mouse) $\implies C(\text{The, Mouse}) \leftarrow 1$
 5. (mouse, happily) $\implies C(\text{Mouse, Happily}) \leftarrow 1$
- From Sentence 2 (*"The mouse ate the cheese"*):
 6. (The, mouse) $\implies C(\text{The, Mouse}) \leftarrow 2$
 7. (mouse, ate) $\implies C(\text{Mouse, Ate}) \leftarrow 1$
 8. (ate, the) $\implies C(\text{Ate, The}) \leftarrow 1$
 9. (the, cheese) $\implies C(\text{The, Cheese}) \leftarrow 1$

Unigram normalization counts. Summing across rows provides the total unigram occurrences $C(w_{t-1})$:

$$\begin{aligned}
 C(\text{The}) &= 4 && (\text{cat} : 1, \text{mouse} : 2, \text{cheese} : 1) \\
 C(\text{Cat}) &= 1 && (\text{chased} : 1) \\
 C(\text{Chased}) &= 1 && (\text{the} : 1) \\
 C(\text{Mouse}) &= 2 && (\text{happily} : 1, \text{ate} : 1) \\
 C(\text{Ate}) &= 1 && (\text{the} : 1) \\
 C(\text{Cheese}) &= 0 && (\text{terminal word, no outgoing transitions}) \\
 C(\text{Happily}) &= 0 && (\text{terminal word, no outgoing transitions})
 \end{aligned}$$

Full empirical transition probability matrix. Applying Equation (1.8) yields the complete 7×7 transition matrix $\mathbf{M} \in \mathbb{R}^{7 \times 7}$ where row i represents condition w_{t-1} and column j represents next token w_t . Table 1.2 presents both raw counts and normalized MLE probabilities.

Table 1.2: Empirical transition count matrix and normalized bigram conditional probabilities $\mathbb{P}(w_t | w_{t-1})$ for vocabulary $\mathcal{V} = \{\text{Ate, Cat, Chased, Cheese, Happily, Mouse, The}\}$.

Condition (w_{t-1})	Ate	Cat	Chased	Cheese	Happily	Mouse	The	Row Total
Ate	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cat	0 (0.0)	0 (0.0)	1 (1.00)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1
Chased	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	1 (1.00)	1
Cheese	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Happily	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0 (0.0)	0
Mouse	1 (0.50)	0 (0.0)	0 (0.0)	0 (0.0)	1 (0.50)	0 (0.0)	0 (0.0)	2
The	0 (0.0)	1 (0.25)	0 (0.0)	1 (0.25)	0 (0.0)	2 (0.50)	0 (0.0)	4

1.5 Autoregressive estimation and generation

With the transition matrix $\mathbf{M}$ constructed, the language model can execute two distinct operational tasks:

1. **Sequence Probability Estimation:** Assign a likelihood score to any proposed test sentence.
2. **Autoregressive Generation:** Incrementally sample tokens to produce novel synthetic text.

1.5.1 Worked sentence probability calculation

Consider evaluating the joint probability of the sentence:

“the cat chased the cheese”

Under the first-order Markov assumption, the joint probability factors as:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = \mathbb{P}(\text{the}) \cdot \mathbb{P}(\text{cat} | \text{the}) \cdot \mathbb{P}(\text{chased} | \text{cat}) \cdot \mathbb{P}(\text{the} | \text{chased}) \cdot \mathbb{P}(\text{cheese} | \text{the}). \quad (1.9)$$

Assuming our generation begins deterministically with the initial token $\mathbb{P}(\text{the}) = 1.0$, we substitute the corresponding conditional probabilities from Table 1.2:

$$\begin{aligned}\mathbb{P}(\text{cat} \mid \text{the}) &= \frac{1}{4} = 0.25 \\ \mathbb{P}(\text{chased} \mid \text{cat}) &= 1.0 \\ \mathbb{P}(\text{the} \mid \text{chased}) &= 1.0 \\ \mathbb{P}(\text{cheese} \mid \text{the}) &= \frac{1}{4} = 0.25\end{aligned}$$

Multiplying these values step by step:

$$\mathbb{P}(\text{the, cat, chased, the, cheese}) = 1.0 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16} = 0.0625. \quad (1.10)$$

Despite the sequence “*the cat chased the cheese*” never occurring as an entire sentence in the training corpus, the bigram model assigns it a strictly positive probability of 6.25%. By decomposing text into local constituent pairs, n -gram models exhibit rudimentary combinatorial generalization.

1.5.2 5-step autoregressive generation trace

Now consider using the bigram model for open-ended text generation. Autoregressive sampling begins with a seed prefix, samples the next token from the resulting conditional probability distribution, appends the sampled token to the history, and repeats.

We trace the exact 5-step generation sequence recorded in the lecture slides, starting from the seed token “*the*”:

Step 1: Condition on prefix: history = (“the”).

- Active row in transition matrix: Row “The”.
- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: “**mouse**”.
- Generated prefix: “*the mouse*”.

Step 2: Condition on prefix: history = ($\dots$, “mouse”).

- Active row in transition matrix: Row “Mouse”.
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: “**ate**”.
- Generated prefix: “*the mouse ate*”.

Step 3: Condition on prefix: history = ($\dots$, “ate”).

- Active row in transition matrix: Row “Ate”.
- Candidate distribution: $\mathbb{P}(\text{the} \mid \text{ate}) = 1.00$ (deterministic transition).
- Emitted token: “**the**”.
- Generated prefix: “*the mouse ate the*”.

Step 4: Condition on prefix: history = ($\dots$, “the”).

- Active row in transition matrix: Row “The”.

- Candidate distribution: $\mathbb{P}(\text{cat} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{cheese} \mid \text{the}) = 0.25$, $\mathbb{P}(\text{mouse} \mid \text{the}) = 0.50$.
- Random sample: **“mouse”**.
- Slide Note: 😞 (A repetitive structural loop has emerged because the model cannot remember that it already emitted “mouse” two words ago).
- Generated prefix: *“the mouse ate the mouse”*.

Step 5: Condition on prefix: history = ($\dots$, “mouse”).

- Active row in transition matrix: Row “Mouse”.
- Candidate distribution: $\mathbb{P}(\text{ate} \mid \text{mouse}) = 0.50$, $\mathbb{P}(\text{happily} \mid \text{mouse}) = 0.50$.
- Random sample: **“happily”**.
- Slide Note: 😱 (**shocked face**) (A syntactically valid but bizarre, cannibalistic sentence is generated).
- Generated sequence: *“The mouse ate the mouse happily”*.

Because the word “happily” has zero outgoing transitions in Table 1.2, the process reaches an absorbing terminal state and generation halts naturally.

Exam insights: Classroom Q&A: absorbing terminal states and infinite loops

During the lecture walkthrough, two crucial student questions were analyzed by the instructor:

1. Question 1: What happens after words like “happily” or “cheese”?

In Table 1.2, terminal sentence-ending words have a row count of zero ($C(\text{Happily}) = 0$), making the conditional probability division $0/0$ mathematically undefined. In standard production language modeling, this is formalized by appending an explicit end-of-sequence token $\langle /s \rangle$ (or [EOS]) to every training sentence, guaranteeing that every word possesses a non-zero outgoing transition to either another word or $\langle /s \rangle$.

2. Question 2: Why did greedy decoding enter an infinite cyclic loop?

Because bigram models condition exclusively on the single preceding token w_{t-1} , once the model transitions from “ate” to “the”, it retains zero memory that it already described the mouse eating cheese two steps earlier. If greedy decoding is used ($\arg \max$), the transition path becomes fully deterministic and falls into a stationary limit cycle: *“the $\rightarrow$ mouse $\rightarrow$ ate $\rightarrow$ the $\rightarrow$ mouse $\rightarrow \dots$ ”*. Modern LLMs resolve this through long-context self-attention horizons, temperature-controlled stochastic sampling, and repetition penalties.

1.6 Fundamental limitations of statistical n -gram models

While n -gram models provided the foundation for speech recognition, statistical machine translation, and spelling correction throughout the 1980s and 1990s, they suffer from three fatal structural limitations that ultimately motivated the deep learning revolution.

1.6.1 1. Catastrophic data sparsity (V^n parameter explosion)

To condition on $n - 1$ words, an n -gram model requires storing a discrete probability table of dimensionality $|V|^n$. For realistic natural language applications:

- Vocabulary size: $|\mathcal{V}| \approx 50,000$ to $100,000$ unique words.
- Bigram table ($n = 2$): $|\mathcal{V}|^2 = 50,000^2 = 2.5 \times 10^9$ parameters (approx. 10 GB at 32-bit floating point).
- Trigram table ($n = 3$): $|\mathcal{V}|^3 = 50,000^3 = 1.25 \times 10^{14}$ parameters (over 500 Terabytes).
- 4-gram table ($n = 4$): $|\mathcal{V}|^4 = 50,000^4 = 6.25 \times 10^{18}$ parameters (Exabyte scale).

Even if storage were free, the number of distinct 4-grams in all digitized human literature is dwarfed by 10^{18} . Consequently, the overwhelming majority of valid phrases never appear in training corpora. In standard MLE, any unobserved n -gram receives count $C = 0$, yielding $\mathbb{P}(w_t | w_{<t}) = 0$. By Equation (1.2), a single zero-count factor collapses the joint probability of an entire document to zero:

$$\mathbb{P}(\text{unseen word} | \text{context}) = 0 \implies \mathbb{P}(\text{Entire Book}) = 0.$$

While smoothing techniques (such as Laplace add-one, Good-Turing discounting, and Kneser-Ney backoff) alleviate zero probabilities, they reallocate probability mass heuristically without addressing the underlying representational pathology.

1.6.2 2. Finite context horizon limitations

An n -gram model is strictly blind to any token that occurred more than $n - 1$ steps in the past:

$$\mathbb{P}(w_t | w_1, w_2, \dots, w_{t-1}) \approx \mathbb{P}(w_t | w_{t-n+1}, \dots, w_{t-1}). \quad (1.11)$$

Because the conditioning window is rigidly restricted to the preceding $n - 1$ tokens, any semantic constraint, topic coherence, or syntactic agreement established earlier in the discourse is discarded. This finite context horizon causes severe memory amnesia across two critical domains:

Linguistic agreement breakdown in natural language. Consider long-range syntactic agreements where an arbitrary number of modifiers, prepositional phrases, or subordinate relative clauses separate the grammatical subject from its governing predicate:

*“The **engineers** who designed the complex distributed compiler infrastructure [...] **were** vs **was** awarded the patent.”*

If $n = 4$ (a 4-gram model), the model predicts the verb by conditioning exclusively on the three preceding tokens:

$$\mathbb{P}(\text{verb} | \text{compiler, infrastructure, [...]}).$$

Because the singular noun “*infrastructure*” is immediately adjacent to the verb position, the local 4-gram transition statistics strongly favor the singular verb “*was*”. The model has completely forgotten the true plural antecedent subject “*engineers*”, producing an ungrammatical agreement violation.

Structural and scoping breakdown in software engineering code. In programming languages, code validity requires strict, long-range determinism governed by context-free grammars and scope resolution rules:

1. **Lexical Scoping and Variable Lifecycles:** A variable declared in a function header:

```
void process_batch(std::vector<TransactionRequest>& transactions) { ...}
```

must be invoked using valid `std::vector` member methods (e.g., `.push_back()`, `.size()`) thirty lines later. An n -gram model sees only transactions. without access to the declaration, leading to syntactically invalid completions.

2. **Bracket and Block Matching:** Program code requires exact hierarchical pairing of opening and closing delimiters (`{ ...}`, `[ ...]`, `( ...)`) and control flow constructs (`try ...catch ...finally`). Because n -gram models lack internal state stacks, they frequently emit mismatched braces or premature block terminations.

These structural failures illustrate why classical statistical counting models could never serve as reliable code completion or software synthesis engines, necessitating neural architectures capable of tracking long-range relational dependencies.

1.6.3 3. Lack of semantic generalization

In classical statistical NLP, words are treated as discrete, orthogonal atomic symbols. If the model observes:

*“The **cat** chased the mouse happily”*

it gains zero statistical confidence regarding the identical sentence containing synonyms:

*“The **feline** chased the mouse happily”*

Because “*cat*” and “*feline*” are completely separate row indices in Table 1.2, observations of one do not inform the other. There is no concept of semantic distance, synonymy, or conceptual overlap.

1.7 Evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [4, 5].

1.7.1 Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

Definition 1.4: Shannon entropy

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution P [4]:

$$H(P) = - \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathbb{E}_{x \sim P} \left[\log_2 \frac{1}{P(x)} \right]. \quad (1.12)$$

By Shannon’s source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by P without loss.

When constructing a language model, we approximate the true distribution P with an estimated parameterized distribution Q_θ . To quantify the statistical discrepancy between the

true distribution P and our approximating model Q_θ , we employ the *Kullback-Leibler (KL) divergence*.

Definition 1.5: *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution Q to target distribution P is:

$$D_{\text{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) = \mathbb{E}_{x \sim P} \left[\log_2 \frac{P(x)}{Q(x)} \right]. \quad (1.13)$$

Theorem 1.1: *Gibbs' inequality*

For any two discrete probability distributions P and Q , the KL divergence is non-negative:

$$D_{\text{KL}}(P \parallel Q) \geq 0, \quad (1.14)$$

with equality $D_{\text{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

Proof. Applying the natural logarithm and utilizing the standard inequality $\ln(u) \leq u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\text{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned}$$

Multiplying both sides by -1 reverses the inequality, yielding $D_{\text{KL}}(P \parallel Q) \geq 0$. ■

1.7.2 Cross-entropy decomposition

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\text{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left(- \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \quad (1.15)$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\text{KL}}(P \parallel Q). \quad (1.16)$$

Why optimizing cross-entropy optimizes KL divergence. Equation (1.16) reveals the theoretical justification for cross-entropy training:

- The true language distribution P is determined by human linguistic behavior; therefore, the true entropy $H(P)$ is a fixed, immutable physical constant.
- When we minimize cross-entropy with respect to model parameters θ :

$$\arg \min_{\theta} H(P, Q_{\theta}) = \arg \min_{\theta} (H(P) + D_{\text{KL}}(P \parallel Q_{\theta})) = \arg \min_{\theta} D_{\text{KL}}(P \parallel Q_{\theta}). \quad (1.17)$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model Q_{θ} and the true language distribution P to zero.

1.7.3 Empirical sequence cross-entropy

Because we do not have closed-form access to the true distribution $P(x)$, we cannot compute the expectation $\mathbb{E}_{x \sim P}[\cdot]$ directly. Instead, by the Shannon-McMillan-Breiman theorem (the ergodic theorem for stationary information sources), the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \dots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \dots, w_T) = -\frac{1}{T} \sum_{t=1}^T \log_2 Q(w_t \mid w_{<t}). \quad (1.18)$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using our model Q .

1.7.4 Perplexity (PPL)

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners overwhelmingly report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\text{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T} \sum_{t=1}^T \log_2 Q(w_t \mid w_{<t})} \quad (1.19)$$

$$= \left(\prod_{t=1}^T \frac{1}{Q(w_t \mid w_{<t})} \right)^{\frac{1}{T}}. \quad (1.20)$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

1.7.5 Rigorous proof: perplexity as the effective branching factor

The fundamental intuition taught across NLP literature is that perplexity quantifies the *effective branching factor* of a language model. We now formalize this property through a rigorous proof.

Theorem 1.2: *Perplexity of a uniform K -ary distribution*

Let a language model Q predict next tokens such that at each timestep t , the probability mass is distributed uniformly across exactly K equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence W of length T is identically K :

$$\text{PPL}(W) = K. \quad (1.21)$$

Proof. Substituting the uniform prediction probability $Q(w_t | w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula:

$$\begin{aligned}\mathcal{H}(W) &= -\frac{1}{T} \sum_{t=1}^T \log_2 \left(\frac{1}{K} \right) = -\frac{1}{T} \sum_{t=1}^T (-\log_2 K) \\ &= \frac{1}{T} (T \log_2 K) = \log_2 K.\end{aligned}$$

Exponentiating the cross-entropy:

$$\text{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \quad \blacksquare$$

Physical interpretation. When a model evaluates a text corpus and achieves $\text{PPL} = 42$, it indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words:

- A uniform random guesser across vocabulary $\mathcal{V}$ achieves $\text{PPL} = |\mathcal{V}|$ (e.g., $\text{PPL} = 50,000$).
- An optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound $\text{PPL} = 1.0$.
- State-of-the-art LLMs on natural language benchmarks typically achieve perplexities between 8 and 15.

Exam insights: Base-2 vs base- e logarithmic conversions and worked exam example

A recurrent point of confusion in oral and written examinations at Politecnico di Torino is mixing base-2 information theory with base- e deep learning frameworks:

Property	Information Theory (Bits)	Deep Learning / PyTorch (Nats)
Loss Formula	$\mathcal{H}_2 = -\frac{1}{T} \sum \log_2 Q(w_t w_{<t})$	$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum \ln Q(w_t w_{<t})$
Unit	Bits per token	Nats per token
Perplexity Formula	$\text{PPL} = 2^{\mathcal{H}_2}$	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$
Conversion Identity	$\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$	$\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$

Worked exam calculation. Suppose a PyTorch model trained on code completion reports an evaluation loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats.

1. Direct Base- e Perplexity:

$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00.$$

2. Conversion to Information Bits:

$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token.}$$

3. Base-2 Perplexity Verification:

$$\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00.$$

Common Exam Error: If a student incorrectly evaluates $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$, they underestimate the model's true branching factor by more than 57%. Always verify whether the logarithm used is base-2 or base- e before computing perplexity.

1.8 Historical evolution of language models

Understanding modern large language models requires appreciating the 75-year evolutionary trajectory that progressively dismantled the mathematical and computational barriers of discrete symbol counting. Language modeling has evolved not merely through increases in compute or dataset scale, but through five profound conceptual paradigm shifts:

1. **From discrete orthogonal symbols to continuous geometric embeddings:** Transitioning from sparse, mutually orthogonal representations in $|\mathcal{V}|$ -dimensional space to dense vectors in $\mathbb{R}^d$ ($d \ll |\mathcal{V}|$) where topological and geometric distance encodes syntactic and semantic similarity.
2. **From rigid Markov horizons to flexible recurrent state memories:** Replacing strict $(n - 1)$ -token context cutoffs with recurrent hidden states capable of maintaining dynamic sequential memory across variable-length histories.
3. **From sequential recurrence to parallelized self-attention:** Eliminating step-by-step sequential bottlenecks in favor of pairwise all-to-all attention mechanisms that compress the maximum interaction path length between any two tokens in a sequence to $O(1)$.
4. **From task-specific architectures to universal pre-trained foundation models:** Shifting from designing specialized models trained from scratch for each separate task to universal autoregressive base models capable of general-purpose in-context learning.
5. **From passive token predictors to aligned, agentic reasoning systems:** Augmenting next-token predictors with post-training alignment, test-time reasoning scaffolding, and interactive tool-use execution loops for complex software engineering tasks.

1.8.1 Chronological roadmap of language modeling

Figure 1.2 synthesizes the macro-historical progression of language modeling from Claude Shannon’s foundational 1948 information theory to modern autonomous software engineering agents.

1.8.2 1948–1980s: statistical information theory and Markov n-grams

The conceptual origin of language modeling stems from Claude Shannon’s 1948 landmark paper “*A Mathematical Theory of Communication*” [4]. Shannon treated written human language as a discrete stochastic process generated by an underlying stationary Markov source over a finite alphabet.

Shannon introduced the *Shannon game*—a pedagogical thought experiment where a human or an automated statistical predictor attempts to guess the next letter in an unknown sequence of text given the preceding letters. Shannon demonstrated that as the conditioning context length increases from zero-order (independent uniform character selection) to unigram (individual character frequencies), bigram (pairs), and trigram (triplets) models, the generated text rapidly converges toward natural English orthography and grammar.

In this framework, the conditional probability of token w_t given context $w_1, \dots, w_{t-1}$ is simplified via an order- $(n - 1)$ Markov assumption:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}^{t-1}) = \frac{C(w_{t-n+1}^t)}{C(w_{t-n+1}^{t-1})}, \quad (1.22)$$

where $C(\cdot)$ denotes the empirical frequency count observed in a training corpus.

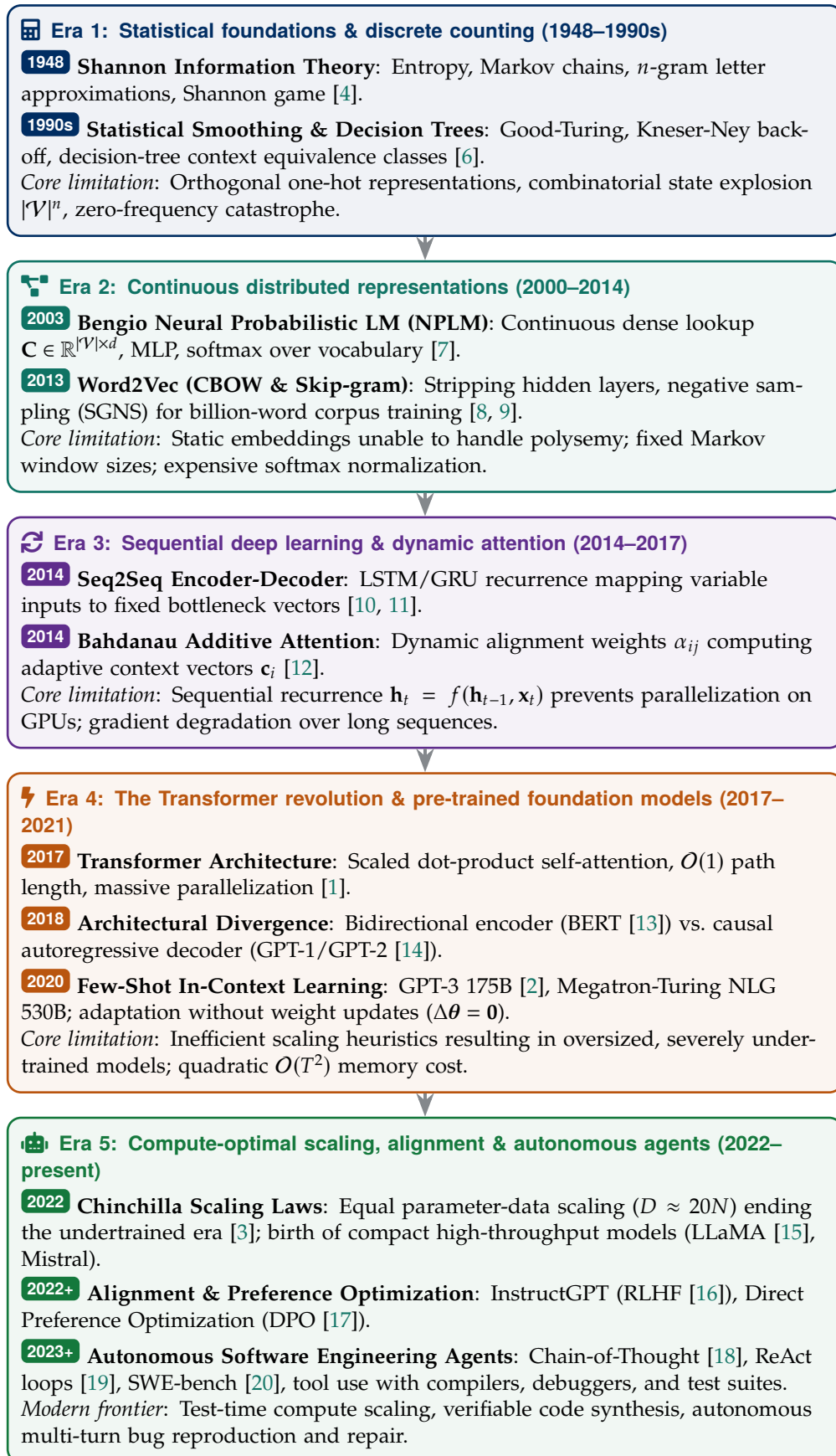


Figure 1.2: Chronological timeline of language model evolution from 1948 to the present.

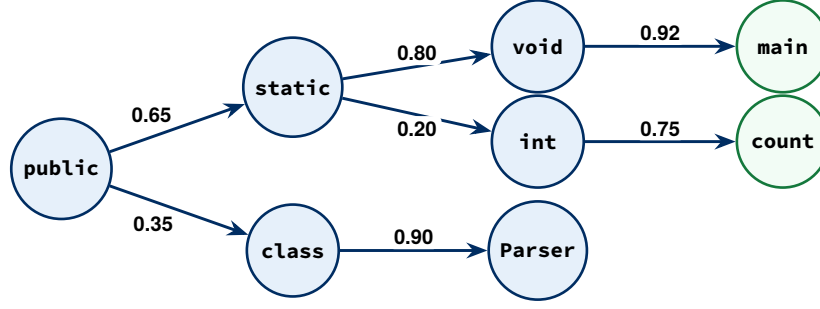


Figure 1.3: Markov chain state transition graph for software tokens.

Mathematical and operational bottlenecks. While mathematically transparent and computationally trivial to estimate via hash tables, statistical n -gram models suffer from catastrophic structural limitations:

1. **Combinatorial State Explosion:** The number of possible transition parameters scales exponentially as $|\mathcal{V}|^n$. For a modest software vocabulary of $|\mathcal{V}| = 50,000$ tokens, a 4-gram model entails $|\mathcal{V}|^4 = 6.25 \times 10^{18}$ potential state combinations, rendering full parameter estimation mathematically intractable.
2. **The Zero-Frequency Catastrophe:** If a valid code sequence (e.g., `public static void run()`) never appeared verbatim in the training set, $C(w_{t-n+1}^t) = 0 \implies P(w_t | w_{t-n+1}^{t-1}) = 0$. When evaluating test corpora, a single zero probability assigns infinite perplexity ($\text{PPL} = \infty$) to the entire document.
3. **Orthogonality of Surface Symbols:** In discrete token space, the tokens `int`, `long`, and `short` are represented as completely independent, orthogonal basis vectors. The statistical model possesses zero capacity to transfer learned syntactic patterns (such as `int x = 0;`) to semantically related tokens (`long x = 0;`).

1.8.3 1990s: statistical smoothing and decision-tree language models

Throughout the 1990s, statistical language modeling was extensively refined to address the zero-frequency barrier and structural sparsity. Researchers developed two primary paradigms: statistical smoothing (interpolated backoff) and decision-tree context clustering.

Smoothing and discounting. Rather than assigning zero probability to unseen sequences, smoothing algorithms reserve a fraction of the probability mass from frequent events and redistribute it to unseen combinations. The pinnacle of this era was *Modified Kneser-Ney smoothing*, which combines absolute discounting with a novel *continuation probability*:

$$P_{\text{KN}}(w_t | w_{t-n+1}^{t-1}) = \frac{\max(C(w_{t-n+1}^t) - d, 0)}{C(w_{t-n+1}^{t-1})} + \lambda(w_{t-n+1}^{t-1})P_{\text{KN}}(w_t | w_{t-n+2}^{t-1}), \quad (1.23)$$

where $d \in (0, 1)$ is an absolute discount parameter, $\lambda(w_{t-n+1}^{t-1}) = \frac{d}{C(w_{t-n+1}^{t-1})} \cdot |\{w : C(w_{t-n+1}^{t-1}, w) > 0\}|$ is a normalizing interpolation weight, and the lower-order continuation probability measures how versatile word w_t is across diverse historical contexts:

$$P_{\text{continuation}}(w_t) = \frac{|\{w_{t-1} : C(w_{t-1}, w_t) > 0\}|}{\sum_{w'} |\{w_{t-1} : C(w_{t-1}, w') > 0\}|}. \quad (1.24)$$

Decision-tree language models. In parallel, Potamianos and Jelinek (1998) [6] introduced decision-tree language models to overcome the rigid Markov window. Rather than conditioning probability on an exact sequence of $n - 1$ tokens, a binary decision tree asks a sequence of syntactic and lexical questions about the historical context $\mathcal{H} = (w_{t-1}, w_{t-2}, \dots)$.

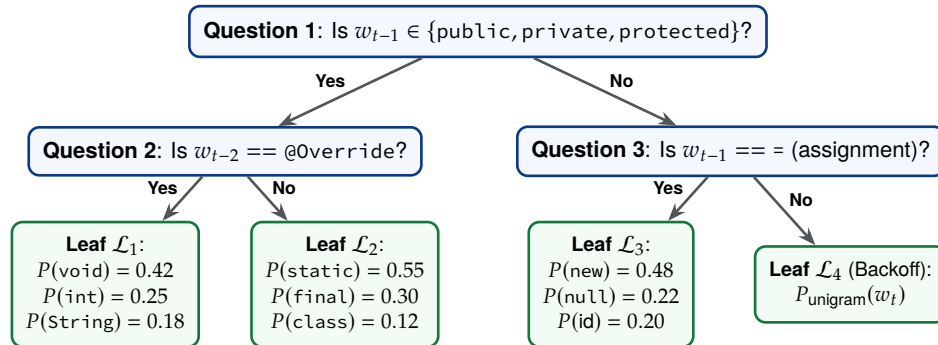


Figure 1.4: Decision tree context partitioning for conditional language modeling.

As illustrated in Figure 1.4, context histories that share identical predictive characteristics are clustered into the same leaf node $\mathcal{L}_k$, yielding conditional distributions $P(w_t | \mathcal{L}_k)$. While this reduced parameter count compared to flat tables, the model remained fundamentally bound to discrete categorical splitting without true semantic interpolation.

1.8.4 2000–2003: Bengio’s neural probabilistic language model

In their seminal 2003 paper “A Neural Probabilistic Language Model”, Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin [7] permanently transformed NLP by introducing continuous distributed representations to defeat the curse of dimensionality.

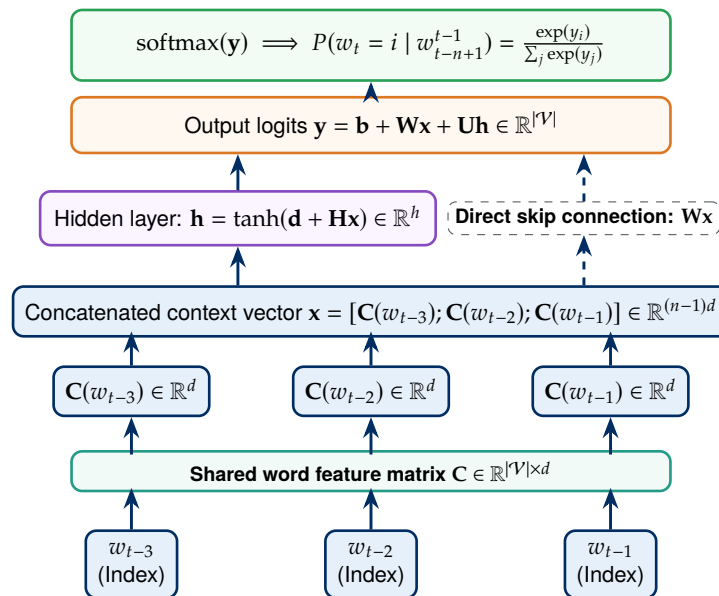


Figure 1.5: Bengio neural probabilistic language model architecture.

Architecture and mathematical formulation. As detailed in Figure 1.5, Bengio et al. decoupled representation learning from probability estimation across four stages:

1. **Continuous Embedding Matrix:** Each word $w \in \mathcal{V}$ is mapped to a continuous column

vector $\mathbf{C}(w) \in \mathbb{R}^d$ via a shared embedding matrix $\mathbf{C} \in \mathbb{R}^{|\mathcal{V}| \times d}$, with $d \ll |\mathcal{V}|$ (typically $d \in [30, 100]$ in 2003).

2. **Context Vector Concatenation:** The embeddings of the preceding $n - 1$ words are concatenated into a joint context vector:

$$\mathbf{x} = [\mathbf{C}(w_{t-n+1}); \mathbf{C}(w_{t-n+2}); \dots; \mathbf{C}(w_{t-1})] \in \mathbb{R}^{(n-1)d}. \quad (1.25)$$

3. **Non-linear Hidden Layer with Direct Skip Connections:** Context vector $\mathbf{x}$ is passed through a hyperbolic tangent hidden layer $\mathbf{h} = \tanh(\mathbf{d} + \mathbf{H}\mathbf{x})$, where $\mathbf{H} \in \mathbb{R}^{h \times (n-1)d}$ and $\mathbf{d} \in \mathbb{R}^h$. The network also incorporates an optional linear direct connection from input to output:

$$\mathbf{y} = \mathbf{b} + \mathbf{W}\mathbf{x} + \mathbf{U}\mathbf{h}, \quad (1.26)$$

where $\mathbf{U} \in \mathbb{R}^{|\mathcal{V}| \times h}$ projects the hidden state to vocabulary logits, $\mathbf{W} \in \mathbb{R}^{|\mathcal{V}| \times (n-1)d}$ is the direct skip matrix, and $\mathbf{b} \in \mathbb{R}^{|\mathcal{V}|}$ is the output bias.

4. **Softmax Multi-Class Output Head:** Unnormalized logits $\mathbf{y}$ are normalized into a proper conditional probability distribution over the vocabulary:

$$P(w_t = i \mid w_{t-n+1}^{t-1}) = \frac{\exp(y_i)}{\sum_{j=1}^{|\mathcal{V}|} \exp(y_j)}. \quad (1.27)$$

Smooth generalization and the curse of dimensionality. Because the function mapping $\mathbf{x} \mapsto P(w_t \mid \mathbf{x})$ is continuous and differentiable, words that occupy neighboring positions in $\mathbb{R}^d$ (e.g., $\mathbf{C}(\text{cat}) \approx \mathbf{C}(\text{feline})$, or $\mathbf{C}(\text{int}) \approx \mathbf{C}(\text{short})$) produce virtually identical activation vectors $\mathbf{h}$ and logit outputs $\mathbf{y}$. Consequently, seeing a sentence like “the feline climbed the tree” during training automatically increases the probability of “the cat climbed the tree”, even if the latter sequence never appeared in the corpus.

Computational bottleneck. The fatal computational bottleneck of Bengio’s NPLM lay in the output projection matrix $\mathbf{U} \in \mathbb{R}^{|\mathcal{V}| \times h}$ and the denominator of the softmax. For a vocabulary $|\mathcal{V}| = 100,000$ and hidden size $h = 500$, computing the output layer required 5×10^7 floating-point operations *per single token step*. On early-2000s CPU hardware, training on modest corpora took weeks, limiting practical applications.

External deep dive: From Shannon to Bengio: Karpathy’s Makemore perspective

In his pedagogical series *Makemore*, Andrej Karpathy demonstrates that a bigram language model can be implemented in two distinct ways that perfectly illuminate the shift from statistical counting to neural estimation:

1. **The Classical Counting Approach:** Iterate over all adjacent character pairs (w_{t-1}, w_t) in a training corpus, populate a 27×27 frequency matrix $\mathbf{C}$, normalize each row by its row-sum to obtain transition probability matrix $\mathbf{M}_{ij} = \frac{C_{ij}}{\sum_k C_{ik}}$, and generate tokens using categorical sampling: $w_t \sim \text{Categorical}(\mathbf{M}_{w_{t-1}, :})$.
2. **The Gradient-Based Neural Approach:** Encode token w_{t-1} as a one-hot vector $\mathbf{x} \in \{0, 1\}^{27}$, pass it through a single linear layer parameterized by weight matrix $\mathbf{W} \in \mathbb{R}^{27 \times 27}$ to compute logits $\mathbf{z} = \mathbf{x}\mathbf{W}$, evaluate probabilities via softmax $\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$,

and minimize the negative log-likelihood cross-entropy loss:

$$\mathcal{L}(\mathbf{W}) = - \sum_{t=1}^T \log \hat{y}_{t,\text{target}}. \quad (1.28)$$

The Unifying Mathematical Duality: When optimized to theoretical convergence via gradient descent without weight regularization, the weights $\mathbf{W}$ of the single-layer neural network converge exactly to the natural logarithm of the normalized statistical counts:

$$\lim_{t \rightarrow \infty} \mathbf{W}_{ij} = \ln \mathbf{M}_{ij} + C_i, \quad (1.29)$$

where C_i is an arbitrary row-wise constant that cancels out during softmax normalization. Neural networks do not replace statistical count estimation; rather, they generalize count optimization into continuous, differentiable parameter spaces capable of hierarchical non-linear composition.

1.8.5 2013–2014: efficient distributed representations and Word2Vec

A decade after Bengio’s NPLM, Tomas Mikolov and colleagues at Google (2013) [8, 9] identified that the primary impediment to scaling neural representations was the non-linear hidden layer. By discarding the tanh layer entirely, Mikolov et al. demonstrated that high-quality distributed representations could be trained across billions of tokens in hours on commodity hardware, introducing the *Word2Vec* framework.

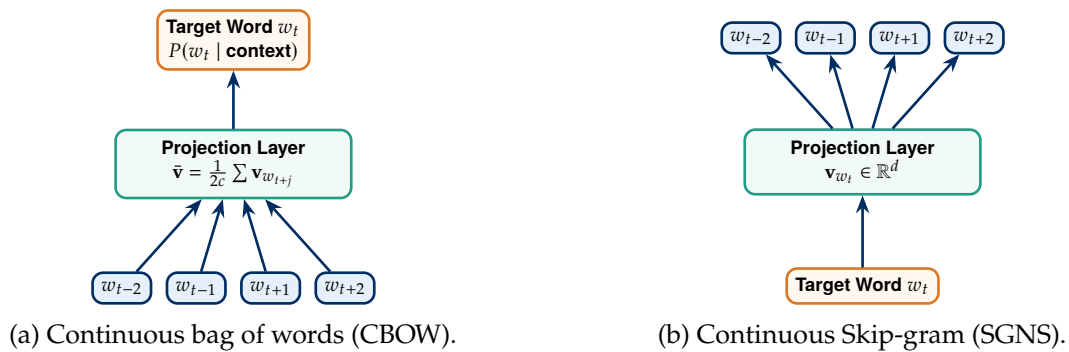


Figure 1.6: Continuous bag of words (CBOW) versus Skip-gram architectures in Word2Vec.

Word2Vec introduced two complementary architectures, shown in Figure 1.6:

1. **Continuous Bag of Words (CBOW):** Predicts the target token w_t conditioned on surrounding context words within a symmetric window $[-c, +c]$. The context embeddings are simply averaged, discarding word order:

$$\bar{\mathbf{v}} = \frac{1}{2c} \sum_{-c \leq j \leq c, j \neq 0} \mathbf{v}_{w_{t+j}}. \quad (1.30)$$

2. **Continuous Skip-Gram:** Inverts the CBOW objective by using the target token w_t to predict surrounding context tokens w_{t+j} within $[-c, +c]$.

Negative sampling (SGNS). To bypass the computationally intractable softmax denominator over $|\mathcal{V}|$, Mikolov et al. formulated *Skip-Gram with Negative Sampling* (SGNS). SGNS reframes

language modeling from a multi-class probability task into a set of binary logistic regressions: distinguishing true observed co-occurrences (w_t, c) from K randomly sampled noise pairs $(w_t, w_{n,k})$:

$$\mathcal{L}_{\text{SGNS}}(\mathbf{v}_w, \mathbf{v}'_c) = \log \sigma(\mathbf{v}'_c^\top \mathbf{v}_w) + \sum_{k=1}^K \mathbb{E}_{w_{n,k} \sim P_n} \left[\log \sigma(-\mathbf{v}'_{w_{n,k}}^\top \mathbf{v}_w) \right], \quad (1.31)$$

where $\mathbf{v}_w \in \mathbb{R}^d$ is the center word vector, $\mathbf{v}'_c \in \mathbb{R}^d$ is the context vector, $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid activation, and $P_n(w) \propto U(w)^{3/4}$ is a smoothed unigram noise distribution that slightly elevates the sampling frequency of rare words relative to stopwords.

Emergence of linear semantic vector arithmetic. SGNS produced clean vector spaces where linear geometric displacement corresponds to semantic and syntactic relationships:

$$\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} + \mathbf{v}_{\text{Woman}} \approx \mathbf{v}_{\text{Queen}}. \quad (1.32)$$

In software engineering, similar vector arithmetic emerged over AST tokens and code identifiers:

$$\mathbf{v}_{\text{ArrayList}} - \mathbf{v}_{\text{Java}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{list}}. \quad (1.33)$$

Bottlenecks. Despite its brilliance, Word2Vec had critical deficiencies:

1. **Static, Context-Independent Embeddings (Polysemy Failure):** Each token w receives exactly one fixed vector $\mathbf{v}_w \in \mathbb{R}^d$. In natural language, “*bank*” (financial institution vs. river bank) cannot be disambiguated. In software engineering, the keyword `class` possesses identical static vectors regardless of whether it defines a Java object, a CSS style selector, or a C++ template parameter.
2. **Lack of Autoregressive Generative Capability:** Word2Vec produces static vector spaces for downstream classifiers; it cannot generate sequences or predict complex multi-token programs.

1.8.6 2014–2017: recurrent sequence models and Seq2Seq attention

To model variable-length sequences without rigid Markov window cutoffs, the field adopted Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTM, Hochreiter and Schmidhuber 1997 [21]), and Gated Recurrent Units (GRU, Cho et al. 2014 [11]).

Recurrent hidden states. In an autoregressive RNN language model, the conditioning history is compressed into a dynamic continuous hidden state vector $\mathbf{h}_t \in \mathbb{R}^h$ updated sequentially at each token step:

$$\mathbf{h}_t = f(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h), \quad (1.34)$$

where $\mathbf{x}_t = \mathbf{C}(w_t) \in \mathbb{R}^d$ is the input word vector, $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$ is the recurrent transition matrix, $\mathbf{W}_{xh} \in \mathbb{R}^{h \times d}$ is the input projection, and f is a gating activation (such as the LSTM forget/input/output cell update). The next-token distribution is evaluated from $\mathbf{h}_t$:

$$P(w_{t+1} | w_1, \dots, w_t) = \text{softmax}(\mathbf{W}_{\text{out}}\mathbf{h}_t + \mathbf{b}_{\text{out}}). \quad (1.35)$$

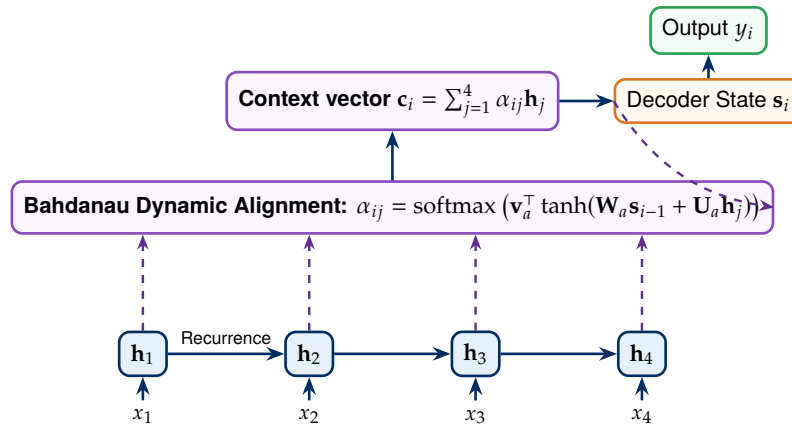


Figure 1.7: Unrolled recurrent sequence model with dynamic Bahdanau attention alignment.

Sequence-to-sequence and the Bahdanau attention breakthrough. For conditional sequence tasks (e.g., Code Summarization or Neural Machine Translation), Sutskever et al. (2014) [10] introduced the *Sequence-to-Sequence (Seq2Seq)* encoder-decoder framework. However, compressing an entire source code method containing hundreds of tokens into a single static bottleneck vector $\mathbf{h}_T$ caused performance to degrade rapidly for sequences longer than 20 tokens.

To resolve this bottleneck, Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio (2014) [12] invented *Additive Dynamic Attention*, illustrated in Figure 1.7. Rather than compressing the source into a fixed vector, the decoder computes a dynamic alignment score e_{ij} comparing current decoder state $\mathbf{s}_{i-1}$ with each encoder hidden state $\mathbf{h}_j$:

$$e_{ij} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j), \quad (1.36)$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad (1.37)$$

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j. \quad (1.38)$$

The dynamic context vector $\mathbf{c}_i$ allows the model to look directly back at the relevant parts of the input program, establishing the foundation of modern attention mechanisms.

The sequential computational barrier. Despite solving the informational bottleneck, recurrent models hit a hard hardware execution barrier: **sequential temporal dependency**. Because $\mathbf{h}_t$ cannot be computed until $\mathbf{h}_{t-1}$ is evaluated, training cannot be parallelized across the time dimension T . GPUs—which excel at massive parallel matrix operations—are forced into sequential execution during RNN training. Furthermore, gradients backpropagating through time (BPTT) across hundreds of steps suffer from exponential dissipation or explosion, rendering modeling across repository-scale context windows impossible.

1.8.7 2017: the Transformer revolution and self-attention

In June 2017, Ashish Vaswani and a team at Google Brain published “*Attention Is All You Need*” [1]. They introduced the **Transformer** architecture by completely eliminating recurrence and convolutions, relying entirely on self-attention mechanisms.

Mathematical mechanics of scaled dot-product attention. As shown in Figure 1.8a, given an input sequence representation $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, linear projections produce Queries (**Q**), Keys (**K**),

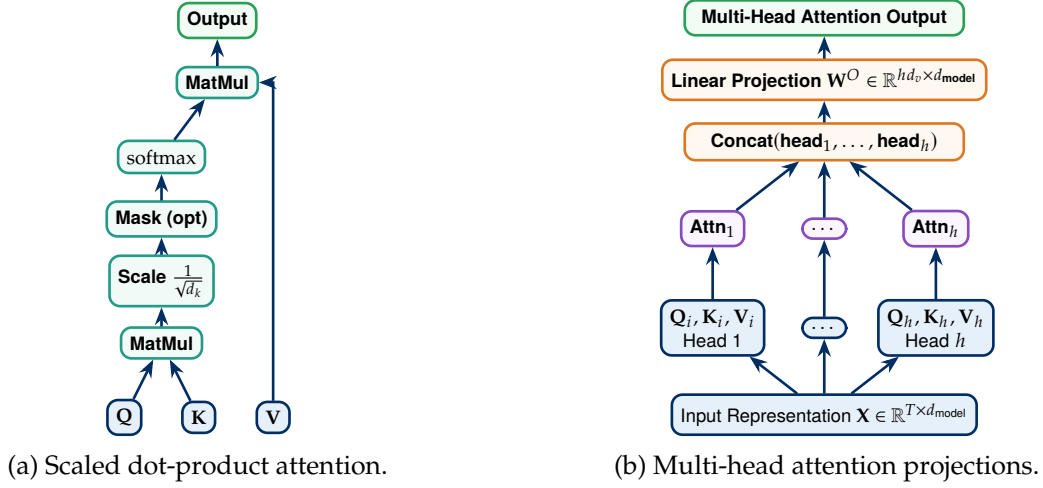


Figure 1.8: Scaled dot-product attention and multi-head attention mechanisms.

and Values ($\mathbf{V}$):

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V, \quad (1.39)$$

where $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. The attention map is computed across all token pairs simultaneously via matrix multiplication:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V}, \quad (1.40)$$

where $\frac{1}{\sqrt{d_k}}$ prevents vanishing gradients in the softmax for large inner products, and $\mathbf{M} \in \{0, -\infty\}^{T \times T}$ is an optional attention mask.

Multi-head attention (MHA). To allow the model to jointly attend to information from different representation subspaces at different positions (e.g., one head tracking variable definitions, another tracking function calls, and a third tracking syntax parentheses), Multi-Head Attention linearly projects queries, keys, and values h times in parallel:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O, \quad (1.41)$$

$$\text{where } \text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V). \quad (1.42)$$

Why Transformers conquered NLP and software engineering.

1. **$O(1)$ Path Length:** In an RNN, signal from token 1 must pass through T recurrent multiplications to reach token T . In a Transformer, every token attends directly to every other token in exactly $O(1)$ operations, eliminating vanishing gradient dissipation.
2. **Total Hardware Parallelization:** Because there are no sequential temporal dependencies across sequence length T during training, the entire attention matrix $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{T \times T}$ is evaluated as a single dense tensor contraction on GPU tensor cores.
3. **Trade-off:** Memory and computational complexity scale quadratically with sequence length: $O(T^2 \cdot d_{\text{model}})$, which motivated modern KV-caching, FlashAttention, and sparse context optimizations.

1.8.8 2018–2019: pre-trained foundation models and architectural divergence

With the Transformer established as a highly scalable backbone, NLP shifted from training task-specific models with static Word2Vec embeddings to the **pre-train then fine-tune** paradigm. Models were pre-trained on massive unlabeled web corpora using self-supervised objectives, then fine-tuned with task-specific heads.

This era witnessed a major architectural divergence into two primary branches, illustrated in Figure 1.9:

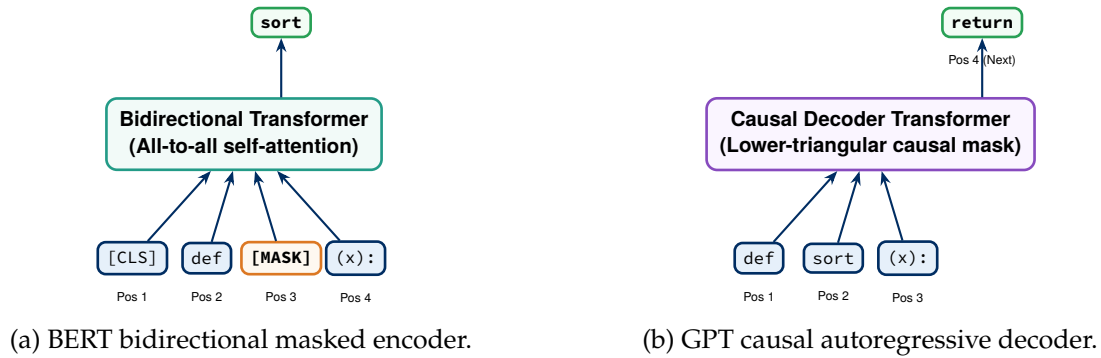


Figure 1.9: Bidirectional masked representation (BERT) versus causal autoregressive generation (GPT).

1. Bidirectional Encoders (BERT, Devlin et al. 2019 [13]):

- *Mechanism:* Tokens attend bidirectionally to all past and future positions in the sequence. Pre-trained via **Masked Language Modeling (MLM)**, where 15% of tokens are replaced with a special [MASK] token, and the network predicts the masked words:

$$\mathcal{L}_{\text{MLM}} = - \sum_{i \in \mathcal{M}} \log P(w_i | \tilde{\mathbf{w}}). \quad (1.43)$$

- *Software Engineering Dominance:* Bidirectional encoders excel at representation and discriminative tasks, powering tools like CodeBERT [22] for vulnerability detection, code search, defect prediction, and clone detection.
- *Limitation:* BERT cannot naturally generate variable-length code completions because bidirectional attention creates circular dependencies during sequential left-to-right decoding.

2. Autoregressive Decoders (GPT-1 & GPT-2, Radford et al. 2019 [14]):

- *Mechanism:* Retains strictly causal (unidirectional) attention by enforcing a lower-triangular causal attention mask:

$$M_{ij} = \begin{cases} 0 & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases} \quad (1.44)$$

The model optimizes the standard autoregressive language modeling objective:

$$\mathcal{L}_{\text{AR}} = - \sum_{t=1}^T \log P(w_t | w_1, \dots, w_{t-1}). \quad (1.45)$$

- *Radford’s Core Insight (Unsupervised Multitask Learners)*: Radford et al. demonstrated that scaling decoder-only models trained on massive, diverse web text (WebText) produced zero-shot multitask capabilities without task-specific architectural heads or fine-tuning. The decoder-only architecture became the undisputed foundation for generative AI.

1.8.9 2020–2021: extreme scale and few-shot in-context learning

In 2020, Tom Brown and colleagues at OpenAI published “*Language Models are Few-Shot Learners*” [2], introducing GPT-3 (175 billion parameters). GPT-3 revealed that scaling autoregressive decoders by an order of magnitude unlocked qualitative emergent behaviors, most notably **In-Context Learning (ICL)**.

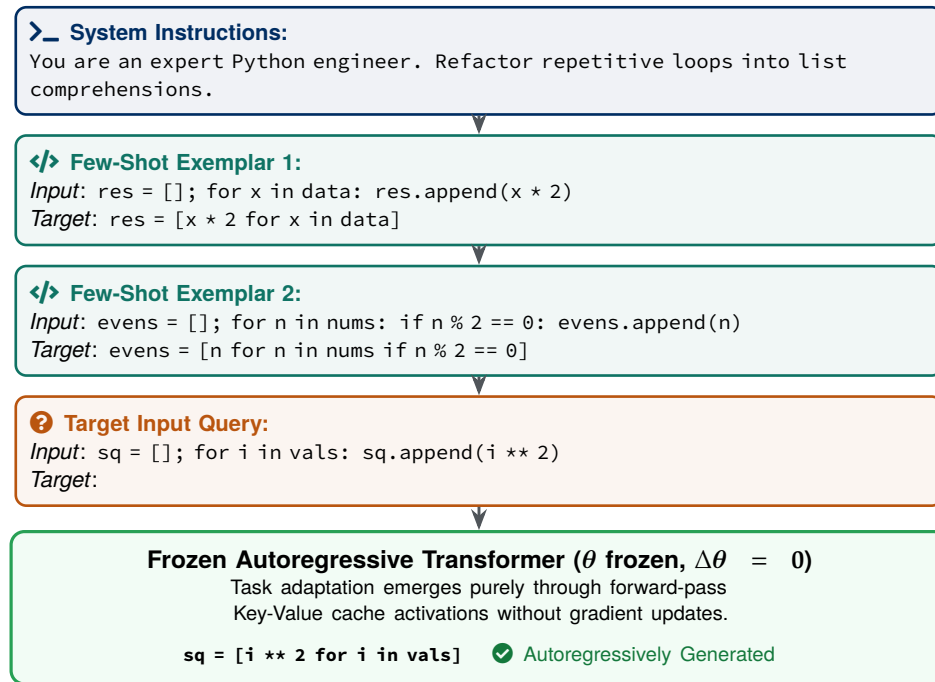


Figure 1.10: Anatomy of in-context few-shot learning within a frozen foundation model.

Mathematical formalization of in-context learning. Prior to GPT-3, adapting a model to a downstream task $\mathcal{T}$ required computing gradients and modifying parameters: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_{\mathcal{T}}$. In contrast, In-Context Learning conditions a completely frozen parameter set θ on a prompt containing K input-output exemplar demonstrations $\mathcal{D}_{\text{demo}} = \{(x^{(1)}, y^{(1)}), \dots, (x^{(K)}, y^{(K)})\}$ and a target query x :

$$P(y \mid x, \mathcal{D}_{\text{demo}}; \theta) = \prod_{k=1}^{|y|} P(y_k \mid x_{\text{prompt}}, \mathcal{D}_{\text{demo}}, y_{<k}; \theta), \quad \text{with } \Delta\theta = 0. \quad (1.46)$$

As illustrated in Figure 1.10, task adaptation occurs purely during the forward pass through the attention mechanism: self-attention across the exemplar activations acts as an implicit, gradient-free meta-optimization process.

The parameter scaling arms race and the undertraining crisis. Following GPT-3, the industry initiated an aggressive scaling arms race, culminating in dense models like Megatron-Turing NLG 530B. However, these models were trained following the scaling laws proposed by Kaplan

et al. (2020) [23], which recommended scaling parameters significantly faster than training data. Consequently, models like GPT-3 (175B parameters trained on only 300B tokens) were severely **undertrained** relative to their parameter capacity, resulting in enormous inference latency and compute inefficiency.

1.8.10 2022: compute-optimal scaling laws and the Chinchilla shift

In March 2022, Jordan Hoffmann and colleagues at DeepMind published “*Training Compute-Optimal Large Language Models*” [3], introducing the **Chinchilla** scaling laws and fundamentally correcting the trajectory of modern AI.

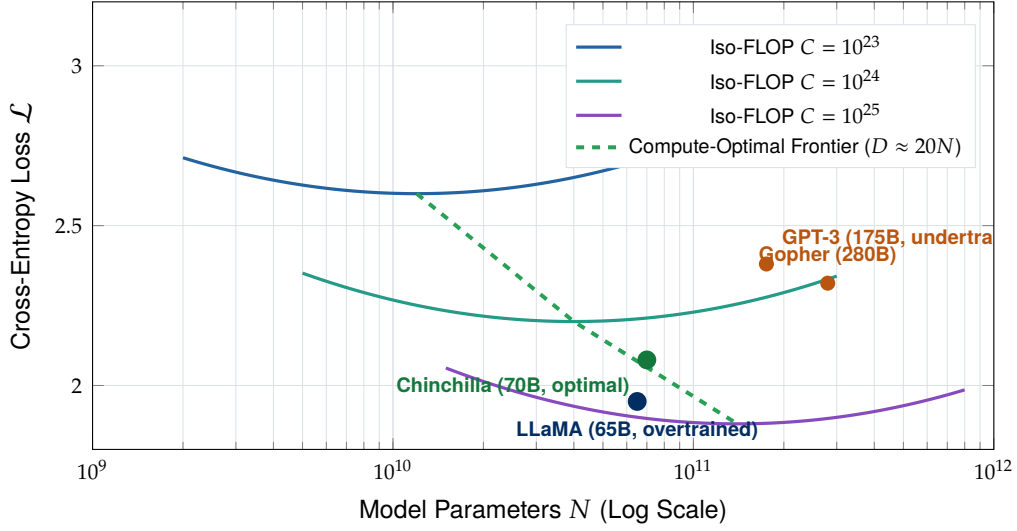


Figure 1.11: Compute-optimal scaling frontier and iso-FLOP loss trajectories.

Mathematical derivation of compute optimality. Given a fixed compute budget $C \approx 6ND$ floating-point operations (where N is parameter count and D is training token count), the cross-entropy loss is modeled as a power-law parametric surface:

$$\mathcal{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad (1.47)$$

where E is irreducible entropy, A, B are scaling coefficients, and $\alpha \approx 0.34, \beta \approx 0.28$ are empirical exponents.

To find the optimal allocation of compute between model parameters N and dataset size D , we formulate the constrained optimization problem:

$$\min_{N, D} \mathcal{L}(N, D) \quad \text{subject to} \quad 6ND = C. \quad (1.48)$$

Substituting $D = \frac{C}{6N}$ into Equation (1.47) and taking the derivative with respect to N :

$$\frac{\partial \mathcal{L}}{\partial N} = -\alpha A N^{-\alpha-1} + \beta B \left(\frac{C}{6}\right)^{-\beta} N^{\beta-1} = 0. \quad (1.49)$$

Solving for the optimal parameter count N_{opt} and token count D_{opt} :

$$N_{\text{opt}} \propto C^{\frac{\beta}{\alpha+\beta}} \approx C^{0.45}, \quad D_{\text{opt}} \propto C^{\frac{\alpha}{\alpha+\beta}} \approx C^{0.55}. \quad (1.50)$$

The Chinchilla revelation: $D \approx 20N$. Because $\frac{\beta}{\alpha+\beta} \approx 0.45 \approx \frac{\alpha}{\alpha+\beta} \approx 0.55$, Hoffmann et al. proved that **parameters N and training tokens D should be scaled in equal proportion**. For compute-optimal training, a model requires approximately 20 tokens per parameter:

$$D_{\text{opt}} \approx 20N. \quad (1.51)$$

To validate this theory, DeepMind trained **Chinchilla** (70 billion parameters on 1.4 trillion tokens) using the exact same compute budget as Gopher (280 billion parameters on 300 billion tokens). Despite possessing only one-fourth the parameters, Chinchilla uniformly outperformed Gopher, GPT-3 (175B), and Megatron-Turing NLG (530B) across virtually all benchmarks, as plotted in Figure 1.11.

The modern inference-efficient paradigm: over-training. Chinchilla’s compute-optimal laws determine training efficiency. However, in production software engineering systems, models are evaluated billions of times during inference. Meta AI’s LLaMA (Touvron et al. 2023 [15]) took this insight a step further: by intentionally **over-training** compact models (training a 7B or 13B model on 1.4T to 2T tokens, and later LLaMA-3 on 15T tokens), the training compute exceeds the Chinchilla minimum, but downstream inference latency, memory footprint, and deployment costs drop by up to 80%. This gave birth to the modern open-source LLM ecosystem.

1.8.11 2022–present: alignment, reasoning, and software engineering agents

The latest evolutionary frontier marks the transition from passive text generators to goal-directed, autonomous reasoning and software engineering agents. A base foundation model trained solely on web tokens predicts statistical continuations; it does not natively follow instructions, verify factual correctness, or debug software.

Transforming base models into coding assistants requires a multi-stage post-training pipeline, illustrated in Figure 1.12:

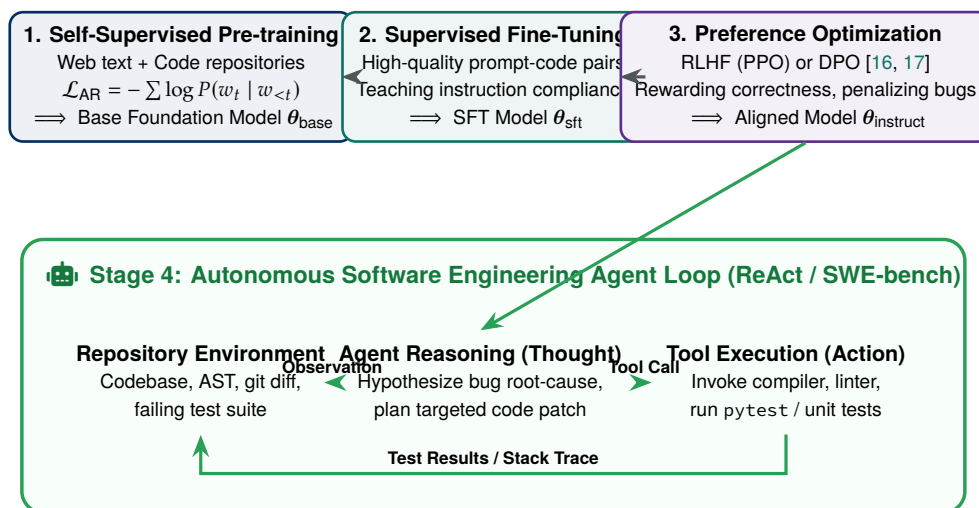


Figure 1.12: End-to-end foundation model lifecycle and autonomous software engineering agent loop.

1. **Supervised Fine-Tuning (SFT):** The base model θ_{base} is fine-tuned on curated datasets of instruction-response pairs (e.g., bug descriptions paired with git diffs, docstrings paired with function implementations). This conditions the model into an interactive dialogue assistant persona.

2. Preference Alignment (RLHF and DPO):

- *RLHF* (Ouyang et al. 2022 [16]): Employs Proximal Policy Optimization (PPO) to maximize a learned reward function $r_\phi(\mathbf{x}, \mathbf{y})$ subject to a Kullback-Leibler penalty preventing policy collapse:

$$\max_{\theta} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \mathcal{D}} \left[r_\phi(\mathbf{x}, \mathbf{y}) - \beta D_{\text{KL}}(\pi_\theta(\cdot | \mathbf{x}) \parallel \pi_{\text{ref}}(\cdot | \mathbf{x})) \right]. \quad (1.52)$$

- *Direct Preference Optimization* (DPO, Rafailov et al. 2023 [17]): Bypasses explicit reward model training by directly optimizing the language model policy on paired preferences ($y_w > y_l$) using the closed-form loss:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = - \mathbb{E}_{(\mathbf{x}, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | \mathbf{x})}{\pi_{\text{ref}}(y_w | \mathbf{x})} - \beta \log \frac{\pi_\theta(y_l | \mathbf{x})}{\pi_{\text{ref}}(y_l | \mathbf{x})} \right) \right]. \quad (1.53)$$

3. **Test-Time Reasoning Scaffolding:** Wei et al. (2022) [18] demonstrated that prompting models to generate intermediate natural-language reasoning steps (*Chain-of-Thought*) dramatically improves multi-step logic and mathematical reasoning. Modern reasoning models extend this by scaling test-time compute through tree search, self-correction, and iterative verification.
4. **Autonomous Tool-Using Software Agents:** The culmination of modern LLM research is the **ReAct** (Reasoning + Acting, Yao et al. 2022 [19]) paradigm. An autonomous software engineering agent interacts iteratively with a development environment:

$$\text{Observation } o_t \longrightarrow \text{Thought } \tau_t \longrightarrow \text{Action } a_t \in \mathcal{A} \longrightarrow \text{Environment Feedback } o_{t+1}. \quad (1.54)$$

On benchmarks like **SWE-bench** (Jimenez et al. 2024 [20]), which tests whether models can resolve real-world GitHub issues by writing multi-file patches, agents navigate complex codebases, execute terminal commands, run test suites, inspect stack traces, and iteratively refine code patches until all regression tests pass.

Language models have thus transformed from 1948 discrete statistical approximations of character frequencies into neuro-symbolic software engineering collaborators capable of reasoning across repository-scale programs.

Deep learning foundations for NLP

2.1 Chapter overview and learning outcomes

Having established the probabilistic formulation of language modeling and the combinatorial limitations of discrete symbol counting in Chapter 1, we now develop the neural computational foundations that allow language models to scale. In contemporary Natural Language Processing and AI-assisted Software Engineering, next-token prediction is reformulated as a high-dimensional, non-linear classification problem over a continuous vector space.

This chapter develops the atomic units of deep learning from the ground up. Beginning with the biological abstraction of the perceptron, we formalize the affine projection and activation mechanism of artificial neurons. We prove rigorously why multi-layer linear networks collapse mathematically into a single linear transformation without non-linear activations, state the Universal Approximation Theorem with an intuitive bump-function derivation, and compare the major activation functions used across modern architectures. We examine concrete task formulations for Software Engineering (binary classification, multi-class classification, regression), unpack loss functions (MSE, BCE, CCE) with rigorous proofs of why MSE fails for classification, analyze iterative optimization via Gradient Descent and Adam, and conduct an exhaustive adjoint trace of backpropagation on a directed acyclic computational graph (DAG).

Upon completing this chapter, you will be able to:

- Formulate the artificial perceptron as an affine transformation followed by an activation function in scalar, vector, and matrix forms.
- Visualize the 2D hyperplane decision boundary and calculate the orthogonal distance from the origin to the boundary.
- Count neural network parameters rigorously across fully connected layers, distinguishing weights and biases.
- Prove analytically why any cascade of linear layers without non-linear activations collapses into a single equivalent linear layer ($\mathbf{z} = \mathbf{W}'^T \mathbf{x} + \mathbf{b}'$).
- Analyze the properties, operating ranges, and gradient saturation pathologies of key activation functions (σ , $\tanh$, ReLU, Leaky ReLU, GELU, softmax, and linear identity).
- Explain the Universal Approximation Theorem [24–26] and reconstruct the intuitive bump-function derivation using opposing sigmoids.
- Configure appropriate output activation heads and loss functions for binary classification, multi-class classification, and regression in software engineering contexts.
- Prove mathematically why Mean Squared Error fails for classification and why Cross-Entropy provides linear, unsaturated gradients.

- Trace forward intermediate values and backward adjoint gradients ($\partial\mathcal{L}/\partial\cdot$) across a computational DAG using the multivariate chain rule [27].
- Explain the limitations of vanilla Gradient Descent in non-convex loss landscapes and the mechanics of Momentum and Adam optimizers [28].

2.2 From biological neurons to the artificial perceptron

The foundational building block of modern deep learning is the *perceptron*, originally inspired by biological neurons in the human nervous system. A biological neuron receives electrochemical signals from upstream cells through branching *dendrites*, integrates these incoming potentials in its central cell body (*soma*), and, when the accumulated electrical voltage exceeds a physiological threshold (the action potential), fires a spike through its *axon* to downstream synapses.

In computational neural networks, this biological mechanism was first formalized by Warren McCulloch and Walter Pitts in 1943 [29] as a threshold logic unit and subsequently refined by Frank Rosenblatt in 1958 [30] into the trainable *Perceptron*.

Definition 2.1: The artificial perceptron

Let $\mathbf{x} = [x_1, x_2, \dots, x_n]^\top \in \mathbb{R}^n$ represent an input feature vector, and let $\mathbf{w} = [w_1, w_2, \dots, w_n]^\top \in \mathbb{R}^n$ represent the corresponding parameter weights, with scalar bias $w_0 = b \in \mathbb{R}$. The artificial perceptron computes:

$$y = f\left(w_0 + \sum_{i=1}^n w_i x_i\right) = f(w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n), \quad (2.1)$$

where $f : \mathbb{R} \rightarrow \mathbb{R}$ is the activation function. By adopting the standard convention of augmenting the input vector with a constant dummy feature $x_0 = 1$ and defining the parameter vector as $\mathbf{w} = [w_0, w_1, \dots, w_n]^\top \in \mathbb{R}^{n+1}$, Equation (2.1) is expressed in compact vector inner-product notation:

$$y = f\left(\sum_{i=0}^n w_i x_i\right) = f(\mathbf{w}^\top \mathbf{x}) \quad \text{with } x_0 = 1, w_0 = \text{bias}. \quad (2.2)$$

Figure 2.1 illustrates the structural mechanics of the artificial perceptron: input signals are weighted by synaptic connection strengths, summed at a central junction, and passed through an activation function to generate the output signal.

2.2.1 Geometric interpretation in two dimensions

When $n = 2$, the input feature vector resides in the Cartesian plane $\mathbf{x} = [x_1, x_2]^\top$. The pre-activation affine combination:

$$z = w_1 x_1 + w_2 x_2 + w_0 = 0 \quad (2.3)$$

defines a straight line in $\mathbb{R}^2$, acting as a linear *decision boundary*.

Hyperplane properties.

- **Normal Vector $\mathbf{w}$:** The parameter vector $\mathbf{w}_{1:2} = [w_1, w_2]^\top$ is strictly orthogonal (perpendicular) to the decision boundary line. It points in the direction of the positive half-space ($z > 0$).

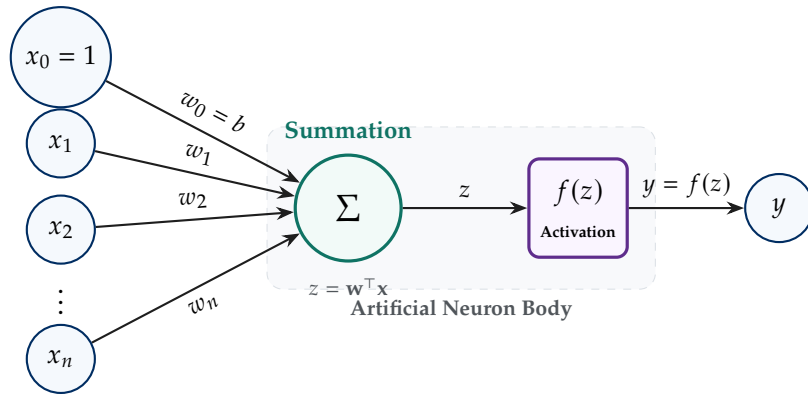


Figure 2.1: Schematic diagram of the artificial perceptron. Synaptic inputs x_i are scaled by parameter weights w_i , accumulated at the summation node with additive bias w_0 , and transformed by the non-linear activation function $f(z)$ to produce the scalar activation y .

- **Half-Space Partitioning:** The line partitions $\mathbb{R}^2$ into two disjoint open half-spaces:

$$\mathcal{H}^+ = \{(x_1, x_2) \mid w_1 x_1 + w_2 x_2 + w_0 > 0\}, \quad \mathcal{H}^- = \{(x_1, x_2) \mid w_1 x_1 + w_2 x_2 + w_0 < 0\}.$$

- **Perpendicular Distance from Origin:** The shortest Euclidean distance from the origin $(0, 0)$ to the hyperplane is:

$$d = \frac{|w_0|}{\|\mathbf{w}_{1:2}\|_2} = \frac{|w_0|}{\sqrt{w_1^2 + w_2^2}}. \quad (2.4)$$

If $w_0 = 0$, the decision boundary passes directly through the origin. The bias w_0 shifts the hyperplane along the direction of $\mathbf{w}$.

Figure 2.2 illustrates this 2D geometry, showing how the weight vector $\mathbf{w}$ governs boundary orientation while bias w_0 controls spatial offset.

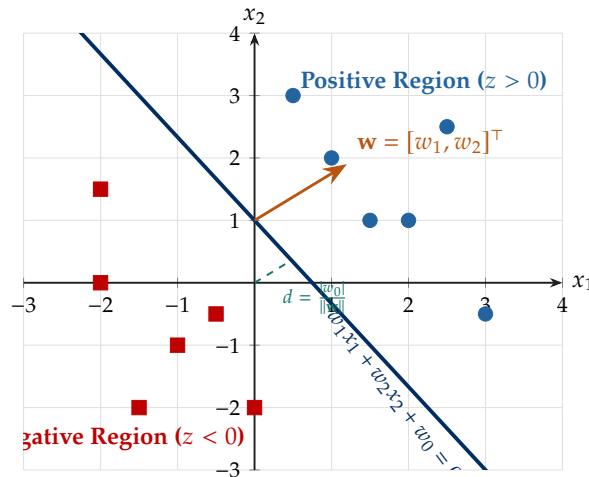


Figure 2.2: Geometric representation of the perceptron decision boundary in $\mathbb{R}^2$. The line $w_1 x_1 + w_2 x_2 + w_0 = 0$ partitions feature space into positive ($\mathcal{H}^+$) and negative ($\mathcal{H}^-$) half-spaces. The weight vector $\mathbf{w}$ defines the normal vector to the separating hyperplane, and the scalar bias w_0 determines its distance from the origin.

2.2.2 Connection to classical linear and logistic regression

The artificial perceptron unifies two foundational statistical models studied in classical data analysis:

- **Linear Regression:** If the activation function is the linear identity $f(z) = z$, Equation (2.2) computes:

$$y = \mathbf{w}^\top \mathbf{x},$$

predicting an unbounded real-valued continuous response variable.

- **Logistic Regression:** If the activation function is the logistic sigmoid $f(z) = \sigma(z) = \frac{1}{1+e^{-z}}$, Equation (2.2) computes:

$$y = \sigma(\mathbf{w}^\top \mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{x}),$$

predicting the posterior Bernoulli probability of belonging to the positive class.

2.3 Activation functions and the proof of linear stacking collapse

Activation functions serve three indispensable architectural roles in deep neural networks:

1. **Output Range Bounding:** Restricting network outputs to mathematically valid intervals (e.g., $(0, 1)$ via σ for Bernoulli probabilities, or the unit probability simplex Δ^{C-1} via softmax for categorical distributions).
2. **Topological Space Folding:** Introducing non-linear operations that fold and bend the coordinate space, enabling networks to separate linearly inseparable manifolds (such as the XOR problem).
3. **Gradient Flow Regulation:** Facilitating stable gradient backpropagation without numerical saturation or catastrophic vanishing.

2.3.1 Fully connected linear layers (dense layers)

In modern deep learning architectures, individual perceptrons do not operate in isolation; instead, multiple neurons run concurrently across a shared input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$, forming a *dense layer* (or fully connected layer).

Let d_{in} denote the input feature dimensionality and d_{out} denote the number of hidden or output units. Stacking the individual weight vectors $\mathbf{w}_j \in \mathbb{R}^{d_{\text{in}}}$ as columns of a weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ and collecting the scalar biases b_j into a bias vector $\mathbf{b} \in \mathbb{R}^{d_{\text{out}}}$, the layer computation expresses compactly in matrix notation:

$$\mathbf{y} = f(\mathbf{W}^\top \mathbf{x} + \mathbf{b}), \quad (2.5)$$

where $f(\cdot)$ is applied elementwise. Figure 2.3 diagrams this fully connected bipartite structure with explicit tensor dimensions and connection weights.

Exam insights: Parameter counting across dense layers

A standard oral and written examination question tests exact parameter counting across stacked dense layers. For an affine transformation mapping $\mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$:

$$N_{\text{params}} = \underbrace{d_{\text{in}} \times d_{\text{out}}}_{\text{weight parameters}} + \underbrace{d_{\text{out}}}_{\text{bias parameters}} = d_{\text{out}}(d_{\text{in}} + 1). \quad (2.6)$$

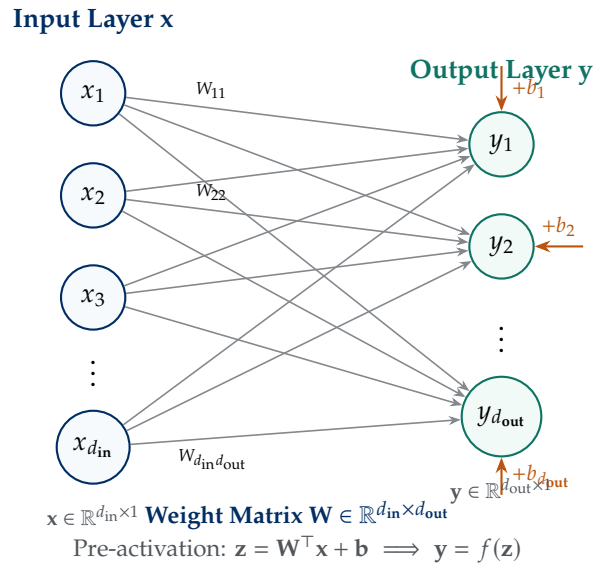


Figure 2.3: Architecture of a fully connected dense layer mapping an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ to output $\mathbf{y} \in \mathbb{R}^{d_{\text{out}}}$. Each directed edge carries an individual scalar weight W_{ij} , and each output neuron incorporates an additive bias b_j .

Example Calculation: Consider a two-layer Multi-Layer Perceptron (MLP) mapping an embedding of dimension $d_{\text{in}} = 768$ through a hidden layer $d_{\text{hidden}} = 3072$ to an output projection $d_{\text{out}} = 768$:

1. Layer 1: $3072 \times (768 + 1) = 2,359,296 + 3072 = 2,362,368$ parameters.
2. Layer 2: $768 \times (3072 + 1) = 2,359,296 + 768 = 2,360,064$ parameters.
3. Total Parameters: $2,362,368 + 2,360,064 = 4,722,432$ parameters.

Neglecting bias terms during parameter inventory counts is a frequent source of point deductions.

2.3.2 The mathematical proof of linear stacking collapse

A student encountering neural networks for the first time might ask: *Why can we not simply stack dozens of linear layers without activation functions to build a deep model?*

The following theorem demonstrates that without non-linear activations, stacking arbitrary numbers of linear layers provides zero additional representational power over a single-layer linear model.

Theorem 2.1: Linear stacking collapse

Let $f_1 : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $f_2 : \mathbb{R}^m \rightarrow \mathbb{R}^k$ denote two consecutive fully connected layers operating without non-linear activations ($f(x) = x$). Let $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ be the parameters of the first layer, and let $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ and $\mathbf{b}_2 \in \mathbb{R}^k$ be the parameters of the second layer:

$$\mathbf{h} = f_1(\mathbf{x}) = \mathbf{W}_1^T \mathbf{x} + \mathbf{b}_1, \quad (2.7)$$

$$\mathbf{z} = f_2(\mathbf{h}) = \mathbf{W}_2^T \mathbf{h} + \mathbf{b}_2. \quad (2.8)$$

The composite function $\mathbf{z} = f_2(f_1(\mathbf{x}))$ collapses identically to a single affine transformation:

$$\begin{aligned}\mathbf{z} &= \mathbf{W}_2^\top (\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 \\ &= \mathbf{W}_2^\top \mathbf{W}_1^\top \mathbf{x} + \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \\ &= (\mathbf{W}_1 \mathbf{W}_2)^\top \mathbf{x} + (\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2) \\ &= \mathbf{W}'^\top \mathbf{x} + \mathbf{b}',\end{aligned}\tag{2.9}$$

where $\mathbf{W}' = \mathbf{W}_1 \mathbf{W}_2 \in \mathbb{R}^{n \times k}$ and $\mathbf{b}' = \mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2 \in \mathbb{R}^k$.

Proof by mathematical induction. Matrix multiplication is associative and closed under composition. The product of two matrices $\mathbf{W}_1 \in \mathbb{R}^{n \times m}$ and $\mathbf{W}_2 \in \mathbb{R}^{m \times k}$ is simply a single matrix $\mathbf{W}' \in \mathbb{R}^{n \times k}$. Likewise, the affine transformation of bias vector $\mathbf{b}_1$ by $\mathbf{W}_2^\top$ added to $\mathbf{b}_2$ is simply a single constant vector $\mathbf{b}' \in \mathbb{R}^k$.

By mathematical induction, for any cascade of L linear layers:

$$\mathbf{z} = \mathbf{W}_L^\top (\dots (\mathbf{W}_2^\top (\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \dots) + \mathbf{b}_L = \mathbf{W}_{\text{net}}^\top \mathbf{x} + \mathbf{b}_{\text{net}},\tag{2.10}$$

where:

$$\mathbf{W}_{\text{net}} = \prod_{l=1}^L \mathbf{W}_l \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \quad \mathbf{b}_{\text{net}} = \mathbf{b}_L + \sum_{l=1}^{L-1} \left(\prod_{j=l+1}^L \mathbf{W}_j \right)^\top \mathbf{b}_l.\tag{2.11}$$

The composite mapping is strictly linear. Stacking 100 purely linear layers introduces millions of redundant matrix multiplications while learning nothing beyond standard linear regression. ■

💡 Intuition & analogy: Why non-linearity bends space: the XOR problem

Linear transformations can only rotate, reflect, scale, and shear Euclidean space; they preserve parallel lines and planar flat manifolds.

In their famous 1969 critique, Marvin Minsky and Seymour Papert proved that a single linear perceptron cannot compute the basic exclusive-or (XOR) logical function:

$$(0, 0) \rightarrow 0, \quad (1, 1) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1.$$

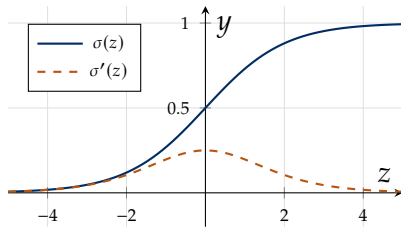
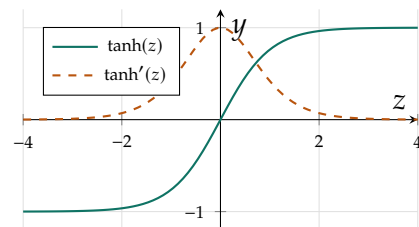
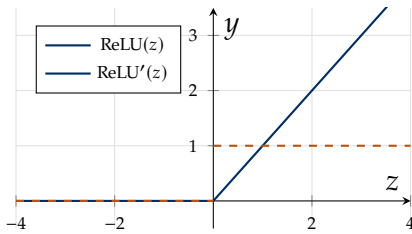
Plotting these four points reveals that no single line in $\mathbb{R}^2$ can separate $(0, 1)$ and $(1, 0)$ from $(0, 0)$ and $(1, 1)$. Non-linear activation functions act as topological “folds” and “creases” on the input space. By warping the coordinate system, intermediate hidden layers map entangled data manifolds into new representations where they become linearly separable.

2.3.3 Taxonomy of neural activation functions

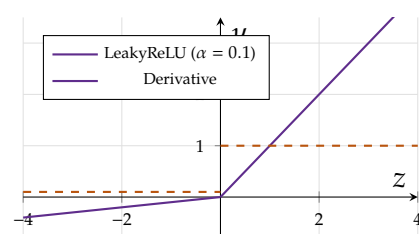
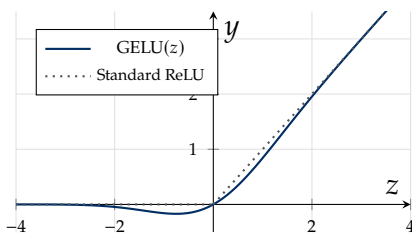
Modern neural NLP architectures deploy diverse activation functions tailored to hidden representations and output heads. Figure 2.4 presents exact mathematical plots and derivatives for the primary activation functions.

1. Logistic sigmoid. Squashes real-valued inputs $\mathbb{R}$ into the bounded probability interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.\tag{2.12}$$

(a) Logistic Sigmoid (σ) & Derivative (σ')(b) Hyperbolic Tangent ($\tanh$) & Derivative

(c) Rectified Linear Unit (ReLU) & Subgradient

(d) Leaky ReLU ($\alpha = 0.1$) & Derivative

(e) Gaussian Error Linear Unit (GELU) vs ReLU

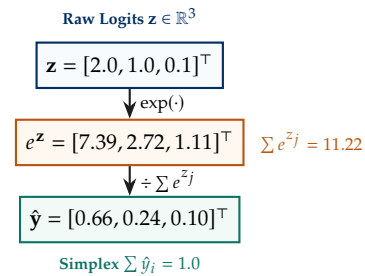
(f) Softmax Vector Mapping onto Simplex Δ^2

Figure 2.4: Exact mathematical function profiles of primary neural activation functions. (a) Logistic sigmoid squashes to $(0, 1)$ with maximum derivative 0.25; (b) Tanh is zero-centered with maximum derivative 1.0; (c) ReLU eliminates vanishing gradients for $z > 0$ but suffers from dead neurons for $z < 0$; (d) Leaky ReLU preserves a slight gradient α in the negative regime; (e) GELU provides smooth, non-monotonic probabilistic gating; (f) Softmax exponentiates and normalizes raw logit vectors into valid categorical probability distributions.

Its derivative expresses neatly in terms of the function itself:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (2.13)$$

The maximum derivative occurs at the inflection point $z = 0$, where $\sigma'(0) = 0.5 \times 0.5 = 0.25$. For $|z| \gg 0$, $\sigma'(z) \rightarrow 0$, causing severe **gradient saturation** that halts backpropagation in deep feedforward stacks.

2. Hyperbolic tangent (tanh). Squashes real inputs into the zero-centered symmetric interval $(-1, 1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1. \quad (2.14)$$

Its derivative is $\tanh'(z) = 1 - \tanh^2(z)$, with maximum value $\tanh'(0) = 1.0$. Because it is zero-centered, its expected gradient does not systematically bias weight updates in a single direction, making it vastly superior to sigmoid in recurrent hidden state transitions.

3. Rectified linear unit (ReLU). Introduced to mitigate vanishing gradients in deep networks:

$$\text{ReLU}(z) = \max(0, z). \quad (2.15)$$

Its subgradient is a piecewise step function: $\text{ReLU}'(z) = 1$ if $z > 0$, and $\text{ReLU}'(z) = 0$ if $z < 0$. It is scale-invariant and computationally cheap, but susceptible to the **dying ReLU** pathology: if a large gradient updates weights such that a neuron outputs $z < 0$ across the entire dataset, its gradient is permanently 0, rendering the neuron permanently inactive.

4. Leaky ReLU. Prevents dying neurons by introducing a small non-zero slope $\alpha \approx 0.01$ (or 0.1) in the negative regime:

$$\text{LeakyReLU}(z) = \max(\alpha z, z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases}. \quad (2.16)$$

5. Gaussian error linear unit (GELU). The standard activation function in modern Transformers (BERT, GPT-2, GPT-3, LLaMA) [13, 14, 31]:

$$\text{GELU}(z) = z \cdot \Phi(z) = z \cdot \mathbb{P}(X \leq z), \quad X \sim \mathcal{N}(0, 1). \quad (2.17)$$

GELU weights inputs by their percentile value under a standard Gaussian cumulative distribution. It is smooth, non-monotonic (exhibiting a small negative dip near $z \approx -0.75$), and approximated in deep learning libraries as:

$$\text{GELU}(z) \approx 0.5z \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (z + 0.044715z^3) \right) \right). \quad (2.18)$$

6. Softmax. Maps an unnormalized C -dimensional logit vector $\mathbf{z} \in \mathbb{R}^C$ to the probability simplex:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad \text{guaranteeing } \text{softmax}(\mathbf{z})_i \in [0, 1] \text{ and } \sum_{i=1}^C \text{softmax}(\mathbf{z})_i = 1. \quad (2.19)$$

7. Linear identity activation. Defined as $f(z) = z$. Used exclusively in continuous regression output heads where the predicted target is an unbounded real scalar.

Table 2.1 provides a comprehensive comparative summary of these activation functions.

Table 2.1: Comprehensive taxonomy and characteristics of neural activation functions.

Function	Domain	Range	Max Derivative	Primary NLP Usage & Key Features
Sigmoid (σ)	$\mathbb{R}$	$(0, 1)$	$0.25 (z = 0)$	Binary classification output head; severe saturation plateaus for $ z > 4$.
Tanh	$\mathbb{R}$	$(-1, 1)$	$1.00 (z = 0)$	Recurrent state gating (LSTM/GRU); zero-centered symmetric output.
ReLU	$\mathbb{R}$	$[0, \infty)$	$1.00 (z > 0)$	Feedforward layers in early architectures; prone to dying neurons ($z < 0$).
Leaky ReLU	$\mathbb{R}$	$\mathbb{R}$	$1.00 (z > 0)$	Robust feedforward variant; retains small negative slope α to prevent neuron death.
GELU	$\mathbb{R}$	$[-0.17, \infty)$	≈ 1.08	Canonical Transformer activation (GPT, BERT); smooth, probabilistic gating.
Softmax	$\mathbb{R}^C$	Δ^{C-1}	Matrix Jacobian	Next-token prediction vocabulary head; maps logits to discrete probability simplex.
Identity	$\mathbb{R}$	$\mathbb{R}$	$1.00 (\forall z)$	Regression output head; unbounded linear prediction.

2.4 Universal approximation theorem

The theoretical foundation validating neural networks as general-purpose function approximators is the *Universal Approximation Theorem*.

Theorem 2.2: Universal approximation theorem

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be any continuous, non-constant, bounded, and monotonically increasing activation function (e.g., sigmoid) [24]. Let $I_n \subset \mathbb{R}^n$ denote a compact (closed and bounded) subset of n -dimensional Euclidean space, and let $C(I_n)$ denote the space of continuous real-valued functions on I_n . Then, for any target continuous function $g \in C(I_n)$ and any arbitrary error tolerance $\epsilon > 0$, there exist a finite integer $M \in \mathbb{N}$, parameter weights $\mathbf{w}_j \in \mathbb{R}^n$, outer weights $\alpha_j \in \mathbb{R}$, and biases $b_j \in \mathbb{R}$ such that the single-hidden-layer feedforward network:

$$F(\mathbf{x}) = \sum_{j=1}^M \alpha_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j) \quad (2.20)$$

satisfies uniform approximation error:

$$\sup_{\mathbf{x} \in I_n} |F(\mathbf{x}) - g(\mathbf{x})| < \epsilon. \quad (2.21)$$

Originally proved by George Cybenko [24] for sigmoidal activations and generalized by Kurt Hornik et al. [25] and Moshe Leshno et al. [26] to arbitrary continuous non-polynomial activations, this theorem guarantees that a feedforward network with a single hidden layer has the mathematical capacity to approximate any continuous function.

2.4.1 Intuitive geometric derivation: bump-function synthesis

To understand intuitively why a single hidden layer can approximate any arbitrary continuous curve, consider a 1D function $g(x) : [a, b] \rightarrow \mathbb{R}$.

Step 1: Constructing a localized rectangular bump. Consider two sigmoid neurons with very high slope $w \gg 0$ located at spatial offsets x_1 and x_2 (with $x_1 < x_2$). As $w \rightarrow \infty$, the sigmoid

$\sigma(w(x - x_1))$ converges to a Heaviside step function $H(x - x_1)$.

Subtracting the second sigmoid from the first produces a localized *bump function* (or rectangular wavelet):

$$B(x) = \sigma(w(x - x_1)) - \sigma(w(x - x_2)) \approx \begin{cases} 1 & \text{if } x_1 \leq x \leq x_2 \\ 0 & \text{otherwise} \end{cases}. \quad (2.22)$$

Figure 2.5(a) illustrates how two opposing sigmoids subtract to create a localized rectangular pulse.

Step 2: Riemann sum function reconstruction. By the definition of continuous functions on compact intervals, $g(x)$ can be approximated to arbitrary accuracy ϵ by a piecewise-constant step function consisting of M uniform sub-intervals $[x_k, x_{k+1}]$ with heights $c_k = g(x_k)$, exactly as in standard Riemann integration:

$$g(x) \approx \sum_{k=1}^M c_k B_k(x) = \sum_{k=1}^M c_k [\sigma(w(x - x_k)) - \sigma(w(x - x_{k+1}))]. \quad (2.23)$$

Each bump $B_k(x)$ requires exactly two hidden sigmoid neurons. Consequently, a single hidden layer containing $2M$ neurons can synthesize any continuous function to arbitrary precision, as visualized in Figure 2.5(b).

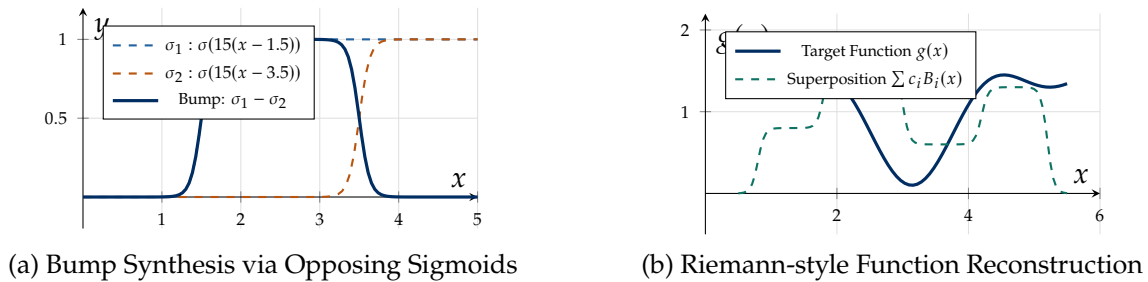


Figure 2.5: Visual derivation of the Universal Approximation Theorem. (a) Subtracting two shifted steep sigmoids produces a localized rectangular bump function $B(x)$; (b) Superimposing multiple scaled bump functions reconstructs any continuous target function to arbitrary accuracy ϵ .

2.4.2 Why deep networks if shallow networks are universal approximators?

If a single hidden layer can approximate any function, why does modern deep learning exclusively use deep architectures with dozens or hundreds of stacked layers (e.g., 96 layers in GPT-3)?

1. **Non-Constructive Existence:** The theorem guarantees that parameters exist, but provides no gradient-based algorithm to discover them and establishes no finite bound on the width M .
2. **Exponential Parameter Inefficiency:** Approximating highly compositional functions in n dimensions with a single hidden layer requires a width M that scales exponentially with input dimensionality ($M \sim O(2^n)$).
3. **Hierarchical Compositionality:** Deep networks factorize functions into hierarchical abstractions: early layers learn low-level primitives (subwords, syntax), intermediate layers learn grammatical structure, and deep layers capture long-range semantic discourse. Deep networks achieve equal approximation power with exponentially fewer parameters than shallow networks.

2.5 Architectural task formulations and software engineering case studies

In machine learning applications for software engineering, task formulations map specific software artifacts (commits, ASTs, bug reports, source code tokens) into continuous predictions. Three primary architectural heads dominate:

2.5.1 1. Binary classification: automated vulnerability detection

Mathematical formulation. The model projects the final hidden state $\mathbf{h} \in \mathbb{R}^d$ to a single scalar logit $z \in \mathbb{R}$. The sigmoid activation maps z to predicted probability $\hat{y} = \sigma(z) \in [0, 1]$. The decision rule uses a threshold $\tau = 0.5$:

$$\hat{C} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \ (z \geq 0) \\ 0 & \text{if } \hat{y} < 0.5 \ (z < 0) \end{cases}. \quad (2.24)$$

Software engineering case study. Consider **Just-In-Time (JIT) Vulnerability Prediction**: predicting whether an incoming Git commit introduces a security vulnerability ($y = 1$) or is clean ($y = 0$). Features include:

- x_1 : Code Churn (total lines of code added plus modified).
- x_2 : Cyclomatic Complexity delta (ΔCC).

Figure 2.6(a) illustrates the 2D feature plane where a trained linear perceptron separates high-churn, complex vulnerable commits from standard maintenance commits.

2.5.2 2. Multi-class classification: defect category prediction

Mathematical formulation. For C mutually exclusive classes, the model projects $\mathbf{h} \in \mathbb{R}^d$ to a logit vector $\mathbf{z} = [z_1, \dots, z_C]^\top \in \mathbb{R}^C$. Softmax converts logits into a normalized categorical distribution:

$$\hat{y}_i = \mathbb{P}(Y = c_i | \mathbf{x}) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}, \quad \hat{C} = \arg \max_{i \in \{1, \dots, C\}} \hat{y}_i. \quad (2.25)$$

Software engineering case study. Consider **Automated Bug Report Triage**: classifying an unassigned software defect into $C = 4$ mutually exclusive failure categories:

$$C = \{\text{NullPointerException}, \text{ResourceLeak}, \text{ConcurrencyDeadlock}, \text{SyntaxError}\}.$$

Figure 2.6(b) diagrams the 2D feature space partitioned into multi-class decision regions alongside the resulting softmax probability distribution.

2.5.3 3. Regression: code review effort prediction

Mathematical formulation. For unbounded continuous numeric quantities, the output head applies the linear identity activation $f(z) = z$:

$$\hat{y} = \mathbf{w}_{\text{out}}^\top \mathbf{h} + b_{\text{out}} \in \mathbb{R}. \quad (2.26)$$

Software engineering case study. Consider **Pull Request Review Time Estimation**: predicting the expected developer-hours ($\hat{y} \in \mathbb{R}^+$) required to review and merge a pull request based on the number of modified files and author repository experience. Figure 2.6(c) displays a fitted regression line through empirical software engineering data points with residual errors.

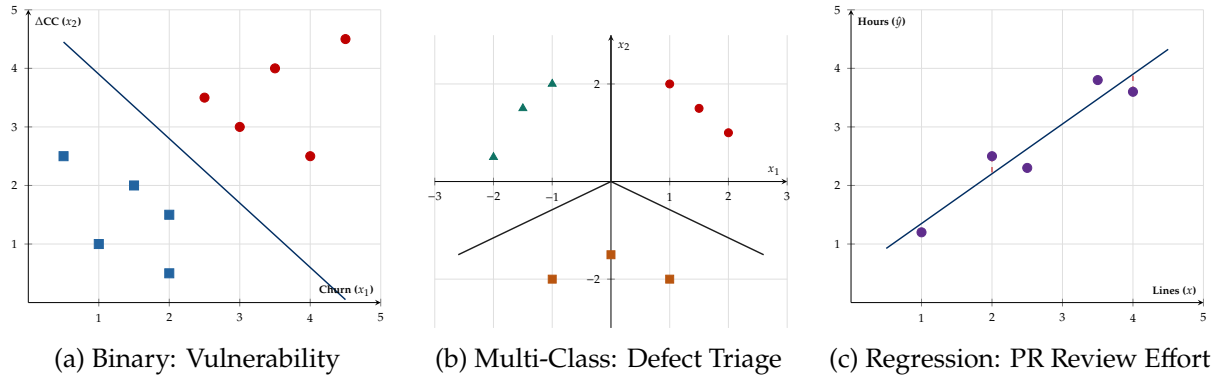


Figure 2.6: Task formulations across Software Engineering workflows: (a) Binary vulnerability detection with separating hyperplane; (b) Multi-class defect categorization with partitioned decision regions; (c) Continuous PR review effort regression with residual errors.

2.6 Loss function formulations and optimization landscapes

2.6.1 Loss function formulations

Training a neural network requires an objective function $\mathcal{L}(\theta)$ that measures the discrepancy between model predictions $\hat{y}$ and ground-truth targets y .

1. Mean squared error (MSE). For continuous regression targets:

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2. \quad (2.27)$$

2. Binary cross-entropy (BCE). For binary classification with target $y \in \{0, 1\}$ and predicted probability $\hat{y} = \sigma(z) \in [0, 1]$:

$$\mathcal{L}_{\text{BCE}}(\theta) = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y}). \quad (2.28)$$

3. Categorical cross-entropy (CCE). For multi-class classification over C mutually exclusive classes with one-hot target vector $\mathbf{y} \in \{0, 1\}^C$:

$$\mathcal{L}_{\text{CE}}(\theta) = - \sum_{i=1}^C y_i \ln(\hat{y}_i) = - \ln(\hat{y}_{\text{target}}) = - \ln \left(\frac{e^{z_{\text{target}}}}{\sum_{j=1}^C e^{z_j}} \right). \quad (2.29)$$

Figure 2.7 compares the mathematical profiles of MSE and BCE for a positive target $y = 1$.

2.6.2 Rigorous proof: why MSE fails for classification

A fundamental question in deep learning is: *Why do we never use Mean Squared Error with sigmoid activation for classification?*

The answer lies in gradient saturation and non-convexity.

Theorem 2.3: Gradient saturation of MSE under sigmoid activation

Let a binary classification model predict $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ with ground-truth target $y \in \{0, 1\}$. Under the quadratic loss $\mathcal{L}_{\text{MSE}} = \frac{1}{2}(y - \hat{y})^2$, the gradient with respect to the

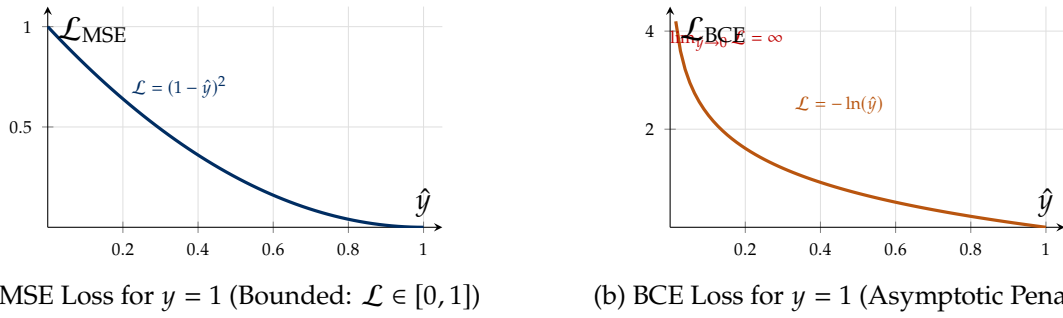


Figure 2.7: Comparison of loss function behaviors for target $y = 1$. While MSE is bounded quadratically at 1.0, BCE imposes an asymptotic infinite penalty ($\mathcal{L} \rightarrow \infty$) as predicted probability $\hat{y} \rightarrow 0$, providing steep, non-vanishing corrective gradients.

pre-activation logit z is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}). \quad (2.30)$$

Conversely, under Binary Cross-Entropy $\mathcal{L}_{\text{BCE}} = -y \ln(\hat{y}) - (1 - y) \ln(1 - \hat{y})$, the gradient is:

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y. \quad (2.31)$$

Proof. 1. For MSE, applying the calculus chain rule:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = \frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z}. \quad (2.32)$$

The derivative of the loss with respect to $\hat{y}$ is:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left[\frac{1}{2} (y - \hat{y})^2 \right] = -(y - \hat{y}).$$

The derivative of the sigmoid is $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$. Substituting:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(y - \hat{y}) \hat{y}(1 - \hat{y}). \quad (2.33)$$

2. For BCE, expanding the loss:

$$\mathcal{L}_{\text{BCE}} = -y \ln(\sigma(z)) - (1 - y) \ln(1 - \sigma(z)). \quad (2.34)$$

Differentiating directly with respect to logit z :

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} &= -y \frac{\sigma'(z)}{\sigma(z)} - (1 - y) \frac{-\sigma'(z)}{1 - \sigma(z)} \\ &= -y \frac{\sigma(z)(1 - \sigma(z))}{\sigma(z)} + (1 - y) \frac{\sigma(z)(1 - \sigma(z))}{1 - \sigma(z)} \\ &= -y(1 - \sigma(z)) + (1 - y)\sigma(z) \\ &= -y + y\sigma(z) + \sigma(z) - y\sigma(z) \\ &= \sigma(z) - y = \hat{y} - y. \quad \blacksquare \end{aligned}$$

Analytical comparison of optimization behavior. The fundamental architectural flaws of MSE for classification are immediately apparent from Equations (2.33) and (2.31):

- **The Worst-Case MSE Pathology (Confident and Wrong):** Suppose the true label is $y = 1$, but the network is completely mistaken, predicting $z = -10 \implies \hat{y} \approx 0.000045$. The prediction error is maximal: $(y - \hat{y}) \approx 1.0$. However, evaluating the MSE gradient:

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial z} = -(1.0) \times (0.000045) \times (0.999955) \approx -0.000045 \approx 0!$$

The gradient completely vanishes! Even though the model is entirely wrong, parameter updates halt because the sigmoid derivative $\hat{y}(1 - \hat{y})$ saturates to zero. Optimization gets trapped on a flat plateau.

- **The BCE Remedy:** Under BCE, for the identical scenario ($y = 1, \hat{y} \approx 0$):

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial z} = \hat{y} - y = 0 - 1 = -1.0.$$

The sigmoid derivative in the numerator cancels perfectly with the denominator in the derivative of the logarithm! The gradient is strictly linear with respect to the error ($y - \hat{y}$) and provides maximum learning signal precisely when the model is most wrong.

- **Non-Convexity:** While MSE with linear regression is strictly convex, combining MSE with non-linear sigmoid activations creates a non-convex loss surface with numerous local plateaus. In contrast, BCE is strictly convex with respect to logit z .

2.6.3 Maximum likelihood estimation derivation of cross-entropy

Cross-entropy is not an empirical heuristic; it is the mathematically canonical loss derived from Maximum Likelihood Estimation (MLE).

Bernoulli likelihood derivation (BCE). For binary classification, target $y_i \in \{0, 1\}$ follows a Bernoulli distribution parameterized by probability $p_i = \hat{y}_i = \sigma(z_i)$:

$$\mathbb{P}(Y = y_i \mid \mathbf{x}_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}. \quad (2.35)$$

For an independent dataset of N samples, the total likelihood is $\mathcal{L}_{\text{data}}(\theta) = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$. Taking the negative log-likelihood (NLL):

$$\mathcal{L}(\theta) = -\ln \mathcal{L}_{\text{data}}(\theta) = -\sum_{i=1}^N [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)] = \sum_{i=1}^N \mathcal{L}_{\text{BCE},i}. \quad (2.36)$$

Minimizing BCE is mathematically identical to maximizing the data likelihood under a Bernoulli distribution.

Categorical likelihood derivation (CCE). For multi-class classification over C classes with one-hot targets $\mathbf{y}_i$, the likelihood follows a Categorical distribution:

$$\mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = \prod_{c=1}^C \hat{y}_{ic}^{y_{ic}} \implies -\ln \mathbb{P}(\mathbf{y}_i \mid \mathbf{x}_i) = -\sum_{c=1}^C y_{ic} \ln \hat{y}_{ic} = \mathcal{L}_{\text{CE}}. \quad (2.37)$$

2.6.4 Gradient descent and adaptive optimizers

Parameters θ are updated iteratively along the negative gradient:

$$\theta^{(t+1)} := \theta^{(t)} - \alpha \nabla_{\theta} \mathcal{L}(\theta^{(t)}), \quad (2.38)$$

where $\alpha > 0$ is the learning rate.

Limitations of vanilla gradient descent. In deep neural architectures, vanilla Gradient Descent struggles with:

1. **Ravines and Ill-Conditioned Curvature:** Valleys where gradient magnitude is large across the ravine walls but tiny along the base, causing erratic cross-valley oscillation.
2. **Saddle Points:** Points where $\nabla \mathcal{L} = 0$ while the Hessian possesses both positive and negative eigenvalues.

Adaptive remedies: Momentum and Adam.

- **Momentum:** Incorporates an exponentially decaying velocity vector $\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla \mathcal{L}(\theta_t)$, accelerating descent through saddle regions.
- **Adam (Adaptive Moment Estimation) [28]:** Tracks running averages of both gradients (first moment $\mathbf{m}_t$) and squared gradients (second uncentered moment $\mathbf{v}_t$), with bias corrections:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (2.39)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (2.40)$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t. \quad (2.41)$$

Adam scales learning rates adaptively per parameter, making it the standard optimizer across LLM pre-training and fine-tuning pipelines.

2.7 Computational graphs and exact adjoint backpropagation

To compute gradients efficiently across billions of parameters, deep learning frameworks (PyTorch, JAX) represent mathematical workflows as a *Directed Acyclic Graph (DAG)*, known as a *computational graph*.

Definition 2.2: Computational graph

A computational graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed acyclic graph where:

- Each node $v \in \mathcal{V}$ represents an input variable, parameter, or primitive mathematical operation ($+$, $\times$, $-$, $\exp$).
- Each directed edge $e = (u, v) \in \mathcal{E}$ represents intermediate data flow from variable u into operation v .

2.7.1 Exact backpropagation walkthrough from slide 24

We reproduce and derive the exact pedagogical computational graph and adjoint trace from Slide 24 of the course materials.

Problem setup. Consider a model with two scalar parameters $\theta_1, \theta_2 \in \mathbb{R}$ evaluated on training pair $(x, y) \in \mathbb{R}^2$:

$$\text{Model Prediction: } \hat{y} = \theta_1 \theta_2 x, \quad (2.42)$$

$$\text{Objective Loss: } \mathcal{L} = (\theta_1 \theta_2 x - y)^2. \quad (2.43)$$

Forward decomposition. The graph decomposes Equation (2.43) into four primitive operations:

$$a = \theta_1 \theta_2 \quad (\text{parameter interaction product}), \quad (2.44)$$

$$b = ax = \theta_1 \theta_2 x \quad (\text{prediction } \hat{y}), \quad (2.45)$$

$$c = b - y = \theta_1 \theta_2 x - y \quad (\text{residual error}), \quad (2.46)$$

$$\mathcal{L} = c^2 = (\theta_1 \theta_2 x - y)^2 \quad (\text{scalar squared loss}). \quad (2.47)$$

Backward adjoint chain rule derivation. Backpropagation computes the derivative of scalar loss $\mathcal{L}$ with respect to every intermediate variable and parameter by traversing the graph in reverse topological order:

$$\text{Step 1: } \frac{\partial \mathcal{L}}{\partial c} = \frac{\partial(c^2)}{\partial c} = 2c. \quad (2.48)$$

$$\text{Step 2: } \frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \frac{\partial c}{\partial b} = 2c \cdot \frac{\partial(b - y)}{\partial b} = 2c \cdot 1 = 2c. \quad (2.49)$$

$$\text{Step 3: } \frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \frac{\partial b}{\partial a} = 2c \cdot \frac{\partial(ax)}{\partial a} = 2c \cdot x = 2cx. \quad (2.50)$$

$$\text{Step 4: } \frac{\partial \mathcal{L}}{\partial \theta_1} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_1} = (2cx) \cdot \frac{\partial(\theta_1 \theta_2)}{\partial \theta_1} = 2cx \cdot \theta_2 = 2(\theta_1 \theta_2 x - y)x \theta_2. \quad (2.51)$$

$$\text{Step 5: } \frac{\partial \mathcal{L}}{\partial \theta_2} = \frac{\partial \mathcal{L}}{\partial a} \frac{\partial a}{\partial \theta_2} = (2cx) \cdot \frac{\partial(\theta_1 \theta_2)}{\partial \theta_2} = 2cx \cdot \theta_1 = 2(\theta_1 \theta_2 x - y)x \theta_1. \quad (2.52)$$

Figure 2.8 illustrates this complete forward and backward adjoint execution.

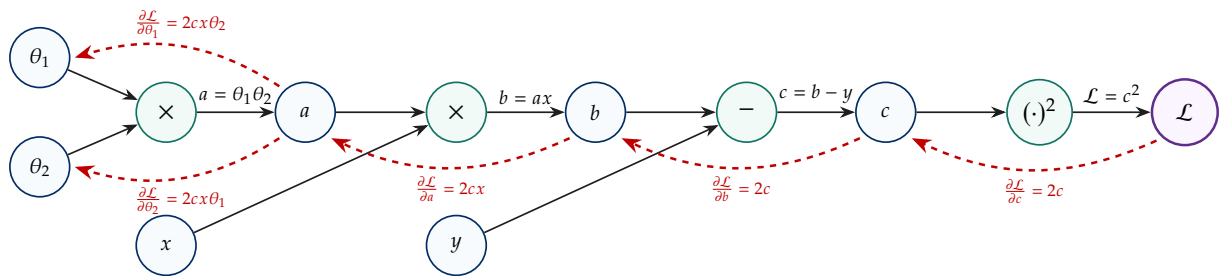


Figure 2.8: Directed Acyclic Computational Graph (DAG) for model $\hat{y} = \theta_1 \theta_2 x$ with quadratic loss $\mathcal{L} = (\theta_1 \theta_2 x - y)^2$ (reproduced from Slide 24). Solid black arrows trace forward execution; dashed red arrows trace reverse-mode adjoint backpropagation via the chain rule [27].

💡 Intuition & analogy: 3Blue1Brown's geometric view of backpropagation

Grant Sanderson (3Blue1Brown) articulates backpropagation not as abstract calculus algebra, but as a systematic tracking of sensitivity *nudges*. Imagine wiggling parameter θ_1 by an infinitesimal perturbation ϵ . That nudge ripples through the product node to scale a by $\epsilon\theta_2$. The change in a wiggles b by $\epsilon\theta_2x$, which directly shifts error c and finally scales loss $\mathcal{L}$ by $2c\epsilon\theta_2x$. Backpropagation simply reads these connected multiplicative sensitivity tracks in reverse, gathering the composite derivative at each fork in the graph.

🔗 External deep dive: Building autograd from scratch: Andrej Karpathy's Micrograd

Andrej Karpathy's open-source library *Micrograd* implements reverse-mode automatic differentiation in fewer than 100 lines of pure Python, demonstrating the exact computational graph mechanics shown in Figure 2.8.

Each scalar variable is wrapped inside a `Value` class containing:

- `.data`: the floating-point scalar computed during the forward pass.
- `.grad`: the accumulated adjoint derivative $\partial\mathcal{L}/\partial\cdot$, initialized to 0.0.
- `._prev`: a set storing pointers to operand children nodes.
- `._backward`: a closure storing the local multivariate chain rule.

For example, multiplication is implemented as:

```

1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*)
4     def _backward():
5         self.grad += other.data * out.grad
6         other.grad += self.data * out.grad
7     out._backward = _backward
8     return out

```

When calling `loss.backward()`, the engine performs a **topological sort** on the DAG to establish reverse dependency ordering, sets `loss.grad = 1.0`, and executes `node._backward()` across all parent nodes in reverse topological order. Modern deep learning frameworks like PyTorch execute this exact process on multi-dimensional GPU tensors.

Distributed word representations and Word2Vec

3.1 The representation problem: why continuous embeddings are mandatory

Natural language and software engineering source code are fundamentally composed of discrete, symbolic tokens. In written English or in source files, programs and sentences present themselves as strings of alphanumeric characters: words such as "compiler", "optimizer", "gradient", and keywords such as "class", "def", "return".

Conversely, modern artificial neural networks are continuous, differentiable mathematical operators. An L -layer deep neural network $\mathcal{F}(\mathbf{x}; \Theta)$ computes outputs through compositions of parameterized linear projections and non-linear activation functions:

$$\mathcal{F}(\mathbf{x}; \Theta) = \sigma_L \left(\mathbf{W}_L \sigma_{L-1} \left(\cdots \sigma_1 (\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots \right) + \mathbf{b}_L \right), \quad (3.1)$$

where $\mathbf{W}_l \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathbf{b}_l \in \mathbb{R}^{d_l}$. Such architectures rely fundamentally on the continuous mathematics of vector spaces $\mathbb{R}^d$, multivariable calculus, and gradient-based backpropagation. The optimization algorithm minimizes an empirical risk loss $\mathcal{L}(\Theta)$ by iteratively updating weight tensors along the negative gradient direction:

$$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}(\Theta). \quad (3.2)$$

This mathematical reality creates an immediate, fundamental impasse: **discrete text strings cannot be differentiated**. The partial derivative of a scalar loss with respect to a raw string literal, $\frac{\partial \mathcal{L}}{\partial \text{"cat"}}$, is mathematically undefined. Discrete symbols possess no notion of infinitesimal perturbation, tangent spaces, or linear vector combinations. One cannot add "cat" to "dog", nor compute an inner product between "while" and "for".

To bridge this chasm between discrete symbolic communication and continuous gradient-based computation, we must establish a formal mapping from the discrete vocabulary into a continuous vector space.

Definition 3.1: Continuous word embedding

Let $\mathcal{V} = \{w_1, w_2, \dots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ distinct discrete symbolic tokens. A **continuous word embedding** is an injective mapping:

$$\phi : \mathcal{V} \longrightarrow \mathbb{R}^d, \quad \text{with } d \ll V, \quad (3.3)$$

that maps each discrete symbol $w_i \in \mathcal{V}$ to a low-dimensional, dense continuous vector $\mathbf{v}_i = \phi(w_i) \in \mathbb{R}^d$. The target vector space $(\mathbb{R}^d, \|\cdot\|, \langle \cdot, \cdot \rangle)$ is a continuous metric space equipped with an inner product, where geometric distance $D(\mathbf{v}_i, \mathbf{v}_j)$ and angular orientation $\cos(\mathbf{v}_i, \mathbf{v}_j)$ preserve semantic and syntactic relationships.

In typical natural language and software engineering applications, vocabulary sizes range from $V \approx 30,000$ to upwards of $V = 1,000,000$ unique tokens, while the latent embedding dimension is chosen to be compact: $d \in [100, 1024]$ (e.g., $d = 300$ in classical Word2Vec and FastText; $d = 768$ in standard Transformer baselines). By mapping discrete words into $\mathbb{R}^d$, language tokens become continuous variables that participate directly in matrix algebra, linear transformations, and gradient backpropagation.

To understand why dense continuous representations are indispensable, we must first examine the standard categorical baseline: local one-hot encoding. While intuitive and trivial to construct, one-hot vectors suffer from severe geometric and computational pathologies that fundamentally incapacitate neural learning.

3.2 The baseline approach: local one-hot encodings and why they fail

The most straightforward, historical baseline for converting discrete categorical symbols into numeric vectors is **one-hot encoding**. Under this local representational scheme, every word $w_i \in \mathcal{V}$ is mapped to a standard basis coordinate vector $\mathbf{e}_i$ in a V -dimensional Euclidean space $\mathbb{R}^V$:

$$\mathbf{e}_i = [\underbrace{0, \dots, 0}_{i-1 \text{ zeros}}, \underbrace{1, 0, \dots, 0}_{V-i \text{ zeros}}]^\top \in \{0, 1\}^V, \quad \text{where } (\mathbf{e}_i)_k = \delta_{ik} = \begin{cases} 1 & \text{if } k = i, \\ 0 & \text{if } k \neq i. \end{cases} \quad (3.4)$$

For example, consider a toy vocabulary of $V = 5$ tokens from Prof. Flavio Giobergia's lecture slides:

$$\mathcal{V} = \{\text{"dog", "cat", "fish", "pen", "pencil"}\}. \quad (3.5)$$

Under one-hot encoding, each word is assigned an isolated Cartesian coordinate axis:

$$\mathbf{e}_{\text{dog}} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{cat}} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{fish}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pen}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{pencil}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \quad (3.6)$$

3.2.1 Algebraic equivalence: one-hot matrix multiplication as embedding lookup

In his foundational textbook *Build a Large Language Model from Scratch* [32], Sebastian Raschka proves that feeding a one-hot vector into a dense linear layer is algebraically identical to retrieving a specific row or column from a weight lookup table.

Theorem 3.1: Embedding lookup equivalence

Let $\mathcal{V}$ be a vocabulary of size V , and let token $w_i \in \mathcal{V}$ be encoded as the standard basis vector $\mathbf{e}_i \in \{0, 1\}^V$.

Case 1: Column-Vector Orientation (Lecture Slide Convention). Let $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ be the embedding matrix whose k -th column corresponds to the d -dimensional embedding vector $\mathbf{w}_k = (\mathbf{W}_{\text{in}})_{:,k} \in \mathbb{R}^d$:

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} | & | & \cdots & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_V \\ | & | & \cdots & | \end{bmatrix}. \quad (3.7)$$

Multiplying $\mathbf{W}_{\text{in}}$ by the one-hot column vector $\mathbf{e}_i$ extracts the i -th column of $\mathbf{W}_{\text{in}}$:

$$\mathbf{h} = \mathbf{W}_{\text{in}} \mathbf{e}_i = \sum_{k=1}^V (\mathbf{e}_i)_k (\mathbf{W}_{\text{in}})_{:,k} = 1 \cdot (\mathbf{W}_{\text{in}})_{:,i} + \sum_{k \neq i} 0 \cdot (\mathbf{W}_{\text{in}})_{:,k} = \mathbf{w}_i \in \mathbb{R}^d. \quad (3.8)$$

Case 2: Row-Vector Orientation (PyTorch and Raschka Convention). Let $\mathbf{W} \in \mathbb{R}^{V \times d}$ be an embedding table whose k -th row contains the embedding vector $\mathbf{w}_k^\top = \mathbf{W}_{k,:} \in \mathbb{R}^{1 \times d}$. Multiplying the one-hot row vector $\mathbf{e}_i^\top$ by $\mathbf{W}$ extracts the i -th row of $\mathbf{W}$:

$$\mathbf{e}_i^\top \mathbf{W} = \sum_{k=1}^V (\mathbf{e}_i)_k \mathbf{W}_{k,:} = 1 \cdot \mathbf{W}_{i,:} + \sum_{k \neq i} 0 \cdot \mathbf{W}_{k,:} = \mathbf{W}_{i,:} = \mathbf{w}_i^\top \in \mathbb{R}^{1 \times d}. \quad (3.9)$$

Hence, matrix multiplication of an embedding weight matrix by a one-hot indicator vector is identical to an indexed memory table lookup:

$$\mathbf{W}_{\text{in}} \mathbf{e}_i \equiv \text{lookup}(\mathbf{W}_{\text{in}}, i). \quad (3.10)$$

3.2.2 Systems-level justification: why frameworks use dedicated embedding lookups

Although dense matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ and table indexing $\text{lookup}(\mathbf{W}, i)$ are algebraically equivalent, no modern deep learning framework (PyTorch, TensorFlow, JAX) ever implements word embeddings as a dense matrix multiplication. Deep learning frameworks provide specialized lookup modules, such as `torch.nn.Embedding(num_embeddings=V, embedding_dim=d)`, for three decisive systems engineering reasons:

1. **Elimination of $O(V \cdot d)$ Wasted FLOPs:** Evaluating the matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ requires executing $V \cdot d$ floating-point multiplications and $(V - 1) \cdot d$ additions. In a standard language model with vocabulary $V = 50,257$ (GPT-2 tokenizer) and hidden dimension $d = 768$, computing the input projection for a single token via dense matrix multiplication requires:

$$50,257 \times 768 \approx 3.86 \times 10^7 \text{ arithmetic operations per token.} \quad (3.11)$$

Of these 38.6 million operations, exactly 99.998% evaluate $0 \times W_{k,j} = 0$. By contrast, `torch.nn.Embedding` accepts integer token IDs $i \in \{0, \dots, V - 1\}$ and performs direct $O(1)$ pointer arithmetic into physical GPU/CPU memory:

$$\text{address}(\mathbf{w}_i) = \text{base_pointer} + i \times d \times \text{sizeof}(\text{float32}), \quad (3.12)$$

retrieving the d -dimensional embedding slice in $O(1)$ time with exactly zero arithmetic FLOPs.

2. **Massive VRAM Savings (25,000× Memory Reduction):** Consider a standard training batch with batch size $B = 64$ and sequence context length $T = 1,024$. If input sequences were materialized as dense one-hot tensors, the input tensor $\mathbf{X}_{\text{onehot}}$ would have shape $(B, T, V) = (64, 1,024, 50,257)$. Storing this tensor in single-precision floating point (float32, 4 bytes per element) would require:

$$\text{VRAM}_{\text{one-hot}} = 64 \times 1,024 \times 50,257 \times 4 \text{ bytes} \approx 13,175,095,296 \text{ bytes} \approx 13.18 \text{ GB of VRAM,} \quad (3.13)$$

simply to hold the input batch before computing a single activation. Conversely, storing sequences as integer token IDs produces an `int64` tensor of shape $(B, T) = (64, 1,024)$,

requiring:

$$\text{VRAM}_{\text{indices}} = 64 \times 1,024 \times 8 \text{ bytes} = 524,288 \text{ bytes} \approx 524 \text{ KB of VRAM.} \quad (3.14)$$

This constitutes a memory bandwidth reduction factor of $\frac{13.18 \text{ GB}}{524 \text{ KB}} \approx 25,128\times$, entirely preventing Out-Of-Memory (OOM) GPU crashes.

3. **Sparse Gradient Backpropagation:** Under dense matrix multiplication $\mathbf{y} = \mathbf{e}_i^\top \mathbf{W}$, the backpropagation adjoint with respect to the weight matrix is the outer product:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{e}_i \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \right) \in \mathbb{R}^{V \times d}. \quad (3.15)$$

This gradient matrix is massive ($V \times d$) and strictly sparse: only row i contains non-zero values, while the remaining $V - 1$ rows are populated entirely by zeros. Synchronizing such dense gradient tensors across multi-GPU distributed clusters saturates PCIe and NVLink bandwidth. `torch.nn.Embedding` executes sparse indexed gradient accumulation (`index_add_`), routing the upstream vector $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^d$ directly into the i -th row of $\mathbf{W}$.

3.2.3 The 5 structural failure modes of one-hot representations

While the lookup table implementation rescues computational throughput, local one-hot representations fail catastrophically as semantic representations. They suffer from five fatal pathologies:

1. Extreme Sparsity and Curse of Dimensionality. As vocabulary size scales to web scale ($V \geq 50,000$ to 10^6), one-hot vectors become astronomical sparse vectors where 99.998% of entries are zero. The dimensionality of the vector space equals the number of distinct words. Under this curse of dimensionality, the volume of the enclosing space grows exponentially, leaving data points isolated at the extreme vertices of independent axes.

2. Metric Collapse and Semantic Orthogonality ($\cos = 0, d_E = \sqrt{2}$). Because one-hot vectors form the standard Cartesian basis of $\mathbb{R}^V$, every pair of distinct words $w_i \neq w_j$ is mutually perpendicular and equidistant:

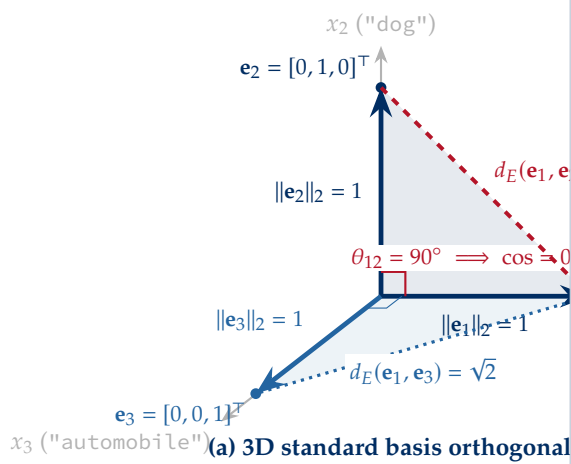
$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2} = \frac{0}{1 \cdot 1} = 0, \quad \forall i \neq j, \quad (3.16)$$

$$d_E(\mathbf{e}_i, \mathbf{e}_j) = \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{(1)^2 + (-1)^2 + \sum_{k \neq i, j} 0^2} = \sqrt{2} \approx 1.414, \quad \forall i \neq j. \quad (3.17)$$

Under one-hot encoding, every word is orthogonal and equidistant to every other word:

$$d_E(\text{"pen"}, \text{"pencil"}) = d_E(\text{"pen"}, \text{"crocodile"}) = \sqrt{2}. \quad (3.18)$$

The underlying metric space collapses into a degenerate topology where open balls $B_r(\mathbf{e}_i) = \{\mathbf{x} \in \mathcal{V} : d_E(\mathbf{e}_i, \mathbf{x}) < r\}$ contain only $\{\mathbf{e}_i\}$ for $r \leq \sqrt{2}$, and immediately engulf the entire universe of vocabulary tokens for $r > \sqrt{2}$. There is zero geometric capacity to reflect semantic proximity, synonymy, or grammatical category.



Pairwise metric equality matrices:

- Cosine similarity matrix (Gramian $E^T E = I_V$):

$$S_{\cos} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 1 & 0 & 0 \\ \text{dog} & 0 & 1 & 0 \\ \text{car} & 0 & 0 & 1 \end{pmatrix}$$

- Euclidean distance matrix ($D_{ij} = \sqrt{2}(1 - \delta_{ij})$):

$$D_{L_2} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 0 & \sqrt{2} & \sqrt{2} \\ \text{dog} & \sqrt{2} & 0 & \sqrt{2} \\ \text{car} & \sqrt{2} & \sqrt{2} & 0 \end{pmatrix}$$

The geometric pathology:

$\text{dist}(\text{"cat"}, \text{"dog"}) = \text{dist}(\text{"cat"}, \text{"car"}) = \sqrt{2}$
 $\cos(\text{"cat"}, \text{"dog"}) = \cos(\text{"cat"}, \text{"car"}) = 0$

(b) Semantic metric collapse
 All tokens are equidistant and mutually perpendicular, completely destroying semantic metric topology.

Figure 3.1: Visual proof of pairwise orthogonality and equidistance in one-hot vector representations. (a) In V -dimensional Euclidean space, each word is an orthogonal standard basis vector of unit norm ($\|e_i\|_2 = 1$). By the Pythagorean theorem, every distinct pair forms a right isosceles triangle with origin 0 , yielding an identical Euclidean distance of $\sqrt{1^2 + 1^2} = \sqrt{2}$ and an inner product of 0 ($\theta = 90^\circ$). (b) The resulting Gramian matrix is the identity I_V , treating related concepts like "cat" and "dog" with the exact same geometric distance as totally unrelated concepts like "cat" and "automobile".

3. Zero Gradient Parameter Sharing During Backpropagation. Because one-hot vectors activate strictly disjoint coordinates, training a neural network on an occurrence of token w_i produces gradients exclusively for the parameters associated with index i :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{k,:}} = 0, \quad \forall k \neq i. \quad (3.19)$$

If a model updates its weights upon observing the sentence “*The feline climbed the tree*”, the parameters associated with “feline” receive gradient updates, but the parameters for “cat”, “kitten”, or “leopard” remain completely unchanged. The model cannot transfer statistical strength across semantically synonymous or related words; it must encounter every word hundreds of times independently in every grammatical context.

4. Simplex Confinement and Absence of Latent Manifolds. One-hot vectors are confined strictly to the V extreme vertices of the standard $(V - 1)$ -dimensional probability simplex:

$$\Delta^{V-1} = \left\{ \mathbf{x} \in \mathbb{R}^V : \sum_{i=1}^V x_i = 1, x_i \geq 0 \right\}. \quad (3.20)$$

The interior and edges of the simplex correspond to fractional probabilistic mixtures, but distinct words themselves inhabit isolated, non-overlapping boundary corners. Words cannot reside on smooth, lower-dimensional semantic submanifolds, nor can they support linear translation arithmetic ($\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$).

5. Software Engineering Identifier Polymorphism. In software engineering source code, developers routinely express identical semantic intents through differing lexical notations, identifier casings, and API conventions:

- **Casing variants:** `getUserById` vs. `get_user_by_id` vs. `GetUserById`.
- **API synonyms:** `push()` vs. `append()` vs. `add()` vs. `enqueue()`.
- **Syntactic equivalents:** Type references such as `ArrayList` and `LinkedList` vs. unrelated punctuation symbols like “;”.

Under one-hot encoding, each casing variation and API synonym is assigned a totally independent, mutually orthogonal basis vector. To a one-hot model:

$$\cos(\mathbf{e}_{\text{get_user_by_id}}, \mathbf{e}_{\text{getUserById}}) = 0, \quad d_E(\mathbf{e}_{\text{ArrayList}}, \mathbf{e}_{\text{LinkedList}}) = d_E(\mathbf{e}_{\text{ArrayList}}, \mathbf{e}_{\text{";"}}) = \sqrt{2}. \quad (3.21)$$

This lexical fragmentation completely prevents code-intelligence models from transferring refactoring patterns, bug-fix templates, or type semantics across codebases.

Because local one-hot encodings confine words to isolated, orthogonal coordinate axes where semantic similarity is geometrically undefinable, we require a fundamentally different representational paradigm. This paradigm shift rests on distributing linguistic meaning across continuous latent dimensions—guided by the foundational Distributional Hypothesis.

3.3 Distributed representations and the distributional hypothesis

To overcome the orthogonality and isolation of local representations, modern deep learning adopts **distributed representations**, a concept first formalized in cognitive science and connectionist neural networks by Geoffrey Hinton and colleagues in 1986 [27].

Definition 3.2: *Local vs. distributed representations*

- **Local Representation:** Each concept or symbol is represented by a single dedicated computational unit (e.g., one coordinate axis in $\mathbb{R}^V$). If that unit is inactive, the concept is entirely absent. There is zero overlap between distinct symbols.
- **Distributed Representation:** Each concept is represented by a continuous pattern of activation across all d dimensions of a shared vector space $\mathbb{R}^d$ ($d \ll V$). Conversely, each individual dimension participates in representing many distinct concepts.

In a distributed representation, meaning is not stored in an isolated slot; rather, it emerges from the continuous coordinates along latent dimensions. For example, latent dimensions might implicitly reflect attributes such as animacy, size, sentiment, syntactic part of speech, or software typing constructs.

3.3.1 The distributional hypothesis: context as meaning

How can an optimization algorithm discover meaningful continuous coordinates $\mathbf{v}_w \in \mathbb{R}^d$ for every word purely from raw text, without human supervision? The answer lies in the **Distributional Hypothesis**, formulated in structural linguistics by Zellig Harris (1954) [33] and popularized by John Rupert Firth (1957) [34]:

“You shall know a word by the company it keeps.”

— John Rupert Firth (1957) [34]

The distributional hypothesis posits that words that appear in similar linguistic contexts tend to share similar semantic and pragmatic meanings. Consider two sentences:

Sentence 1: “I used a **pencil** to write the essay.”

Sentence 2: “You used my **pen** to write a letter.”

Even if a machine has never encountered a dictionary definition of “pencil” or “pen”, it observes that both tokens co-occur with identical context words: “used”, “write”, “essay”, “letter”. Conversely, an unrelated word such as “crocodile” or “refrigerator” almost never occurs in the context of “used a [...] to write”.

Therefore, statistical co-occurrence within local contextual windows serves as an empirical, computable proxy for semantic similarity. If a learning algorithm is tasked with predicting surrounding context words from a target word (or vice versa), the learned vector coordinates must position words with similar co-occurrence profiles close together in vector space.

While the Distributional Hypothesis asserts that context defines meaning, it does not specify an optimization algorithm. To translate linguistic co-occurrence into optimized vector coordinates, we must frame a self-supervised prediction task and establish an end-to-end differentiable training loop.

3.4 The embedding training loop and semantic geometry evolution

To convert the distributional hypothesis into concrete numerical representations, we formulate a **self-supervised prediction cloze task** from unannotated text corpora.

3.4.1 The 7-step self-supervised optimization cycle

Given a raw, unannotated corpus of T running words $C = (w_1, w_2, \dots, w_T)$, we slide a context window of half-width C across the text. At each step t , the model solves a fill-in-the-blank prediction task: estimating the probability distribution over the vocabulary for a missing token given its surrounding context:

$$\mathbb{P}(w_t = w \mid w_{t-C}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+C}). \quad (3.22)$$

As presented in Prof. Flavio Giobergia's lecture slides, this prediction task is executed through an iterative **7-step optimization cycle**:

1. **Initialize Parameter Matrices:** Allocate two weight matrices with small random values: the input embedding matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ and the output embedding matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$.
2. **Context Lookup and Aggregation:** For the active context window, retrieve the input embedding vectors $\mathbf{v}_c = (\mathbf{W}_{\text{in}})_{:,c}$ and aggregate them into a single hidden context vector $\mathbf{h} \in \mathbb{R}^d$ (e.g., via vector summation or averaging).
3. **Output Candidate Projection:** Compute the dot product between the hidden context state $\mathbf{h}$ and every candidate output word vector $\mathbf{u}_w = (\mathbf{W}_{\text{out}})_{:,w}$ in the vocabulary:

$$z_w = \mathbf{u}_w^\top \mathbf{h}, \quad \text{yielding logit vector } \mathbf{z} = \mathbf{W}_{\text{out}}^\top \mathbf{h} \in \mathbb{R}^V. \quad (3.23)$$

4. **Probability Normalization:** Pass the similarity logits through the softmax activation function to produce a valid probability distribution $\hat{\mathbf{y}} \in \Delta^{V-1}$:

$$\hat{y}_w = \mathbb{P}(w \mid \text{context}) = \frac{\exp(z_w)}{\sum_{k=1}^V \exp(z_k)} = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^V \exp(\mathbf{u}_k^\top \mathbf{h})}. \quad (3.24)$$

5. **Loss Evaluation:** Evaluate the prediction error against the true ground-truth target token $w^* = w_t$ using Categorical Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = -\log \hat{y}_{w^*} = -\log \left(\frac{\exp(\mathbf{u}_{w^*}^\top \mathbf{h})}{\sum_{k=1}^V \exp(\mathbf{u}_k^\top \mathbf{h})} \right). \quad (3.25)$$

6. **Backpropagation Error Adjoints:** Apply reverse-mode automatic differentiation to compute error adjoints. The gradient with respect to logit vector $\mathbf{z}$ is the prediction residual $\delta = \hat{\mathbf{y}} - \mathbf{y} \in \mathbb{R}^V$, where $\mathbf{y} = \mathbf{e}_{w^*}$. Propagating this residual backward yields parameter gradients:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L} = \mathbf{h} \delta^\top \in \mathbb{R}^{d \times V}, \quad \nabla_{\mathbf{h}} \mathcal{L} = \mathbf{W}_{\text{out}} \delta \in \mathbb{R}^d, \quad \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L} = (\nabla_{\mathbf{h}} \mathcal{L}) \mathbf{e}_{\text{context}}^\top \in \mathbb{R}^{d \times V}. \quad (3.26)$$

7. **Stochastic Gradient Descent Parameter Update:** Adjust the coordinates in both embedding matrices by stepping along the negative gradient direction:

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} - \eta \nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}, \quad \mathbf{W}_{\text{in}} \leftarrow \mathbf{W}_{\text{in}} - \eta \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L}. \quad (3.27)$$

Figure 3.2 illustrates this complete learning mechanism, juxtaposing the forward/backward computational dataflow in Panel (a) with the resulting geometric evolution of the embedding space across training epochs in Panel (b).

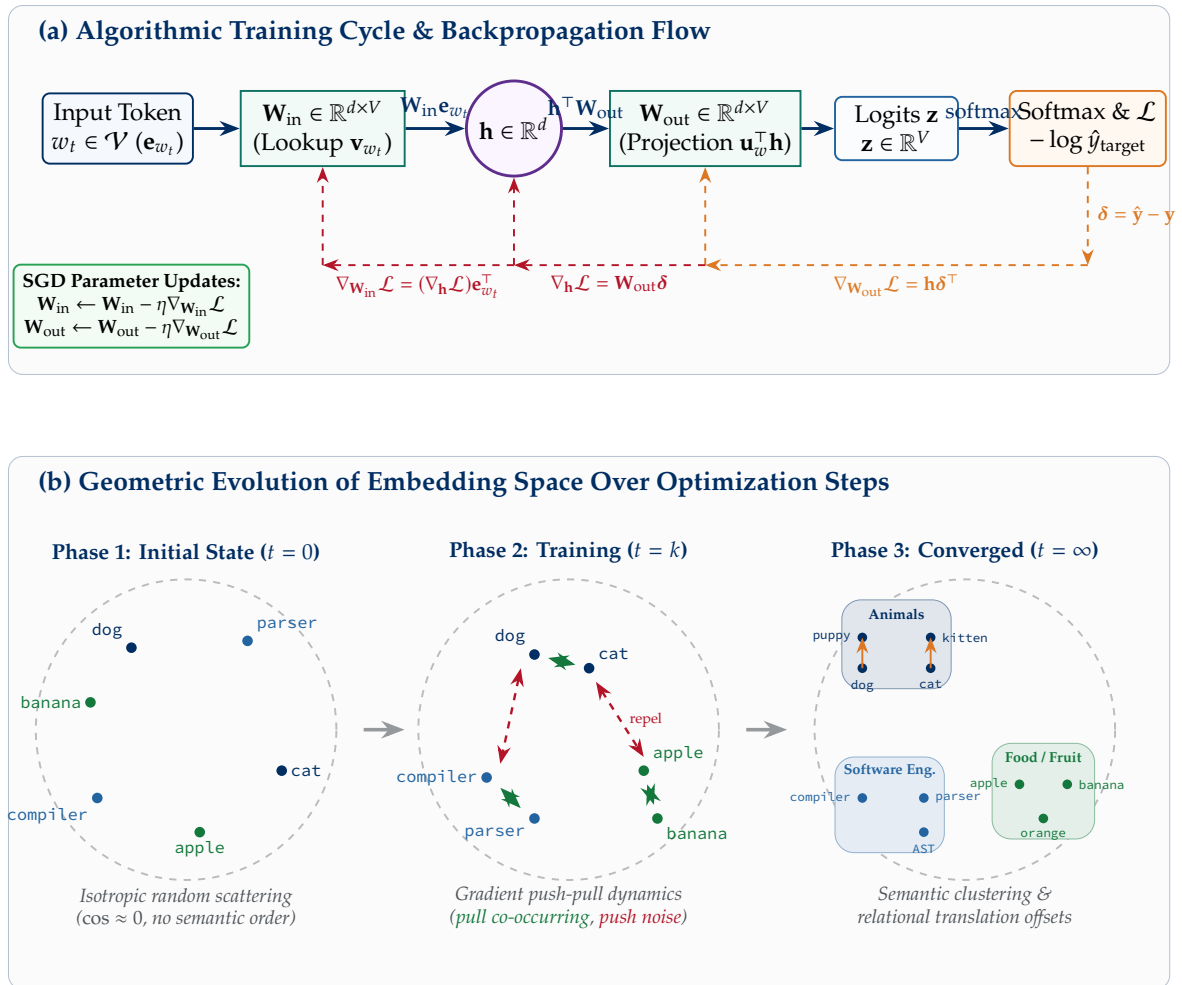


Figure 3.2: The embedding training loop and semantic space evolution. (a) Computational dataflow of the forward prediction pass and backward adjoint parameter updates. (b) Three-phase geometric evolution of embedding space from isotropic random scattering to clustered semantic topology with parallel relational offsets.

3.4.2 Geometric phase transition of the embedding space

As shown in Panel (b) of Figure 3.2, the continuous embedding space undergoes a profound topological self-organization over the course of training:

- **Phase 1: Initial State ($t = 0$).** Vectors are initialized using small Gaussian noise, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$ with $\sigma \approx 0.01$ or $\sigma \approx 1/\sqrt{d}$. In high dimensions ($d \geq 100$), randomly sampled vectors are almost exactly orthogonal ($\cos \approx 0$). Tokens from disparate semantic domains—"dog", "compiler", "apple"—are scattered isotropically across the unit sphere with zero structural order.
- **Phase 2: Training Dynamics ($t = k$).** Whenever the model processes an observed sentence (e.g., "*The compiler parsed the syntax tree*"), backpropagation produces an attractive gradient force pulling the vectors of co-occurring words closer together. Concurrently, negative gradients or normalizer updates exert repulsive forces, pushing non-co-occurring words apart.
- **Phase 3: Converged Semantic Topology ($t = \infty$).** Upon convergence across millions of sentences, words coalesce into dense semantic neighborhoods (e.g., domestic animals, compiler construction terms, fruits). Remarkably, linear vector differences between related concepts become parallel across distinct pairs, giving rise to relational vector arithmetic (e.g., $\mathbf{v}_{\text{dog}} - \mathbf{v}_{\text{puppy}} \approx \mathbf{v}_{\text{cat}} - \mathbf{v}_{\text{kitten}}$).

3.4.3 Initialization dynamics and the hockey-stick loss curve

In his educational analysis of neural language models and lookup tables [35], Andrej Karpathy notes that the scale of initial weights in $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{out}}$ dictates optimization stability. At initialization, before the network has learned any linguistic patterns, the model should assign an approximately uniform probability to every word in the vocabulary:

$$\hat{y}_w \approx \frac{1}{V}, \quad \forall w \in \mathcal{V}. \quad (3.28)$$

Consequently, the theoretical expected initial cross-entropy loss is strictly:

$$\mathcal{L}_{\text{init}} = -\log\left(\frac{1}{V}\right) = \log V. \quad (3.29)$$

For a vocabulary of $V = 50,000$, the initial loss must be $\mathcal{L}_{\text{init}} = \ln(50,000) \approx 10.82$. If $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$ are initialized with excessively large variance, logits $z_w = \mathbf{u}_w^\top \mathbf{h}$ take extreme values; the softmax distribution becomes peaky and confident on arbitrary words, causing the initial loss to spike drastically ($\mathcal{L} \gg 20$). The optimizer must then spend thousands of iterations flattening these unearned confidences—producing a pathological “hockey-stick” loss curve. Proper variance scaling ($\mathcal{N}(0, 0.01^2)$ or $\mathcal{N}(0, 1/d)$) ensures stable, well-conditioned gradient flow from step zero.

With the general self-supervised training cycle established, we must concretize the network architecture. The first foundational architecture proposed by Mikolov et al. is the Continuous Bag-of-Words (CBOW) model, which aggregates surrounding context words to predict an omitted target center word.

3.5 Word2Vec continuous bag-of-words (CBOW)

In 2013, Tomas Mikolov and his research team at Google [8, 9] introduced **Word2Vec**, a suite of log-linear models designed to scale distributed representation learning to web-scale corpora containing billions of tokens.

The first of these architectures is the **Continuous Bag-of-Words (CBOW)** model. In CBOW, the objective is to predict the omitted center target token w_t given the symmetric surrounding context window:

$$C_t = \{w_{t-C}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+C}\} \setminus \{w_t\}. \quad (3.30)$$

3.5.1 Matrix formulation and tensor plumbing

The network consists of two weight matrices:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$: The input embedding matrix, where each column $(\mathbf{W}_{\text{in}})_{:,k} = \mathbf{v}_k \in \mathbb{R}^d$ represents the vector of token k when acting as context.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$: The output embedding matrix, where each column $(\mathbf{W}_{\text{out}})_{:,k} = \mathbf{u}_k \in \mathbb{R}^d$ represents the vector of token k when acting as the target.

Let $\mathbf{e} \in \{0, 1\}^V$ denote the multi-hot context vector indicating the presence of words in C_t :

$$e_k = \sum_{j \in \{-C, \dots, C\} \setminus \{0\}} \mathbf{1}_{\{w_{t+j}=w_k\}}. \quad (3.31)$$

The hidden state $\mathbf{h} \in \mathbb{R}^d$ is obtained by projecting the context vector through $\mathbf{W}_{\text{in}}$ and averaging over the $2C$ context positions:

$$\mathbf{h} = \frac{1}{2C} \mathbf{W}_{\text{in}} \mathbf{e} = \frac{1}{2C} \sum_{\substack{-C \leq j \leq C \\ j \neq 0}} \mathbf{v}_{w_{t+j}} \in \mathbb{R}^d. \quad (3.32)$$

Next, the model scores candidate target words by projecting $\mathbf{h}$ against the columns of $\mathbf{W}_{\text{out}}$:

$$\hat{\mathbf{p}} = \mathbf{h}^\top \mathbf{W}_{\text{out}} \in \mathbb{R}^{1 \times V}, \quad \text{where } \hat{p}_k = \mathbf{u}_k^\top \mathbf{h}. \quad (3.33)$$

As Grant Sanderson (3Blue1Brown) emphasizes in his geometric intuitions of neural representations [36], the inner product $\mathbf{u}_k^\top \mathbf{h} = \|\mathbf{u}_k\|_2 \|\mathbf{h}\|_2 \cos \theta$ acts as a directional alignment sensor: it evaluates how closely the target word's representation points in the same direction as the accumulated context vector.

The raw similarity logits $\hat{\mathbf{p}}$ are normalized via the softmax function to form the conditional probability distribution:

$$\mathbb{P}(w_t = w \mid C_t) = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^V \exp(\mathbf{u}_k^\top \mathbf{h})}. \quad (3.34)$$

The model is trained by minimizing the negative log-likelihood of the ground-truth target word $w^* = w_t$:

$$\mathcal{L}_{\text{CBOW}} = -\log \mathbb{P}(w_t = w^* \mid C_t) = -\mathbf{u}_{w^*}^\top \mathbf{h} + \log \sum_{k=1}^V \exp(\mathbf{u}_k^\top \mathbf{h}). \quad (3.35)$$

Figure 3.3 illustrates this computational flow.

3.5.2 Bengio (2003) vs. Mikolov (2013): the log-linear breakthrough

A neural language model learning distributed embeddings had been introduced a decade prior by Yoshua Bengio and colleagues in 2003 [7]. However, Bengio's architecture had two major differences:

1. **Strict Causal Prediction:** Bengio modeled language autoregressively, predicting w_t strictly from past tokens $w_{t-n+1}, \dots, w_{t-1}$. Mikolov opened the window symmetrically to both left and right context tokens $(w_{t-C}, \dots, w_{t+C})$, doubling the statistical signal.

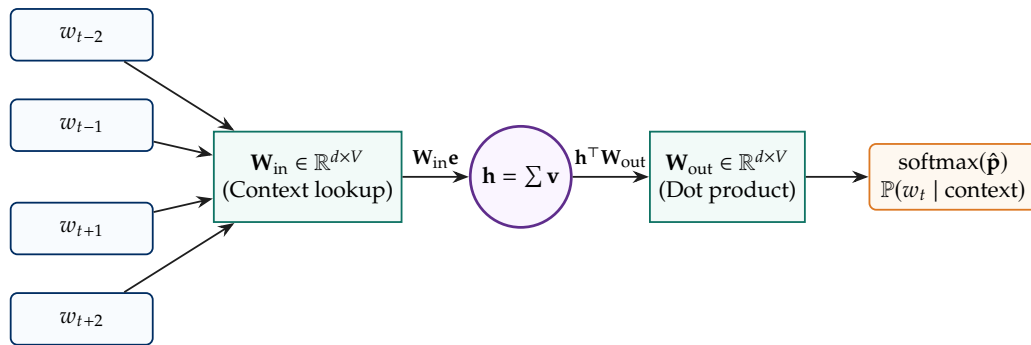


Figure 3.3: Continuous Bag-of-Words (CBOW) architecture. Context words $w_{t-c}, \dots, w_{t+c}$ are looked up in $\mathbf{W}_{\text{in}}$ and summed to form context state $\mathbf{h} \in \mathbb{R}^d$. Inner products with output matrix $\mathbf{W}_{\text{out}}$ yield similarity logits normalized via softmax to predict the center target token w_t .

2. **Elimination of the Non-Linear Hidden Layer:** Bengio passed concatenated context embeddings through a non-linear dense hidden layer $\tanh(\mathbf{W}_h \mathbf{x} + \mathbf{b})$. The matrix multiplication with $\mathbf{W}_h$ dominated computation time ($\mathcal{O}(C \cdot d \cdot d_h)$). Mikolov stripped away the non-linear activation entirely, using a direct linear projection (vector averaging). This made Word2Vec a log-linear model, reducing compute costs by orders of magnitude and enabling training on billions of words in a few hours.

3.5.3 Strengths and representational bottlenecks of CBOW

Strengths:

- **Rapid Training Convergence:** Context averaging smooths the gradient signal, allowing CBOW to converge in fewer epochs over large corpora.
- **Robust Representations for Frequent Words:** For high-frequency tokens with abundant contextual observations, the smoothed averaging stabilizes coordinate estimates against noisy outliers.

Representational Bottlenecks:

- **Loss of Sequential Word Order (Syntactic Blindness):** Vector summation in Equation (3.32) is strictly commutative: $\mathbf{v}_a + \mathbf{v}_b = \mathbf{v}_b + \mathbf{v}_a$. The model is incapable of distinguishing “the dog bit the man” from “the man bit the dog”.
- **Rare Word Dilution:** Because context words are averaged into a single vector $\mathbf{h}$, if a rare technical token appears alongside multiple high-frequency stopwords (e.g., “the *polymorphism* in the”), the rare token’s embedding is diluted by the dominant stopwords: $\frac{1}{2C} \mathbf{v}_{\text{rare}} \approx \mathbf{0}$. The gradient update to the rare word is severely attenuated.

Although CBOW trains rapidly by averaging context representations, this very averaging creates a representational bottleneck: it washes out rare vocabulary tokens and discards syntactic word order. To learn high-fidelity representations for infrequent words, Mikolov et al. inverted the formulation, yielding the Continuous Skip-Gram architecture.

3.6 Word2Vec continuous skip-gram and the softmax bottleneck

To preserve the distinct gradient signal of rare words, Mikolov et al. [8] introduced the **Continuous Skip-Gram** architecture. Instead of aggregating context words to predict a single center token,

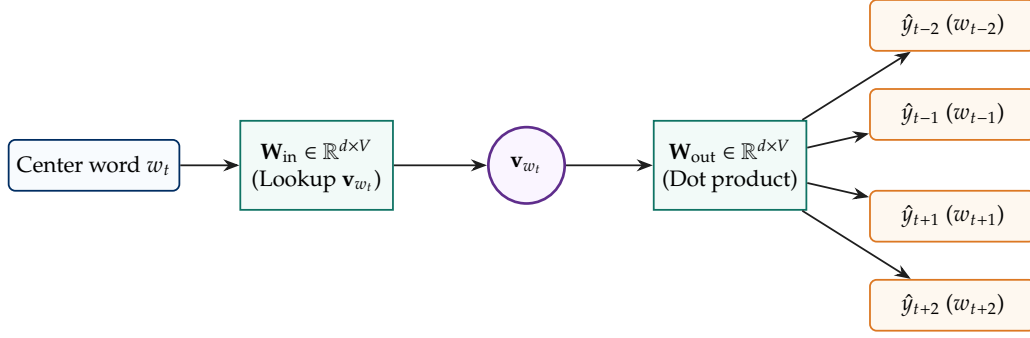


Figure 3.4: Continuous Skip-Gram architecture. A single center target word w_t is projected via $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ into vector $\mathbf{v}_{w_t} \in \mathbb{R}^d$, which predicts the probability distribution of context words across window radius c through output matrix $\mathbf{W}_{\text{out}}$.

Skip-Gram **inverts the predictive direction**: given a single center target word w_t , the model independently predicts each surrounding context word within window radius C .

Definition 3.3: *Skip-gram objective function*

Given a training sequence of T words $w_1, w_2, \dots, w_T$ and a context window of half-width C , the Skip-gram model maximizes the average log-likelihood:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^T \sum_{\substack{-C \leq j \leq C \\ j \neq 0}} \log \mathbb{P}(w_{t+j} | w_t). \quad (3.36)$$

Under standard multinomial softmax, the conditional probability of context word w_O given center word w_I is:

$$\mathbb{P}(w_O | w_I) = \frac{\exp(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I})}{\sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_{w_I})}, \quad (3.37)$$

where $\mathbf{v}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{in}}$ (input embedding) and $\mathbf{u}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{out}}$ (output context embedding).

3.6.1 Why Skip-gram captures rare words

In Skip-Gram, context words are never averaged together. Instead, every context-target co-occurrence (w_I, w_O) generates an independent, dedicated pairwise training instance. When an infrequent, domain-specific term (such as "refactoring" or "polymorphism") appears in text, its vector $\mathbf{v}_{w_I}$ directly predicts each context word:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = \sum_{w_O \in C} \left(\mathbf{u}_{w_O} - \sum_{k=1}^V \mathbb{P}(w_k | w_I) \mathbf{u}_k \right). \quad (3.38)$$

The gradient is never diluted by surrounding words. Consequently, Skip-Gram produces significantly higher-quality embeddings for rare words, specialized technical jargon, and nuanced semantic distinctions than CBOW.

3.6.2 The shared computational wall: the full softmax bottleneck

Despite its empirical superiority, the standard formulation of Skip-Gram crashes into an insurmountable computational wall: **the full Softmax partition function bottleneck**.

Examining the denominator of Equation (3.37):

$$Z(w_I) = \sum_{w=1}^V \exp(\mathbf{u}_w^\top \mathbf{v}_{w_I}). \quad (3.39)$$

Computing this normalizing denominator requires evaluating an inner product $\mathbf{u}_w^\top \mathbf{v}_{w_I}$ and an exponential $\exp(\cdot)$ for **every single token w in the entire vocabulary $\mathcal{V}$** .

- **Computational Complexity per Token:** Computing $Z(w_I)$ takes $O(V \cdot d)$ floating-point operations.
- **Total Corpus Cost:** For a window of radius $C = 5$, predicting $2C = 10$ context words per token over a corpus of $T = 10^9$ words with vocabulary $V = 10^5$ and dimension $d = 300$ requires:

$$10^9 \text{ tokens} \times 10 \text{ predictions} \times 10^5 \text{ words} \times 300 \text{ dims} = 3 \times 10^{17} \text{ FLOPs}. \quad (3.40)$$

- **Gradient Synchronization Bottleneck:** In backpropagation, the derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{u}_w}$ touches every column $w \in \{1, \dots, V\}$ of $\mathbf{W}_{\text{out}}$ on every single training step, forcing full-matrix memory writes.

This $O(V \cdot d)$ complexity renders standard multinomial softmax completely intractable for web-scale corpora.

While Continuous Skip-Gram excels at capturing nuanced semantic relationships and rare tokens, its standard formulation crashes into a severe computational barrier: normalizing over the full vocabulary requires an $O(V \cdot d)$ denominator calculation for every single token prediction. To scale to realistic vocabularies, we must circumvent this partition function bottleneck.

3.7 Scaling softmax: hierarchical softmax and negative sampling

To bypass the $O(V \cdot d)$ partition function barrier, Mikolov et al. [9] introduced two distinct scaling methods: **Hierarchical Softmax** (a structured tree-based approximation) and **Skip-Gram with Negative Sampling (SGNS)** (a sample-based binary discrimination formulation).

3.7.1 Hierarchical softmax ($O(\log_2 V)$)

First developed by Frederic Morin and Yoshua Bengio in 2005 [37] and adapted by Mikolov et al. [9], **Hierarchical Softmax** decomposes the flat multiclass probability distribution into a hierarchical sequence of binary routing decisions over a binary tree.

- **Binary Tree Topology:** The V vocabulary words form the leaves of a binary tree (constructed as a binary Huffman tree where frequent words have shorter code lengths).
- **Path Specification:** For any word $w \in \mathcal{V}$, let $L(w)$ denote the depth of the leaf from the root. There exists a unique path of nodes from the root node $n(w, 1) = \text{root}$ to the leaf node $n(w, L(w)) = w$.
- **Internal Node Parameters:** Each internal node $j \in \{1, \dots, L(w) - 1\}$ is assigned a continuous parameter vector $\mathbf{v}'_{n(w,j)} \in \mathbb{R}^d$.

- **Binary Branching Probability:** At each internal node, the probability of branching to the left child is parameterized by the logistic sigmoid function:

$$\mathbb{P}(\text{branch left} \mid \mathbf{v}_{w_I}) = \sigma \left(\mathbf{v}'_{n(w,j)}{}^\top \mathbf{v}_{w_I} \right) = \frac{1}{1 + \exp \left(-\mathbf{v}'_{n(w,j)}{}^\top \mathbf{v}_{w_I} \right)}. \quad (3.41)$$

The probability of branching to the right child is:

$$\mathbb{P}(\text{branch right} \mid \mathbf{v}_{w_I}) = 1 - \sigma \left(\mathbf{v}'_{n(w,j)}{}^\top \mathbf{v}_{w_I} \right) = \sigma \left(-\mathbf{v}'_{n(w,j)}{}^\top \mathbf{v}_{w_I} \right). \quad (3.42)$$

- **Path Probability Product:** The conditional probability of reaching leaf word w is the product of binary decision probabilities along its unique path:

$$\mathbb{P}(w \mid w_I) = \prod_{j=1}^{L(w)-1} \sigma \left(\text{sign}(w, j) \cdot \mathbf{v}'_{n(w,j)}{}^\top \mathbf{v}_{w_I} \right), \quad (3.43)$$

where $\text{sign}(w, j) = +1$ if the path branches left at node j , and -1 if it branches right.

Proof of Exact Normalization. Because probabilities sum to 1 at each binary branching step ($\sigma(z) + \sigma(-z) = 1$), telescoping products guarantee that the sum of probabilities over all leaves equals 1 identically:

$$\sum_{w \in \mathcal{V}} \mathbb{P}(w \mid w_I) \equiv 1. \quad (3.44)$$

No global partition function $Z(w_I)$ ever needs to be computed!

Complexity Reduction. Evaluating $\mathbb{P}(w \mid w_I)$ and its gradients requires traversing only the $L(w) - 1$ nodes along the path. In a balanced or Huffman binary tree, the average path length is $\mathcal{O}(\log_2 V)$. For a vocabulary of $V = 100,000$:

$$\log_2(100,000) \approx 16.6 \implies \frac{100,000}{17} \approx 5,880 \times \text{acceleration}. \quad (3.45)$$

3.7.2 Skip-gram negative sampling (SGNS)

While Hierarchical Softmax scales to $\mathcal{O}(\log_2 V)$, Mikolov et al. [9] introduced an even more efficient and empirically superior alternative: **Negative Sampling (SGNS)**, derived as a simplified approximation to Noise Contrastive Estimation (NCE).

Instead of treating context prediction as a massive multinomial classification task over V mutually exclusive categories, SGNS reformulates the problem as $1 + K$ **independent binary logistic classifications**:

1. **Positive Sample** ($y = 1$): The true center-context pair (w_c, w_o) observed in the text corpus. The model maximizes the probability that this pair is a true co-occurrence.
2. **Negative Samples** ($y = 0$): K noise words $\{w_{n_1}, w_{n_2}, \dots, w_{n_K}\}$ drawn at random from a noise distribution $P_n(w)$. The model maximizes the probability that these randomly sampled pairs are synthetic noise.

Definition 3.4: *Skip-gram negative sampling objective*

For an observed center-context token pair (w_c, w_o) and K negative samples $\{w_{n_1}, \dots, w_{n_K}\} \sim P_n(w)$, the SGNS loss is:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - \sum_{k=1}^K \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}), \quad (3.46)$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ denotes the standard logistic sigmoid function.

3.7.3 Step-by-step display math analytical gradient derivations

To understand how SGNS shapes the geometry of the embedding space, we derive the exact analytical gradients of $\mathcal{L}_{\text{SGNS}}$ with respect to all parameter vectors.

Recall the fundamental identities for the logistic sigmoid derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \quad (3.47)$$

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z), \quad (3.48)$$

$$\frac{d}{dz} \log \sigma(-z) = \frac{-\sigma'(-z)}{\sigma(-z)} = \frac{-\sigma(-z)(1 - \sigma(-z))}{\sigma(-z)} = -(1 - \sigma(-z)) = -\sigma(z). \quad (3.49)$$

1. Gradient with respect to Positive Target Vector $\mathbf{u}_{w_o}$. The positive target vector $\mathbf{u}_{w_o}$ appears exclusively in the first term of Equation (3.46):

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} &= -\frac{\partial}{\partial \mathbf{u}_{w_o}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \\ &= -[1 - \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})] \frac{\partial(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_o}} \\ &= [\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1] \mathbf{v}_{w_c}. \end{aligned} \quad (3.50)$$

Defining the positive prediction error as $e_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1 = \hat{y}_{\text{pos}} - 1$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} = (\hat{y}_{\text{pos}} - 1) \mathbf{v}_{w_c}. \quad (3.51)$$

2. Gradient with respect to Negative Noise Vector $\mathbf{u}_{w_{n_k}}$. Each negative sample vector $\mathbf{u}_{w_{n_k}}$ appears in exactly one term of the summation in Equation (3.46):

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} &= -\frac{\partial}{\partial \mathbf{u}_{w_{n_k}}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= -[-\sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})] \frac{\partial(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_{n_k}}} \\ &= \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \mathbf{v}_{w_c}. \end{aligned} \quad (3.52)$$

Defining the negative prediction error as $e_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) - 0 = \hat{y}_{\text{neg},k}$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} = \hat{y}_{\text{neg},k} \mathbf{v}_{w_c}. \quad (3.53)$$

3. Gradient with respect to Center Word Vector $\mathbf{v}_{w_c}$. The center word vector $\mathbf{v}_{w_c}$ appears in both the positive term and all K negative terms. Applying the chain rule to the complete loss:

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{v}_{w_c}} &= -\frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - \sum_{k=1}^K \frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= \underbrace{\left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1 \right] \mathbf{u}_{w_o}}_{\text{Attractive Component (Pull Context)}} + \underbrace{\sum_{k=1}^K \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \mathbf{u}_{w_{n_k}}}_{\text{Repulsive Component (Push Noise)}}. \end{aligned} \quad (3.54)$$

3.7.4 Physical vector mechanics interpretation

Under gradient descent, parameter updates subtract the gradient: $\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} - \eta \nabla_{\mathbf{v}_{w_c}} \mathcal{L}$:

$$\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} + \eta (1 - \hat{y}_{\text{pos}}) \mathbf{u}_{w_o} - \eta \sum_{k=1}^K \hat{y}_{\text{neg},k} \mathbf{u}_{w_{n_k}}. \quad (3.55)$$

This update formula admits a clean, intuitive **physical vector mechanics** interpretation:

- **The Attractive Pull Force:** Since $\hat{y}_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \in (0, 1)$, the coefficient $(1 - \hat{y}_{\text{pos}})$ is strictly positive. The update adds a vector component in the direction $+\mathbf{u}_{w_o}$, **pulling the center word vector toward the true context vector**. If the model is already confident ($\hat{y}_{\text{pos}} \approx 1$), the force vanishes; if the model is surprised ($\hat{y}_{\text{pos}} \approx 0$), the force reaches maximum strength $+1 \cdot \mathbf{u}_{w_o}$.
- **The Repulsive Push Force:** For every negative sample k , $\hat{y}_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \in (0, 1)$ is strictly positive. The update subtracts a vector component along each noise vector $-\mathbf{u}_{w_{n_k}}$, **pushing the center word vector away from noise words**. If the noise word is far away ($\hat{y}_{\text{neg},k} \approx 0$), no force is exerted; if the noise word is erroneously close ($\hat{y}_{\text{neg},k} \approx 1$), a strong repulsive kick $-\mathbf{u}_{w_{n_k}}$ repels the points.

Crucially, the computational complexity of evaluating Equation (3.55) is $O(K \cdot d)$, which is **entirely independent of vocabulary size V** . In practice, choosing $K \in [5, 20]$ for small corpora and $K \in [2, 5]$ for web-scale corpora provides sufficient negative contrast, reducing training time from days to hours.

3.7.5 Theoretical justification of the 3/4 unigram noise exponent

How should negative samples be drawn? Mikolov et al. [9] define the noise distribution $P_n(w)$ as a smoothed unigram distribution raised to the power of $\alpha = 3/4 = 0.75$:

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w' \in \mathcal{V}} f(w')^{3/4}}, \quad (3.56)$$

where $f(w)$ denotes the raw unigram occurrence frequency of token w in the corpus.

Why $\alpha = 0.75$ instead of $\alpha = 1.0$ (Raw Frequency) or $\alpha = 0.0$ (Uniform)?

1. **Failure of Raw Frequency ($\alpha = 1.0$):** Natural language word frequencies follow Zipf's law ($f(r) \propto 1/r$). A tiny fraction of stopwords ("the", "of", "and", "to") account for over 50% of all token occurrences in a typical corpus. If $\alpha = 1.0$, more than half of all negative samples drawn are uninformative stopwords. The model wastes its entire discriminative budget separating words from "the", while rare content words ("polymorphism", "compiler") have near-zero probability of ever being selected as negative samples, receiving virtually zero negative gradient updates.

2. **Failure of Uniform Sampling** ($\alpha = 0.0$): Under a uniform distribution $P_n(w) = 1/V$, every vocabulary word has equal probability. Because the vast majority of words in any natural vocabulary reside in the long tail of rare words, sampling uniformly selects almost exclusively rare words as negatives. The network rarely practices distinguishing moderately frequent content words from one another.
3. **The Sub-linear Compromise** ($\alpha = 0.75$): The exponent $3/4$ acts as a concave, sub-linear dampener. It compresses the gap between ultra-frequent stopwords and rare content words.

Numerical Demonstration of the Rare-Word Boost. Consider a frequent stopword w_{freq} with corpus frequency $f(w_{\text{freq}}) = 0.01$ (e.g., 10^6 occurrences in 10^8 words) and a rare technical term w_{rare} with corpus frequency $f(w_{\text{rare}}) = 10^{-6}$ (e.g., 100 occurrences in 10^8 words):

- Under raw unigram frequency ($\alpha = 1.0$), the sampling ratio is:

$$\frac{f(w_{\text{freq}})}{f(w_{\text{rare}})} = \frac{10^{-2}}{10^{-6}} = 10,000. \quad (3.57)$$

The stopword is sampled 10,000 times more frequently than the rare word.

- Under the smoothed distribution ($\alpha = 0.75$), the ratio becomes:

$$\frac{f(w_{\text{freq}})^{0.75}}{f(w_{\text{rare}})^{0.75}} = \left(\frac{10^{-2}}{10^{-6}} \right)^{0.75} = (10,000)^{0.75} = (10^4)^{3/4} = 10^3 = 1,000. \quad (3.58)$$

By applying the $3/4$ power, the relative probability of sampling the rare word is amplified by a factor of:

$$\frac{10,000}{1,000} = 10\times. \quad (3.59)$$

This $10\times$ boost provides rare words with sufficient negative gradient pushes to carve out accurate decision boundaries, while still reflecting broad corpus statistics.

3.7.6 Global count matrix factorization: GloVe (Pennington et al., 2014)

Around the same time, Jeffrey Pennington, Richard Socher, and Christopher Manning (2014) introduced **GloVe (Global Vectors for Word Representation)** [38]. Whereas Word2Vec is a local window-based predictive model that processes streaming word pairs, GloVe demonstrated that distributed embeddings could be trained by directly factorizing a **global co-occurrence matrix** $\mathbf{X} \in \mathbb{R}^{V \times V}$, where X_{ij} counts the number of times word j appears in the context window of word i .

GloVe's core insight is that the semantic distinction between two words i and j relative to a probe word k is encoded not by raw probabilities $P_{ik} = X_{ik}/X_i$, but by the **ratio of co-occurrence probabilities**:

$$\frac{P_{ik}}{P_{jk}} = \frac{P(k | i)}{P(k | j)}. \quad (3.60)$$

Table 3.1 reproduces the seminal example from Pennington et al. [38] comparing $i = \text{"ice"}$ and $j = \text{"steam"}$ across probe words k .

When $k = \text{"solid"}$ is strongly related to "ice" but not "steam", the ratio is large ($8.9 \gg 1$). When $k = \text{"gas"}$ is related to "steam" but not "ice", the ratio is tiny ($0.085 \ll 1$). When $k = \text{"water"}$ is equally related to both (or $k = \text{"fashion"}$ is related to neither), the ratio hovers near 1.0.

Table 3.1: Co-occurrence probability ratios illustrating semantic discrimination in GloVe [38].

Probability & Ratio	$k = \text{"solid"}$	$k = \text{"gas"}$	$k = \text{"water"}$	$k = \text{"fashion"}$
$P(k \mid \text{"ice"})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid \text{"steam"})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \mid \text{"ice"})/P(k \mid \text{"steam"})$	8.9	0.085	1.36	0.96

GloVe enforces this relationship by fitting a log-bilinear model where dot products approximate log co-occurrence counts, minimized via weighted least squares:

$$J = \sum_{i=1}^V \sum_{j=1}^V f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \quad (3.61)$$

where $f(X_{ij}) = \min(1, (X_{ij}/x_{\max})^\alpha)$ is a sub-linear weighting function preventing frequent stopwords from dominating the loss. Both Word2Vec and GloVe construct semantically structured vector spaces, with SGNS operating on streaming text and GloVe operating on aggregated count statistics.

Skip-Gram Negative Sampling and GloVe resolve the computational bottleneck, making large-scale distributed representation learning feasible. However, Word2Vec operates under an implicit architectural assumption: that the word is the minimal, indivisible atomic unit of language. In real-world deployment, this assumption creates severe structural vulnerabilities.

3.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier

Despite their revolutionary impact, classical Word2Vec and GloVe architectures suffer from fundamental linguistic and software engineering limitations.

3.8.1 The atomic word token assumption and out-of-vocabulary (OOV) failure

Word2Vec models assume a closed, fixed vocabulary $\mathcal{V} = \{w_1, \dots, w_V\}$ established during training. Each word is treated as an indivisible, atomic entity: its character composition is completely ignored. This creates the **Out-of-Vocabulary (OOV) catastrophe**:

- **Unseen Words at Test Time:** In open-domain text and evolving codebases, users constantly produce novel tokens: technical neologisms, composite identifiers (e.g., `AsyncHttpClientFactory`), misspelled words (e.g., "recursive"), and inflected stems.
- **Mapping to <UNK>:** When a trained Word2Vec model encounters an OOV word at inference, it has no entry in $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$. Deep learning pipelines are forced to map all OOV tokens to a generic unknown token placeholder:

$$w_{\text{OOV}} \mapsto \text{<UNK>} \implies \mathbf{v}_{w_{\text{OOV}}} = \mathbf{v}_{\text{<UNK>}}. \quad (3.62)$$

This discards all semantic identity. To Word2Vec, a typo like "algorithmmm" is as completely devoid of meaning as an alien string.

3.8.2 Morphological blindness

Word2Vec is entirely blind to internal word morphology. In natural languages with rich inflectional systems (e.g., Italian, German, Turkish) as well as programming languages, words

share sub-morphemic stems, prefixes, and suffixes:

Natural language: "develop", "developer", "developing", "undeveloped",
 Software identifiers: "parseFile", "parseTokens", "parseTree".

Under Word2Vec, each of these variants is assigned a totally disjoint, independent column in $\mathbf{W}_{in}$. The model possesses zero architectural mechanism to share parameters across words containing the common root "develop" or "parse". If "undeveloped" occurs only twice in the corpus, its vector remains poorly trained, unable to inherit semantic properties from the frequent token "develop".

3.8.3 Polysemy collapse: static context invariance

Word2Vec assigns exactly **one static vector** $\mathbf{v}_w \in \mathbb{R}^d$ **to each word string** w , regardless of the surrounding runtime context in which it appears. This results in **polysemy collapse**.

Definition 3.5: Polysemy collapse

Let word w possess M distinct semantic senses $S_1, \dots, S_M$ with respective corpus mixture proportions $\pi_1, \dots, \pi_M$ ($\sum \pi_m = 1$). Because Word2Vec minimizes prediction error across all occurrences using a single vector $\mathbf{v}_w$, the optimal vector converges to the convex combination of distinct sense centroids:

$$\mathbf{v}_w^* \approx \sum_{m=1}^M \pi_m \mathbf{v}_{S_m}. \quad (3.63)$$

For example, consider the polysemous word "bat":

Sense 1 (Zoological): "A **bat** is a nocturnal flying mammal."

Sense 2 (Sports Equipment): "The baseball player swung the wooden **bat**."

In Word2Vec, $\mathbf{v}_{bat}$ is pulled toward biological animals on some iterations, and toward sports equipment on others. The resulting point converges to an intermediate centroid in $\mathbb{R}^d$ that represents neither animal nor sports gear accurately, degrading cosine similarity to both senses.

In software engineering, polysemy is ubiquitous:

- "Python": A biological serpent vs. a high-level dynamic programming language.
- "Spring": A season of the year vs. a popular Java enterprise application framework.
- "rust": Iron oxide corrosion vs. a memory-safe systems programming language.

A static Word2Vec model maps "Rust" to a compromise vector that conflates oxidation chemistry with memory safety guarantees.

While contextual ambiguity requires sequence models that dynamically modulate representations based on full sentence context, the Out-of-Vocabulary barrier and morphological blindness can be solved at the token representation level. By abandoning the word as an atomic unit and descending into subword character n -grams, FastText establishes a morphologically aware embedding paradigm.

3.9 FastText: subword character n -gram embeddings

To conquer the OOV barrier and enable morphological parameter sharing, Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov (2017) at Facebook AI Research introduced **FastText** [39].

3.9.1 Character n -gram decomposition and boundary markers

Rather than treating a word as an atomic token, FastText represents each word as a **bag of character n -grams**.

1. **Boundary Enclosure:** To distinguish prefixes and suffixes from character sequences occurring in the interior of a word, each word w is bracketed by special boundary symbols "<" and ">":

$$\tilde{w} = "<" \circ w \circ ">". \quad (3.64)$$

This ensures that the prefix sequence "<re" (as in "<refactor>") is mathematically distinct from the internal substring "re" (as in "<where>").

2. **Subword Extraction:** All character n -grams of length $n \in [n_{\min}, n_{\max}]$ (typically $n \in [3, 6]$) are extracted, along with the special sequence containing the entire word itself $\tilde{w}$.

Course Slide Example: <where>. Following Slide 13 of Prof. Flavio Giobergia's lecture deck, consider the word "where" with character trigrams ($n = 3$):

$$"<where>" \implies \mathcal{G}_{\text{where}} = \{ "<wh", "whe", "her", "ere", "re>", "<where>" \}. \quad (3.65)$$

3.9.2 Vector sum composition

Each subword n -gram g is assigned a latent continuous embedding vector $\mathbf{z}_g \in \mathbb{R}^d$. The final representation of word w is defined as the **sum of its constituent subword vectors**:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g \in \mathbb{R}^d. \quad (3.66)$$

Under the Skip-Gram Negative Sampling objective, the similarity score between center word w and context word c becomes:

$$s(w, c) = \mathbf{v}_w^\top \mathbf{u}_c = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{u}_c. \quad (3.67)$$

During backpropagation, the prediction error with respect to $\mathbf{v}_w$ distributes gradients to every constituent subword vector $\mathbf{z}_g$.

3.9.3 Resolving OOV tokens dynamically and morphological sharing in code

This simple compositional formulation yields two decisive capabilities:

1. **Zero-Shot Synthesis of Out-of-Vocabulary Tokens:** When an unseen word w_{OOV} appears at test time (e.g., a compound identifier like "refactorable" or a code typo like "recursive"), even if the full token was never seen during training, its constituent character n -grams ("<re", "ref", "fact", "able>") were updated across other vocabulary words. FastText

synthesizes an accurate embedding vector on the fly by summing the vectors of its known subwords:

$$\mathbf{v}_{w_{\text{OOV}}} = \sum_{g \in \mathcal{G}_{w_{\text{OOV}}}} \mathbf{z}_g. \quad (3.68)$$

The OOV barrier is completely eliminated!

2. **Morphological Parameter Sharing:** In natural language, prefixes like "un-", suffixes like "-tion", and inflections like "-ing" accumulate shared vector representations across hundreds of distinct words. In software engineering, identifier substrings automatically share semantics:

$$\text{"parseXmlDocument"} \quad \text{and} \quad \text{"parseJsonDocument"} \quad (3.69)$$

share the character subwords for "parse" and "Document". The model immediately recognizes that both identifiers describe parsing operations on structured documents.

Memory Bounding via the Fowler-Noll-Vo (FNV-1a) Hash Trick. Because the universe of all character n -grams ($n \in [3, 6]$) across large vocabularies exceeds tens of millions of unique strings, allocating an unbounded matrix would cause memory blowout. FastText bounds the embedding table to $K = 2 \times 10^6$ rows using the Fowler-Noll-Vo (FNV-1a) hashing function:

$$\text{index}(g) = \text{FNV-1a}(g) \pmod{K}. \quad (3.70)$$

Colliding n -grams share bucket rows; due to the high dimensionality ($d = 300$) and independent sum aggregation, hash collisions act as benign regularizing noise that does not disrupt semantic retrieval.

3.9.4 Connection to modern subword tokenizers: BPE and WordPiece

As AI educator Enkk highlights in his review of language models [40], the subword philosophy pioneered by FastText directly inspired the tokenization mechanisms powering modern Large Language Models:

- **Byte Pair Encoding (BPE)** (Sennrich et al., 2016 [41]; Radford et al., 2019 [14]): Iteratively merges the most frequent pairs of adjacent characters or bytes to form a deterministic subword vocabulary (used in GPT-2, GPT-4, and LLaMA).
- **SentencePiece** (Kudo and Richardson, 2018 [42]): Treats input text as a raw byte stream without whitespace pre-segmentation, training unigram language model segmentations.

FastText operates continuous character n -gram summations at the embedding layer, whereas BPE operates discrete token mergers during preprocessing. Both architectures fundamentally exist to dismantle the brittle, isolated word-level assumption and establish robust subword parameter sharing.

By decomposing words into character n -grams, FastText conquers the Out-of-Vocabulary barrier and enables morphological parameter sharing. With our embedding models fully trained, we can now step back and examine the remarkable mathematical geometry that spontaneously emerges within these dense vector spaces.

3.10 Emergent vector space geometry: clustering, metrics and relational analogies

When Word2Vec and FastText models are trained over large text corpora, the resulting continuous vector spaces exhibit profound emergent mathematical properties: high-dimensional semantic clustering, metric equivalence, and linear relational translations.

3.10.1 Semantic clustering and dimensionality reduction

In high-dimensional embedding space ($d = 300$), words with similar functional roles organize into distinct, tightly bound clusters.

Following Slide 14 of Prof. Flavio Giobergia's lecture deck, projecting 300-dimensional FastText vectors into two dimensions via Principal Component Analysis (PCA) reveals distinct semantic clusters:

- **Household Items:** "table", "chair", "desk", "sofa".
- **Mammals:** "dog", "cat", "tiger", "elephant".
- **Birds:** "eagle", "sparrow", "parrot", "pigeon".

The leading principal components preserve categorical variance: domestic entities, animal taxonomy, and biological sub-branches form linearly separable clusters in the compressed 2D projection as well as in the original 300-dimensional manifold.

3.10.2 Linear relational translations (vector analogy arithmetic)

The most celebrated emergent property of continuous word embeddings is that semantic and syntactic relationships manifest as **linear vector translation offsets**:

$$\mathbf{v}_{w_2} - \mathbf{v}_{w_1} \approx \vec{d}_{\text{relation}}. \quad (3.71)$$

Country-to-Capital Analogy (Course Slides 15). In Prof. Giobergia's lecture deck, subtracting the vector for "Berlin" from "Germany" produces a directional displacement vector $\vec{d}_{\text{capital_of}}$:

$$\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}} \approx \vec{d}_{\text{capital_of}}. \quad (3.72)$$

Applying this learned offset vector to another country predicts its capital city via simple vector addition:

$$\mathbf{v}_{\text{Spain}} + (\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}}) \approx \mathbf{v}_{\text{Madrid}}. \quad (3.73)$$

The Gender-Royalty Parallelogram. The canonical demonstration discovered by Mikolov et al. [9] is the gender-royalty parallelogram:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}. \quad (3.74)$$

Figure 3.5 illustrates this parallelogram in two dimensions.

Software Engineering Analogy Equivalents. In code representation learning (e.g., CodeBERT [22], OpenAI Codex embeddings [43]), relational vector arithmetic reflects runtime architectures and language toolchains:

$$\mathbf{v}_{\text{Java}} - \mathbf{v}_{\text{JVM}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{CPython}}, \quad (3.75)$$

$$\mathbf{v}_{\text{C++}} - \mathbf{v}_{\text{g++}} + \mathbf{v}_{\text{Rust}} \approx \mathbf{v}_{\text{rustc}}. \quad (3.76)$$

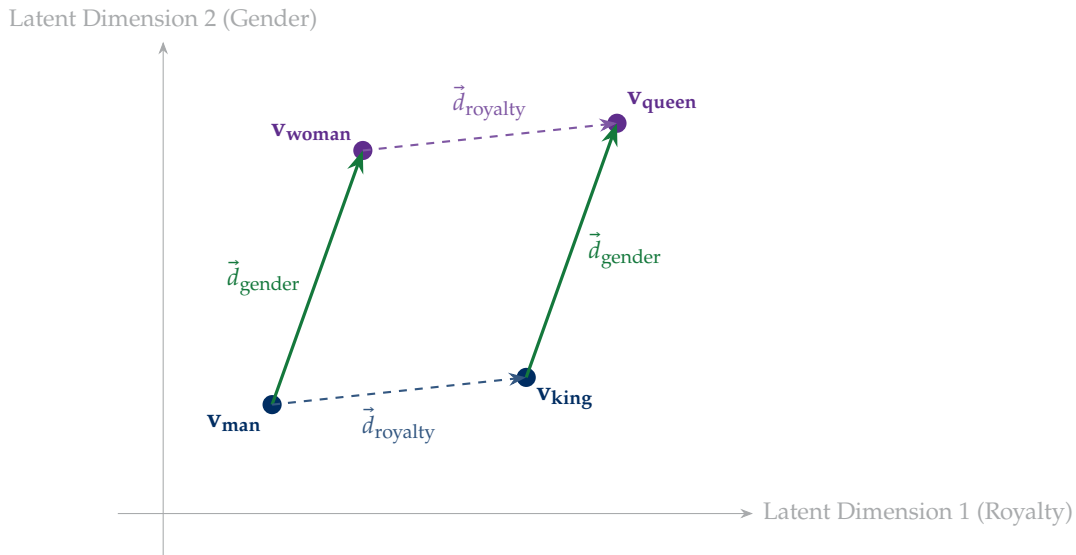


Figure 3.5: Geometric parallelogram in continuous embedding space demonstrating relational vector translations: $\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} \approx \mathbf{v}_{\text{Queen}} - \mathbf{v}_{\text{Woman}}$. The directional displacement $\vec{d}_{\text{gender}}$ isolates the feminine gender transformation, while $\vec{d}_{\text{royalty}}$ isolates the monarchical status offset.

3.10.3 Metric geometry of embedding spaces: cosine vs. Euclidean distance

When querying nearest neighbors or evaluating semantic similarity between two continuous vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, three metrics are commonly considered:

1. **Euclidean (L_2) Distance:**

$$D_{L_2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^d (u_i - v_i)^2}. \quad (3.77)$$

2. **Dot Product (Inner Product):**

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^d u_i v_i = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos \theta. \quad (3.78)$$

3. **Cosine Distance:**

$$D_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \in [0, 2]. \quad (3.79)$$

Theorem 3.2: *Monotonic equivalence under L_2 normalization*

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ be normalized to unit Euclidean length ($\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$). Then squared Euclidean distance is strictly proportional to Cosine distance:

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\mathbf{u}^\top \mathbf{v} = 1 + 1 - 2\cos(\mathbf{u}, \mathbf{v}) = 2(1 - \cos(\mathbf{u}, \mathbf{v})) = 2D_{\cos}(\mathbf{u}, \mathbf{v}). \quad (3.80)$$

Consequently, nearest neighbor rankings under Euclidean distance, Cosine distance, and Dot Product are mathematically identical on the unit hypersphere $\mathbb{S}^{d-1}$.

Why Cosine Distance Dominates Natural Language Processing. As explained by Enkk [40], when word embeddings are trained without explicit length regularization, the Euclidean norm $\|\mathbf{v}_w\|_2$ strongly correlates with unigram token frequency. Highly frequent words undergo millions of gradient pushes, systematically expanding their vector magnitude, whereas rare terms remain close to the origin. Cosine distance normalizes out vector magnitude, projecting all embeddings onto $\mathbb{S}^{d-1}$. It evaluates **pure angular orientation** (conceptual topic) rather than **vector length** (frequency artifact).

3.10.4 Dimensionality trade-offs and the Johnson-Lindenstrauss lemma

Why do word embeddings operate effectively in dimensions $d \in [100, 300]$, and modern LLMs in $d \in [768, 4096]$?

- **Under-parameterization** ($d < 50$): Severe geometric crowding occurs. The space lacks enough orthogonal degrees of freedom to separate independent semantic concepts, forcing unrelated topics into spurious proximity.
- **Over-parameterization** ($d \gg 10,000$): The space suffers from the curse of dimensionality. The volume of the unit ball concentrates entirely in a thin outer shell, distance distributions become uniform, and memory bandwidth demands explode.

The mathematical guarantee underwriting this stability is the **Johnson-Lindenstrauss Lemma**:

Theorem 3.3: Johnson-Lindenstrauss Lemma

Given $0 < \epsilon < 1$, any set of N points in arbitrary Euclidean space can be embedded into a Euclidean space of dimension:

$$d = O\left(\frac{\log N}{\epsilon^2}\right), \quad (3.81)$$

such that all pairwise Euclidean distances are preserved within a distortion factor of $1 \pm \epsilon$:

$$(1 - \epsilon)\|\mathbf{u} - \mathbf{v}\|_2^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|_2^2 \leq (1 + \epsilon)\|\mathbf{u} - \mathbf{v}\|_2^2. \quad (3.82)$$

For a vocabulary of $N = 100,000$ tokens and distortion tolerance $\epsilon = 0.2$, $\log(10^5)/0.04 \approx 287$, precisely justifying why $d = 300$ serves as the golden standard for dense static word vectors.

3.10.5 Vector databases and Approximate Nearest Neighbor (ANN) search

In modern software engineering, distributed embeddings power semantic code search and Retrieval-Augmented Generation (RAG) across millions of functions and documentation files. Evaluating brute-force cosine comparisons across $N = 10^7$ stored vectors requires $O(N \cdot d)$ operations, resulting in unacceptable search latencies (> 500 ms).

To achieve sub-millisecond retrieval, modern vector databases (e.g., Chroma, FAISS, Milvus, Qdrant) implement **Approximate Nearest Neighbor (ANN)** indexing algorithms [40]:

1. **Hierarchical Navigable Small World (HNSW) Graphs**: Constructs a multi-layer graph topology inspired by the “six degrees of separation” principle. The top layers contain sparse long-range highway edges that skip across semantic clusters, while the bottom layer contains dense local nearest-neighbor connections. Search begins at the top layer with greedy routing and transitions downward, achieving logarithmic $O(\log N)$ query complexity with $> 98\%$ recall.

2. **Inverted File with Product Quantization (IVF-PQ):** Partitions the vector space into K Voronoi cells using k -means clustering, while Product Quantization divides each d -dimensional vector into M sub-vectors and quantizes them into compact discrete codebooks. This compresses embeddings by $8\times$ to $32\times$, allowing millions of code embeddings to reside entirely in high-speed RAM with symmetric distance lookup tables.

3.10.6 Python implementation and the query filtering caveat

In his course demonstration on Word2Vec and FastText semantic arithmetic [44], Prof. Flavio Giobergia presents a clean Python implementation for answering relational analogy queries:

```

1 import numpy as np
2
3 def analogy_query(model, word_a, word_b, word_c, top_k=5):
4     """
5     Computes: word_a is to word_b as word_c is to ?
6     Target vector: v = v_b - v_a + v_c
7     """
8     vec_a = model.get_vector(word_a)
9     vec_b = model.get_vector(word_b)
10    vec_c = model.get_vector(word_c)
11
12    # Linear relational translation
13    target_vec = vec_b - vec_a + vec_c
14
15    # Query nearest neighbors by cosine similarity
16    candidates = model.most_similar(positive=[target_vec], topn=top_k + 3)
17
18    # CRITICAL PRACTICAL RULE: Exclude input query words
19    query_tokens = {word_a.lower(), word_b.lower(), word_c.lower()}
20    filtered_results = [
21        (word, score) for word, score in candidates
22        if word.lower() not in query_tokens
23    ]
24    return filtered_results[:top_k]
25
26 # Example: Germany is to Berlin as Spain is to ?
27 # target = v_Berlin - v_Germany + v_Spain -> Madrid
28 results = analogy_query(model, "germany", "berlin", "spain")
29 print("Top prediction:", results[0][0]) # Output: 'madrid'

```

Listing 3.1: Implementation of relational vector arithmetic and candidate filtering in Python [44].

Exam insights: The query token exclusion caveat in vector arithmetic

A recurrent practical and examination trap involves candidate ranking in vector arithmetic:

$$\vec{v}_{\text{target}} = \vec{v}_{\text{Berlin}} - \vec{v}_{\text{Germany}} + \vec{v}_{\text{Spain}}. \quad (3.83)$$

Because words naturally have significant baseline norm and self-correlation, if an unconstrained nearest neighbor search is performed across the entire vocabulary $\mathcal{W}$, the query token $\vec{v}_{\text{Spain}}$ or $\vec{v}_{\text{Berlin}}$ will frequently have the highest dot product with $\vec{v}_{\text{target}}$! Practitioners must explicitly filter out the input prompt words ($\mathcal{W} \setminus \{A, B, C\}$), forcing the cosine argmax to search exclusively among genuine unconditioned candidates:

$$w^* = \arg \max_{w \in \mathcal{W} \setminus \{A, B, C\}} \cos(\mathbf{v}_w, \vec{v}_{\text{target}}). \quad (3.84)$$

3.10.7 Causal bridge to Chapter 4: the necessity of sequential recurrence

While distributed word vectors successfully map discrete vocabulary symbols into a structured, continuous semantic space, they suffer from a fundamental, insurmountable temporal limitation: **they treat language as a static collection of isolated tokens.**

In natural language and software engineering source code, tokens do not exist in isolation. Language is an ordered, variable-length temporal sequence where token order, syntactic hierarchy, and long-range dependencies dictate execution and meaning. A static embedding layer has no memory: it cannot track whether a variable was declared before it was referenced, cannot resolve nested control scopes, and cannot retain state across an extended document. To model variable-length sequential context and compute dynamically stateful representations, we must advance from static spatial embeddings to temporal recurrent architectures: **Recurrent Neural Networks (RNNs)**, the subject of Chapter 4.

Recurrent neural networks and sequence modeling

4.1 Chapter overview and learning outcomes

In Chapter 2, we analyzed the artificial perceptron and multi-layer feedforward networks, while Chapter 3 introduced continuous distributed vector representations (Word2Vec and FastText). However, static feedforward architectures treat input tokens as unordered bags of words or rigid fixed-width vectors, fundamentally ignoring temporal order and sequential dependencies.

This chapter formalizes the transition from static feedforward networks to recurrent sequential architectures. We examine why standard Fully-Connected Neural Networks (FCNNs) fail on unstructured sequential text, formulate the Recurrent Neural Network (RNN) cell, derive Backpropagation Through Time (BPTT), and provide the rigorous Jacobian product proof of vanishing and exploding gradients. We inspect the empirical training dynamics of an RNN on Arthur Conan Doyle’s *The Adventures of Sherlock Holmes*, trace the phase transitions across training epochs, and unpack the constant error carousel of Long Short-Term Memory (LSTM) networks [21] and Gated Recurrent Units (GRU) [11]. Finally, we analyze Sequence-to-Sequence (Seq2Seq) Encoder-Decoder pipelines [10], Teacher Forcing, and the information bottleneck that directly motivated the invention of the Transformer.

Upon completing this chapter, you will be able to:

- Identify the three fundamental architectural constraints of Fully-Connected Neural Networks on sequential unstructured data: fixed input dimensionality, lack of parameter sharing/translation invariance, and fixed output size.
- Formulate the vanilla RNN cell forward equations ($\mathbf{h}_t, \hat{\mathbf{y}}_t$) with explicit tensor dimensionality tracking across input projection $\mathbf{W}_{xh}$, recurrent transition $\mathbf{W}_{hh}$, and output emission $\mathbf{W}_{hy}$.
- Unroll recurrent architectures across discrete time steps and explain the mechanics of temporal weight sharing.
- Derive the full Jacobian product for Backpropagation Through Time (BPTT) and prove mathematically why the recurrent transition matrix spectral radius $\rho(\mathbf{W}_{hh})$ causes exponential vanishing ($\lambda < 1$) or exploding ($\lambda > 1$) gradients.
- Trace the epoch-by-epoch training dynamics of character-level RNNs, explaining the role of null-byte padding, identity passthrough, and causal lookahead blindness.
- Derive the gating equations of the LSTM cell (forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, candidate cell $\tilde{\mathbf{c}}_t$, additive highway $\mathbf{c}_t$, and output gate $\mathbf{o}_t$) and prove the Constant Error Carousel ($\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t$).
- Contrast LSTM with the Gated Recurrent Unit (GRU), comparing parameter counts ($4\times$ vs. $3\times$ vanilla weights) and structural trade-offs.

- Explain the limitations of one-to-one sequence mappings on non-monotonic machine translation and formulate the Encoder-Decoder (Seq2Seq) architecture with Teacher Forcing.
- Analyze the persistent information and recurrence bottlenecks that motivated Attention Mechanisms [12] and Transformers [1].

4.2 Sequential data and constraints of feed-forward neural networks

In standard deep learning, a Fully-Connected Neural Network (FCNN) or Multi-Layer Perceptron is represented as a cascade of weight matrices interleaved with non-linear activation functions:

$$f(\mathbf{x}) = \sigma(\mathbf{W}_{\text{out}} \text{ReLU}(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_{\text{out}}). \quad (4.1)$$

While FCNNs excel on tabular datasets with fixed, predefined attributes, they exhibit three critical architectural constraints when applied to natural language and temporal sequences (Slides 2–5):

1. The input is expected to be fixed in size. The input dimensionality is rigidly fixed by the number of columns in the first weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_1 \times d_{\text{in}}}$. For structured tabular data, this constraint is natural because every sample contains the same predefined attributes. However, for unstructured sequential data, fixed input sizing is deeply problematic:

- Natural language sentences come in all lengths: from 2 words (“*Run now*”) to 80 words in complex legal contracts.
- Audio recordings exhibit variable sampling frequencies and durations.
- Images come in varying resolutions and aspect ratios.

Flattening a variable-length text sequence into a rigid vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ requires setting an arbitrary maximum length T_{max} and applying zero-padding to shorter sentences while truncating longer ones. This wastes vast memory on empty pad tokens and destroys the elastic temporal nature of human language.

2. Lack of parameter sharing and translation invariance. In an FCNN, each position in the flattened input vector is associated with its own distinct set of weight parameters. Consequently, if the same linguistic pattern occurs in different portions of the input, the network must learn that pattern multiple times from scratch:

- In computer vision, a dog is a dog regardless of whether it appears on the left or the right side of an image. An FCNN must learn a separate “dog-detector” for the left pixels and the right pixels.
- In NLP, the noun phrase “*a dog*” possesses the exact same grammatical and semantic structure whether it appears at token position 1 (“*A dog barked loudly*”) or token position 10 (“*After a long walk in the park, a dog barked*”).

Because an FCNN does not share weights across sequence positions, it is wildly inefficient and requires massive datasets to observe every linguistic pattern at every conceivable spatial index.

3. The output is expected to be fixed in size. The output dimensionality is rigidly determined by the number of rows in the final projection matrix $\mathbf{W}_{\text{out}}$. In many sequential tasks (such as machine translation, text summarization, and dialogue generation), the output sequence length is unknown a priori and varies dynamically based on the input:

English (5 words): “It is indeed sunny today” $\implies$ Italian (4 words): “Effettivamente oggi è soleggiato”.

An FCNN cannot dynamically adjust its output layer to emit variable-length token sequences.

4.3 The recurrent neural network cell and weight sharing

To overcome the constraints of FCNNs, **Recurrent Neural Networks (RNNs)**—first formalized as the Simple Recurrent Network by Jeffrey L. Elman [45]—process sequential data element by element, maintaining an internal continuous *hidden state* $\mathbf{h}_t \in \mathbb{R}^{d_h}$ that acts as a recurrent memory summarizing the sequence history observed up to time step t (Slides 6–7).

4.3.1 Recurrent cell architecture

At each discrete time step $t \in \{1, 2, \dots, T\}$, the recurrent cell is characterized by two inputs and two outputs:

- **Input 1:** The current sequence element $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ (e.g., a one-hot vector or continuous embedding).
- **Input 2:** The hidden state produced in the previous time step $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$.
- **Output 1:** The model’s prediction for the current time step $\hat{\mathbf{y}}_t \in \mathbb{R}^{d_{\text{out}}}$.
- **Output 2:** The updated hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$, which is recursively forwarded to time step $t + 1$.

4.3.2 Mathematical formulation with explicit tensor dimensions

Internally, the vanilla RNN cell comprises an input projection layer, a recurrent state transition layer, and an output projection layer (Slide 12):

$$\underbrace{\mathbf{h}_t}_{[d_h \times 1]} = \tanh \left(\underbrace{\mathbf{W}_{hh}}_{[d_h \times d_h]} \underbrace{\mathbf{h}_{t-1}}_{[d_h \times 1]} + \underbrace{\mathbf{W}_{xh}}_{[d_h \times d_{\text{in}}]} \underbrace{\mathbf{x}_t}_{[d_{\text{in}} \times 1]} + \underbrace{\mathbf{b}_h}_{[d_h \times 1]} \right), \quad (4.2)$$

$$\underbrace{\hat{\mathbf{y}}_t}_{[V \times 1]} = \text{softmax} \left(\underbrace{\mathbf{W}_{hy}}_{[V \times d_h]} \underbrace{\mathbf{h}_t}_{[d_h \times 1]} + \underbrace{\mathbf{b}_y}_{[V \times 1]} \right). \quad (4.3)$$

4.3.3 Weight sharing across time

The defining feature of RNNs is **temporal weight sharing** (Slide 13). The parameter matrices $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, and $\mathbf{W}_{hy}$ are *strictly identical* across all time steps $t \in \{1, \dots, T\}$. Because the same weights are applied regardless of time step, the model develops **translation invariance**: if the same context occurs at step $t = 2$ or step $t = 50$, the recurrent cell behaves identically. The network learns grammatical rules (such as capitalizing the first letter after a quotation mark) once, and applies that rule everywhere in the text.

💡 Intuition & analogy: Recurrent dynamics as a trajectory through high-dimensional phase space

As visualized by Grant Sanderson in 3Blue1Brown [36], the hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$ can be understood geometrically as the position of a particle traveling through a continuous d_h -dimensional *phase space*:

- At rest ($t = 0$), the system sits at initial coordinate $\mathbf{h}_{-1} = \mathbf{0}$.
- When each discrete input token $\mathbf{x}_t$ arrives, the linear projection $\mathbf{W}_{xh}\mathbf{x}_t$ applies an external directional impulse or “vector nudge” to the particle.
- Simultaneously, the recurrent transformation $\mathbf{W}_{hh}\mathbf{h}_{t-1}$ rotates, scales, and folds the previous position, while the non-linear squashing activation $\tanh$ confines the entire state space within the hypercube $(-1, 1)^{d_h}$.
- The unfolding sequence of text traces a smooth continuous trajectory through state space. Regions of this phase space function as **attractor basins**: for instance, reading an opening quote mark (") knocks the state into a specific sub-manifold where the emission projection $\mathbf{W}_{hy}$ naturally maps coordinates into uppercase character distributions. The state remains trapped in this basin until an exit trigger (such as a whitespace or subsequent character) forces the trajectory out into the lowercase manifold.

4.3.4 Unrolling the recurrent cell across time

To trace how an RNN processes a sequence, we *unroll* the recursive feedback loop across discrete time steps $t = 0, 1, \dots, T$ (Slides 8–11).

Consider the character casing task:

- Input sequence: "you are very kind, mr. holmes, but i cannot do that"
- Target sequence: "You are very kind, Mr. Holmes, but I cannot do that"

Each ASCII character is represented as a 256-dimensional one-hot vector $\mathbf{x}_t \in \{0, 1\}^{256}$:

- **Step $t = 0$:** Receives input $s_0 = "$ with initial state $\mathbf{h}_{-1} = \mathbf{0}$. The cell produces state $\mathbf{h}_0$. The network realizes: *“We just encountered an opening quote (”), which signals the beginning of dialogue; the following character should be capitalized.”*
- **Step $t = 1$:** Receives input $s_1 = y$ and prior state $\mathbf{h}_0$. The network emits output $\hat{y}_1 = Y$ and produces updated state $\mathbf{h}_1$. The state records: *“We just emitted a capitalized letter at the start of dialogue; following letters in this word should probably be lowercase.”*
- **Step $t = 2$:** Receives input $s_2 = o$ and prior state $\mathbf{h}_1$. The network emits output $\hat{y}_2 = o$ and produces state $\mathbf{h}_2$, continuing lowercase word progression.

Figure 4.1 illustrates this unrolled computational structure and the backward flow of gradients.

4.4 Backpropagation through time and the proof of vanishing gradients

When an RNN processes a sequence of length T , the computational graph is unrolled T times. The total sequence loss is the sum of cross-entropy losses incurred at each time step (Slides

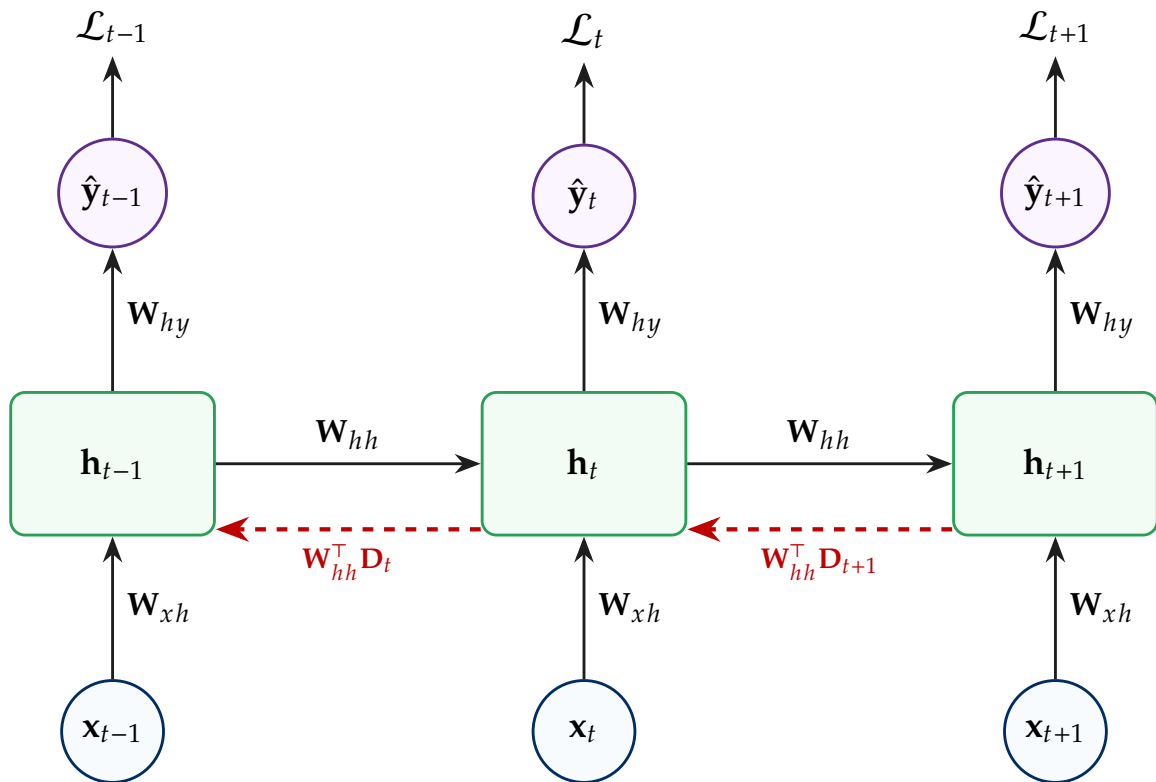


Figure 4.1: Unrolling an RNN cell across time steps $t-1, t, t+1$. Solid black arrows show the forward propagation of inputs $\mathbf{x}_t$ and recurrent states $\mathbf{h}_t$. Dashed red arrows trace Backpropagation Through Time (BPTT), where gradients accumulate across time via repeated multiplications by the Jacobian $\mathbf{W}_{hh}^\top \mathbf{D}_t$.

14–17):

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t. \quad (4.4)$$

To optimize the shared recurrent weight matrix $\mathbf{W}_{hh}$ via Gradient Descent, we apply **Backpropagation Through Time (BPTT)**, first formalized for recurrent structures by Paul J. Werbos [46]. The fundamental difficulty of learning long-term dependencies through gradient descent was rigorously analyzed by Yoshua Bengio et al. [47] and Pascanu et al. [48]. The gradient of the total loss with respect to $\mathbf{W}_{hh}$ accumulates the contributions across all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \quad (4.5)$$

where $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denotes the direct derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$ treating $\mathbf{h}_{k-1}$ as constant.

The temporal transmission term $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the sensitivity of state $\mathbf{h}_t$ to an earlier state $\mathbf{h}_k$ ($t > k$). By the multivariate chain rule, this sensitivity decomposes into a product of single-step Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \quad (4.6)$$

4.4.1 Rigorous mathematical proof of gradient pathology

We now derive the explicit single-step Jacobian and prove why vanilla RNNs suffer from vanishing and exploding gradients.

Theorem 4.1: *Jacobian product and spectral radius gradient decay*

Let the recurrent state transition be $\mathbf{h}_j = \tanh(\mathbf{z}_j)$ where $\mathbf{z}_j = \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{W}_{xh}\mathbf{x}_j + \mathbf{b}_h$. The single-step temporal Jacobian $\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \in \mathbb{R}^{d_h \times d_h}$ is:

$$\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \text{diag}(1 - \mathbf{h}_j^2) \mathbf{W}_{hh}^\top = \mathbf{D}_j \mathbf{W}_{hh}^\top, \quad (4.7)$$

where $\mathbf{D}_j = \text{diag}(1 - \tanh^2(\mathbf{z}_j))$ is a diagonal matrix containing the local derivatives of the hyperbolic tangent function. Consequently, the multi-step gradient evaluates to:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \mathbf{D}_j \mathbf{W}_{hh}^\top. \quad (4.8)$$

Let $\lambda_{\max}$ denote the spectral radius (maximum singular value) of $\mathbf{W}_{hh}^\top$. The Euclidean norm of the transmitted gradient is bounded by:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| (\lambda_{\max})^{t-k}. \quad (4.9)$$

Proof. Differentiating the i -th component of $\mathbf{h}_j$ with respect to the m -th component of $\mathbf{h}_{j-1}$:

$$\frac{\partial h_{j,i}}{\partial h_{j-1,m}} = \frac{\partial \tanh(z_{j,i})}{\partial z_{j,i}} \frac{\partial z_{j,i}}{\partial h_{j-1,m}} = (1 - h_{j,i}^2) W_{hh,i,m}.$$

In matrix notation, this is $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \text{diag}(1 - \mathbf{h}_j^2) \mathbf{W}_{hh}^\top = \mathbf{D}_j \mathbf{W}_{hh}^\top$. Because $|\tanh'(z)| = 1 - \tanh^2(z) \in [0, 1]$, the operator norm of each diagonal matrix satisfies $\|\mathbf{D}_j\|_2 \leq 1$. Applying the sub-multiplicative property of matrix norms to Equation (4.8):

$$\begin{aligned} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\|_2 &= \left\| \prod_{j=k+1}^t \mathbf{D}_j \mathbf{W}_{hh}^\top \right\|_2 \leq \prod_{j=k+1}^t \|\mathbf{D}_j\|_2 \|\mathbf{W}_{hh}^\top\|_2 \\ &\leq \prod_{j=k+1}^t 1 \cdot \lambda_{\max} = (\lambda_{\max})^{t-k}. \end{aligned}$$

Multiplying by $\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\|_2$ yields Equation (4.9). □

Two pathological regimes (slides 21–23):

1. **Vanishing Gradient** ($\lambda_{\max} < 1$): If the singular values of $\mathbf{W}_{hh}$ are slightly less than 1 (e.g., $\lambda_{\max} = 0.9$), over a temporal distance of 50 steps:

$$0.9^{50} \approx 0.00515 \quad (\text{over 99.5\% of the gradient signal is extinguished}).$$

Over 100 steps, $0.9^{100} \approx 0.000026$. The gradient decays exponentially to zero. Parameter updates from distal tokens vanish, meaning the network cannot learn long-range temporal dependencies. Saturated activations (such as $\sigma'(z) \leq 0.25$) accelerate this decay exponentially.

2. **Exploding Gradient** ($\lambda_{\max} > 1$): If the singular values exceed 1 (e.g., $\lambda_{\max} = 1.1$), over 50 steps:

$$1.1^{50} \approx 117.39.$$

Over 100 steps, $1.1^{100} \approx 13,780$. Gradients grow exponentially, destabilizing training, inducing severe numerical overflow, and producing NaN parameter corruptions. (Practitioners counter exploding gradients using heuristic *gradient clipping*: $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, \frac{\text{threshold}}{\|\mathbf{g}\|})$).

The long-term dependency degradation (slides 24–25). Because hidden states are rewritten at every time step without persistent memory protection, vanilla RNNs suffer from catastrophic recency bias: they remember only the most recent tokens. When tested on long-range word prediction:

*“The **mouse** was getting chased by the cat through the big house. After a long chase, the cat finally managed to catch the _____”*

the target word is unambiguously “mouse”. However, vanilla RNNs fail completely because the hidden state has overwritten the subject “mouse” across the intervening 20 tokens.

4.5 Case correction demonstration: Sherlock Holmes training dynamics

To demonstrate how an RNN learns sequential syntax in practice, Slides 18–20 document an empirical case study trained on Arthur Conan Doyle’s *The Adventures of Sherlock Holmes*:

Experimental configuration.

- **Corpus:** Raw text from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*.
- **Sequence Length:** Subsequences sliced between 10 and 100 ASCII characters.
- **Token Representation:** 256-dimensional one-hot vectors ($d_{\text{in}} = 256$).
- **Architecture:** Single-layer vanilla RNN with hidden dimension $d_h = 32$.
- **Optimization:** Adam optimizer (lr = 0.001), batch size 32, trained for 10 epochs.
- **Padding Mechanism:** Null bytes (0x00, rendered visually as X) used to pad variable-length sequences to 100 characters.

Fixed evaluation test sentence. The model is evaluated after each epoch on a fixed unseen test string:

- **Input Test Sequence:** upon his pale face. "it may be so, or it may not, mr. holmes," said he, "but if you are so very sharp
- **Target Test Sequence:** upon his pale face. "It may be so, or it may not, Mr. Holmes," said he, "but if you are so very sharp

Qualitative phase transitions across 10 epochs. Table 4.1 records the loss trajectory and qualitative phase transitions across training:

4.5.1 Character-level modeling, temperature sampling, and hidden state interpretability

The Sherlock Holmes casing experiment exemplifies a broader paradigm popularized by Andrej Karpathy in his landmark analysis "*The Unreasonable Effectiveness of Recurrent Neural Networks*" [49]: ****character-level sequence modeling****.

1. The elegance of character-level modeling. Unlike word-level models that require complex tokenizers, morphological analyzers, and massive vocabularies ($V \geq 50,000$), character-level RNNs operate over an extremely compact alphabet (e.g., standard 256-dimensional ASCII).

- There are **zero Out-of-Vocabulary (OOV) tokens**: any arbitrary string, code snippet, mathematical formula, or neologism can be represented as an sequence of bytes.
- The model is forced to discover the hierarchical structure of language entirely from scratch: it must learn how characters form words, that words are separated by spaces, how punctuation delimits clauses, and how grammatical agreement spans across clauses.

2. Autoregressive sampling and temperature dynamics. During text generation, the RNN emits an unnormalized logit vector $\mathbf{z}_t = \mathbf{W}_{hy} \mathbf{h}_t + \mathbf{b}_y \in \mathbb{R}^V$. Rather than taking the deterministic arg max, we convert logits into a sampling distribution modulated by a scalar hyperparameter: the ****temperature**** $T > 0$:

$$\mathbb{P}(y_t = c \mid \mathbf{h}_t) = \frac{\exp\left(\frac{z_{t,c}}{T}\right)}{\sum_{j=1}^V \exp\left(\frac{z_{t,j}}{T}\right)}. \quad (4.10)$$

The temperature controls the entropy and randomness of the emitted sequence:

Table 4.1: Epoch-by-epoch loss progression and qualitative phase transitions on the Sherlock Holmes casing task.

Epoch	Loss	Observed Generation & Qualitative Linguistic Transition
01	3.1681	XX Phase 1 (Null-byte frequency): The model discovers that null padding bytes (0x00 / X) appear with high statistical frequency due to padding imbalance, predicting all X's.
02	2.3334	eXX Phase 2 (Vowel emergence): The model predicts e (the most frequent character in English orthography) alongside padding characters.
03	1.7436	tst X s te e eee t e e e te se ee se e e ee eee XXXee Phase 3 (Whitespace and unigrams): The model learns that whitespace characters occur regularly, predicting spaces, e, t, and s.
04	0.9519	upon his pale face. cit may be sos or it may nots mr. holresst Phase 4 (The Identity Breakthrough): A massive drop in loss occurs when the network learns to forward lowercase letters directly from input to output (w.grad captures the identity diagonal).
05-06	0.3579	upon his pale face. "it may be so, or it may not, mr. holmes," Phase 5 (Punctuation and Syntax): The model captures quotation marks ("), commas, and spaces accurately.
07-08	0.1009	upon his pale face. "It may be so, or it may not, mr. holmes," Phase 6 (Quote Capitalization): The model learns that characters immediately following opening quotes (") must be capitalized ("It).
09-10	0.0515	upon his pale face. "It may be so, or it may not, mr. Holmes," Phase 7 (Period Capitalization and Causal Blindness): The model capitalizes Holmes because it follows a period (.). Crucially, the model fails to capitalize mr. because it cannot see into the future! It does not know that m begins the title Mr. until the r. arrives.

- **Low Temperature ($T \rightarrow 0$ / Greedy Decoding):** Exaggerates differences between logits. The largest logit absorbs almost all probability mass ($p_{\max} \rightarrow 1$). The model makes confident, conservative predictions, but frequently falls into pathological mode collapse—getting trapped in infinite repetitive loops (e.g., "the the the the" or repeating identical sentence fragments).
- **Balanced Temperature ($T \in [0.6, 0.8]$):** Preserves the syntactic and morphological structure discovered by the network while introducing sufficient stochastic diversity to produce coherent, rich generations. In Karpathy's experiments [49], character-level LSTMs trained at $T = 0.7$ generated structurally sound Shakespearean plays (including character names, stage directions, and poetic meter), syntactically compilable C code from the Linux kernel, and valid algebraic geometry LaTeX documents.
- **High Temperature ($T > 1.2$):** Flattens the probability distribution toward a uniform distribution ($p_c \rightarrow 1/V$). The network makes chaotic, exploratory predictions, generating misspelled words, hallucinated punctuation, and unpronounceable character noise.

3. Interpretable hidden state neurons. By inspecting the internal cell activations $\mathbf{h}_t$ of a trained LSTM as it reads character sequences, Karpathy [49] discovered that individual hidden units act as interpretable linguistic finite-state machines:

1. **The Quote Detection Neuron:** A single scalar unit in $\mathbf{h}_t$ that fires strongly positive (+1) the moment an opening quotation mark (") is parsed, remains persistently active throughout the enclosed dialogue, and resets immediately to -1 when the matching closing quote mark is encountered.
2. **The Code Indentation / Scope Depth Neuron:** In code generation models, a specific neuron's activation ramps up linearly with every tab or whitespace indentation inside nested loops or conditionals, tracking variable scope depth and resetting when closing brackets (}) appear.
3. **The Line Length Tracker:** A unit whose activation monotonically increases across 80 characters, anticipating when to emit the carriage return ($\backslash n$) character to conform to standard formatting conventions.

4. Classroom insights: causal lookahead blindness and overfitting dynamics. In the Politecnico di Torino lecture, Prof. Giobergia highlights two essential practical takeaways from the Sherlock Holmes casing demo:

- **Causal Lookahead Blindness:** Why did the one-to-one RNN fail to capitalize "mr ." into "Mr ." even after 10 epochs? The model operates strictly causally from left to right. When processing the character 'm', the model only sees past history. It cannot see into the future to know whether 'm' begins the proper title "Mr ." or common words like "me", "my", or "much". Without lookahead, capitalizing 'm' would be an unsupported guess. Conversely, the model successfully capitalizes "Ho^lmes" because it has already observed the preceding period and space (" . h"), triggering capitalization.
- **Overfitting vs. Generalization Dynamics:** When asked by students why training was stopped at 10 epochs, Prof. Giobergia explained that extending training for hundreds of epochs on a finite corpus (such as Doyle's novel) causes the model to memorize specific character n -grams verbatim. While training loss continues to drop toward zero, the network loses its ability to generalize to novel sentence structures, reinforcing the necessity of early stopping and validation monitoring.

4.6 Gated architectures: LSTM and GRU

To solve the vanishing gradient problem and allow neural networks to retain dependencies over hundreds of time steps, researchers introduced **Gated RNNs** (Slide 26–29):

4.6.1 Long short-term memory (LSTM)

Introduced by Sepp Hochreiter and Jürgen Schmidhuber [21], the **Long Short-Term Memory (LSTM)** network partitions memory into two separate states:

1. **Cell State** $\mathbf{c}_t \in \mathbb{R}^{d_h}$: The *long-term memory* highway, which runs straight down the entire chain with minimal linear interactions.
2. **Hidden State** $\mathbf{h}_t \in \mathbb{R}^{d_h}$: The *short-term memory*, which acts as the visible output emitted at each step.

Definition 4.1: LSTM gating equations

At time step t , given input $\mathbf{x}_t \in \mathbb{R}^{d_{in}}$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$, the LSTM evaluates three sigmoid gates and a candidate state:

$$\text{Forget Gate: } \mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \in (0, 1)^{d_h}, \quad (4.11)$$

$$\text{Input Gate: } \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \in (0, 1)^{d_h}, \quad (4.12)$$

$$\text{Candidate Cell: } \tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \in (-1, 1)^{d_h}, \quad (4.13)$$

$$\text{Cell State Update: } \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \in \mathbb{R}^{d_h}, \quad (4.14)$$

$$\text{Output Gate: } \mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \in (0, 1)^{d_h}, \quad (4.15)$$

$$\text{Hidden State Update: } \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \in (-1, 1)^{d_h}, \quad (4.16)$$

where $\odot$ denotes the element-wise Hadamard product, and $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h+d_{in}}$ denotes vector concatenation.

Semantic roles of the gates (slide 29): Each gate squashes pre-activations through a sigmoid function $\sigma \in [0, 1]$, functioning as a continuous valve:

- Multiplying by 0 $\implies$ drop/forget everything.
- Multiplying by 1 $\implies$ preserve/pass everything.
- **Forget Gate ($\mathbf{f}_t$):** Regulates what proportion of the past cell state $\mathbf{c}_{t-1}$ to discard. (e.g., when a closing quote or period arrives, the forget gate purges past sentence subjects).
- **Input Gate ($\mathbf{i}_t$):** Regulates what proportion of the new candidate information $\tilde{\mathbf{c}}_t$ to write into the long-term cell state.
- **Output Gate ($\mathbf{o}_t$):** Regulates which portions of the internal cell state $\mathbf{c}_t$ should be revealed as visible hidden state $\mathbf{h}_t$ for current output predictions.

Theorem 4.2: The Constant Error Carousel proof

In an LSTM, the gradient of the loss with respect to past cell state $\mathbf{c}_{t-1}$ contains an additive

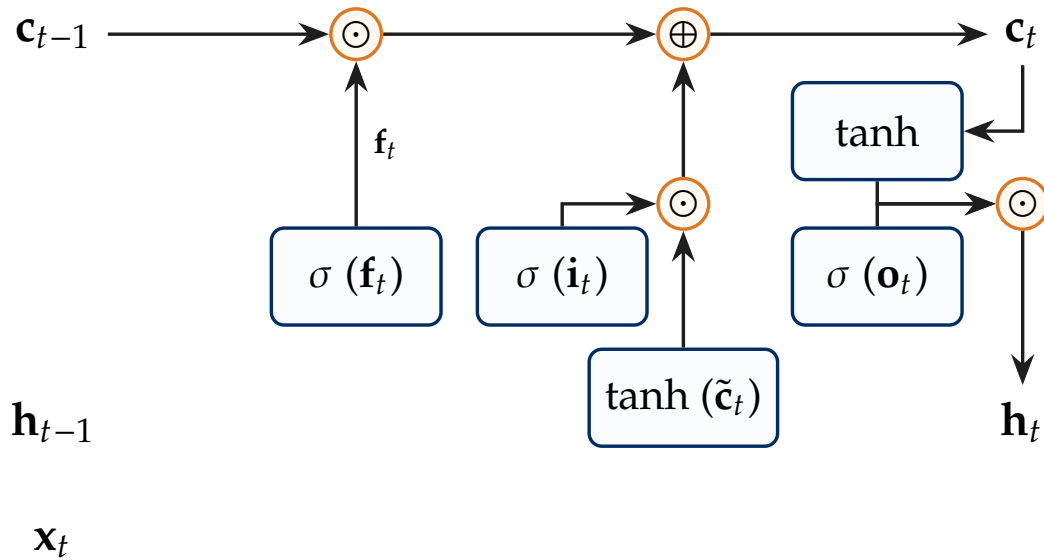


Figure 4.2: The LSTM memory cell architecture. The top horizontal path forms the uninterrupted additive cell state highway $\mathbf{c}_t$ (the Constant Error Carousel). Forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ modulate information flow using multiplicative valves.

direct path:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} \mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} \tilde{\mathbf{c}}_t + \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}} \mathbf{i}_t. \quad (4.17)$$

When the network sets forget gate $\mathbf{f}_t \approx \mathbf{1}$, the leading derivative collapses to $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$. Consequently, over any arbitrary temporal horizon $T - k$:

$$\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_k} = \prod_{j=k+1}^T \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} \approx \prod_{j=k+1}^T \mathbf{I} = \mathbf{I}. \quad (4.18)$$

Gradients flow backward across hundreds of time steps without exponential decay. This linear additive gradient highway is the **Constant Error Carousel (CEC)**.

💡 Intuition & analogy: The conveyor belt of memory

Think of the cell state $\mathbf{c}_t$ as a continuous factory conveyor belt running from the start of the sequence to the end. The standard recurrent transition in a vanilla RNN continually picks up the items, squashes them with weights and tanh, and puts a degraded version back on the belt. Over 50 time steps, the original items are unrecognizable.

In an LSTM, items on the conveyor belt are untouched by default. As the belt passes each station:

1. The forget gate $\mathbf{f}_t$ acts as a discard mechanism: if $\mathbf{f}_t \approx \mathbf{1}$, the past item rides past undisturbed; if $\mathbf{f}_t \approx \mathbf{0}$, it is swept off the belt.
2. The input gate $\mathbf{i}_t$ adds new relevant cargo $\tilde{\mathbf{c}}_t$ onto the belt via simple vector addition ($\oplus$).
3. The output gate $\mathbf{o}_t$ inspects the cargo on the belt, reads off a copy, and displays it to

the outside world as hidden state $\mathbf{h}_t$, leaving the underlying cell state intact on the highway.

Because writing is additive rather than multiplicative, information can ride the conveyor belt across hundreds of words without experiencing exponential decay.

4.6.2 Gated recurrent unit (GRU)

Introduced by Kyunghyun Cho et al. [11], the **Gated Recurrent Unit (GRU)** simplifies the LSTM architecture by merging the cell state and hidden state into a single recurrent state $\mathbf{h}_t$ and coupling the forget and input gates into a single *update gate* $\mathbf{z}_t$:

Definition 4.2: GRU equations

At time step t , given input $\mathbf{x}_t$ and prior state $\mathbf{h}_{t-1}$:

$$\text{Update Gate: } \mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z), \quad (4.19)$$

$$\text{Reset Gate: } \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r), \quad (4.20)$$

$$\text{Candidate Hidden State: } \tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h), \quad (4.21)$$

$$\text{Hidden State Interpolation: } \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (4.22)$$

Exam insights: Parameter counting: vanilla RNN vs. LSTM vs. GRU

A classic written examination question asks students to compare the exact parameter counts of recurrent cells for hidden dimension d and input dimension m :

- **Vanilla RNN:** Contains 1 transition set $(\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h) \implies d^2 + dm + d = d(d + m + 1)$.
- **GRU:** Contains 3 gate sets $(\mathbf{z}_t, \mathbf{r}_t, \tilde{\mathbf{h}}_t) \implies 3 \times d(d + m + 1)$.
- **LSTM:** Contains 4 gate sets $(\mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{c}}_t, \mathbf{o}_t) \implies 4 \times d(d + m + 1)$.

The GRU reduces parameters by 25% compared to LSTM while exhibiting comparable empirical performance on moderate sequence lengths.

4.7 Sequence-to-sequence (Seq2Seq) and encoder-decoder pipelines

4.7.1 Limitations of one-to-one sequence mapping

Thus far, we have considered models where each input step produces exactly one output step ($|X| = |Y|$), such as character-level case correction. However, the majority of core NLP tasks (machine translation, question answering, code generation) violate this assumption (Slides 30–32):

1. **Unequal Sequence Lengths:** In machine translation, source and target languages rarely match in word count:

English (5 words): *[it] [is] [indeed] [sunny] [today]* $\implies$ Italian (4 words): *[effettivamente] [oggi] [è] [soleggiato]*.

A one-to-one recurrent model would process 5 tokens and be forced to produce 5 output tokens.

2. **Non-Monotonic Alignment and Causal Blindness:** Natural languages exhibit different word orders. In the example above, the Italian word “*effettivamente*” must be emitted at step 1, but its English counterpart “*indeed*” does not appear until step 3! In a standard one-to-one model, the cell cannot see the future, rendering early word generation impossible.

4.7.2 The encoder-decoder architecture

To overcome these limitations, Sutskever et al. [10] and Cho et al. [11] introduced the **Encoder-Decoder (Seq2Seq)** architecture (Slides 33–34):

1. **Encoder:** An LSTM processes the entire variable-length source sequence $(x_1, \dots, x_N)$ step by step, updating its internal state. The final hidden state $\mathbf{h}_N$ acts as a dense, compressed summary vector encoding the semantics of the whole input sentence.
2. **Decoder:** A separate LSTM initialized with state $\mathbf{h}_N$. The decoder autoregressively emits target tokens step by step. The initial input token is the special Beginning of Sequence symbol $\langle \text{BOS} \rangle$. At each subsequent step t , the input is the token emitted at step $t - 1$. Generation continues until the model emits the special End of Sequence symbol $\langle \text{EOS} \rangle$.

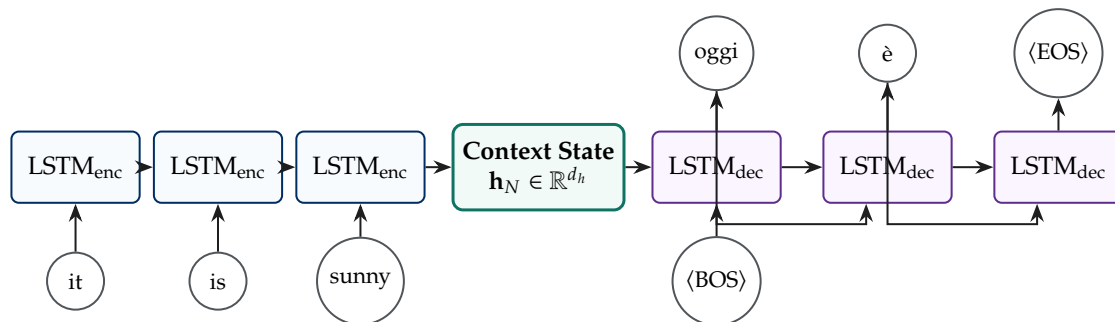


Figure 4.3: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder LSTM compresses the variable-length source sentence into a single context state $\mathbf{h}_N$. The Decoder LSTM is initialized with $\mathbf{h}_N$ and autoregressively emits target tokens from $\langle \text{BOS} \rangle$ until emitting $\langle \text{EOS} \rangle$.

4.7.3 Teacher forcing vs. inference exposure bias

During training, the decoder’s predicted token $\hat{y}_{t-1}$ is often incorrect in early epochs. If the model feeds its own faulty prediction into step t , errors compound catastrophically down the sequence.

To stabilize and accelerate training, practitioners use **Teacher Forcing** (Slide 34):

$$\mathbf{x}_{\text{dec},t} = y_{t-1}^* \quad (\text{pass true ground-truth target token instead of predicted } \hat{y}_{t-1}). \quad (4.23)$$

Exposure Bias Warning: While Teacher Forcing provides rapid convergence during training, it creates a discrepancy at inference time when ground-truth tokens are unavailable, causing models to be fragile when recovering from early generation mistakes.

4.7.4 Sherlock Holmes case correction revised: one-to-one vs. encoder-decoder

Slides 35 and 36 evaluate the sentence:

Input: "indeed i am sherlock holmes," $\implies$ Target: "Indeed I am Sherlock Holmes,"

- **One-to-One Model:** Successfully capitalizes "Indeed" (after quote) and "Holmes" (preceded by "sherlock"). However, it completely fails on the pronoun "I" and the first name "Sherlock" because it cannot look ahead to see that "i" is a standalone word and "s" begins a proper noun.
- **Encoder-Decoder Model:** Successfully capitalizes all four targets (Indeed, I, Sherlock, Holmes) because the encoder encodes the entire sequence before generation begins.
- **Training Trade-off:** Convergence requires **1070 epochs** for Encoder-Decoder versus **10 epochs** for the one-to-one model, illustrating the optimization difficulty of learning unguided sequence-level alignment.

4.8 Persistent bottlenecks of gated RNNs and the road to transformers

While LSTMs and Encoder-Decoder architectures dominated sequence modeling from 2014 to 2017, Slide 37 identifies three irreducible physical and computational bottlenecks that ultimately necessitated a revolutionary paradigm shift:

1. The Shannon channel capacity and vector bottleneck (Sutskever et al., 2014 [10]). In the canonical Seq2Seq framework, an input sequence $(x_1, \dots, x_{T_x})$ of arbitrary length and linguistic entropy is forced to compress into a single continuous vector of fixed dimensionality:

$$\mathbf{h}_{T_x} = \text{Encoder}(x_1, \dots, x_{T_x}) \in \mathbb{R}^{d_h}. \quad (4.24)$$

By Shannon's source coding and rate-distortion theory, a finite-dimensional vector in floating-point precision has bounded information capacity. As sequence length increases beyond $T_x \geq 30$ tokens, the recurrent state suffers from catastrophic information dilution: earlier tokens (often containing essential syntactic subjects and conditioning context) are progressively overwritten by later inputs.

2. The sequential computation wall ($O(T)$ training latency). Modern deep learning hardware accelerators (e.g., NVIDIA Hopper/Blackwell GPUs, Google TPUs) derive their performance from massive SIMD (Single Instruction, Multiple Data) parallelism across thousands of tensor cores. Recurrent architectures impose an unbreakable temporal recurrence dependency:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t). \quad (4.25)$$

Computing hidden state $\mathbf{h}_t$ strictly requires the completion of state $\mathbf{h}_{t-1}$. Consequently, an RNN cannot parallelize training across the temporal dimension T . While convolutional and attention layers process the entire sequence in $O(1)$ parallel sequential steps via batched matrix multiplications, RNNs require $O(T)$ sequential operations, rendering pre-training on multi-trillion-token web-scale corpora computationally intractable.

3. Temporal path length ($O(T)$ vs. $O(1)$ direct routing). In an unrolled recurrent graph, the shortest path between token x_1 and prediction y_T scales linearly with sequence length: $O(T)$. Every additional step introduces an intermediate Jacobian multiplication $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$. Even with the additive Constant Error Carousel of LSTMs, error signals become noisy and diffuse over horizons exceeding 100 to 200 tokens.

The two-stage historical leap to foundation models: These structural ceilings precipitated a two-stage intellectual revolution:

1. **Stage 1: Soft Alignment (Bahdanau et al., 2014 [12]):** Additive Attention shattered the information vector bottleneck by maintaining direct memory access to all encoder annotations $\{\mathbf{h}_1, \dots, \mathbf{h}_{T_x}\}$. However, because the underlying backbone remained recurrent, training was still bound to the sequential execution wall.
2. **Stage 2: Attention Is All You Need (Vaswani et al., 2017 [1]):** Vaswani and colleagues realized that if attention provides direct $O(1)$ information flow between any pair of tokens regardless of temporal distance, **recurrence is completely redundant**. Discarding recurrent cells entirely in favor of multi-head self-attention unlocked full GPU sequence parallelization and dynamic contextualization, establishing the modern Transformer architecture.

4.8.1 Breaking the vector bottleneck: Bahdanau additive attention

In the standard Seq2Seq architecture (Section 4.7), the encoder must squeeze all grammatical, lexical, and structural information of a source sentence of arbitrary length ($T_x = 40, 80$) into the single final state $\mathbf{h}_{T_x} \in \mathbb{R}^{d_h}$. As sentence length increases, empirical translation performance drops precipitously: the fixed-size hidden vector acts as an extreme information bottleneck.

To shatter this bottleneck, Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio [12] introduced ****Additive (Soft) Attention****, allowing the decoder to dynamically search and focus across all encoder hidden states at every generation step.

1. Mathematical formulation with explicit tensor dimensions. Let the source sequence $(x_1, \dots, x_{T_x})$ be encoded by a Bidirectional RNN, producing a sequence of source annotation vectors:

$$\mathbf{h}_j = \begin{bmatrix} \vec{\mathbf{h}}_j \\ \overleftarrow{\mathbf{h}}_j \end{bmatrix} \in \mathbb{R}^{2d_h}, \quad \forall j \in \{1, 2, \dots, T_x\}. \quad (4.26)$$

At decoder generation step i , let $\mathbf{s}_{i-1} \in \mathbb{R}^{d_{\text{dec}}}$ denote the previous decoder hidden state.

1. **Alignment Energy Score:** The relevance of source token j to target emission i is evaluated by a small feed-forward network parameterized by $\mathbf{W}_a \in \mathbb{R}^{d_a \times d_{\text{dec}}}$, $\mathbf{U}_a \in \mathbb{R}^{d_a \times 2d_h}$, and $\mathbf{v}_a \in \mathbb{R}^{d_a}$:

$$e_{ij} = \mathbf{v}_a^\top \tanh(\mathbf{W}_a \mathbf{s}_{i-1} + \mathbf{U}_a \mathbf{h}_j) \in \mathbb{R}. \quad (4.27)$$

2. **Softmax Attention Weights:** Exponentiating and normalizing across all T_x source positions yields a valid categorical distribution over input tokens:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \quad \text{where } \alpha_{ij} \in (0, 1) \text{ and } \sum_{j=1}^{T_x} \alpha_{ij} = 1. \quad (4.28)$$

The scalar α_{ij} represents the degree to which the decoder should attend to source word x_j when generating target word y_i .

3. **Dynamic Context Vector:** The context vector $\mathbf{c}_i$ is computed as the expected annotation under distribution α_i :

$$\mathbf{c}_i = \sum_{j=1}^{T_x} \alpha_{ij} \mathbf{h}_j \in \mathbb{R}^{2d_h}. \quad (4.29)$$

Unlike standard Seq2Seq where context $\mathbf{h}_{T_x}$ is static, $\mathbf{c}_i$ is dynamically recomputed at every decoder step i , steering the model's focus to the relevant source phrase.

4. **Decoder State and Emission Update:** The decoder hidden state updates by conditioning on the previous token y_{i-1} and dynamic context $\mathbf{c}_i$:

$$\mathbf{s}_i = \text{RNN}_{\text{dec}}(\mathbf{s}_{i-1}, [\mathbf{y}_{i-1}, \mathbf{c}_i]), \quad (4.30)$$

and target token probabilities are emitted via $\hat{\mathbf{y}}_i = \text{softmax}(\mathbf{W}_s[\mathbf{s}_i, \mathbf{y}_{i-1}, \mathbf{c}_i] + \mathbf{b}_s)$.

Figure 4.4 illustrates the Bahdanau Additive Attention architecture.

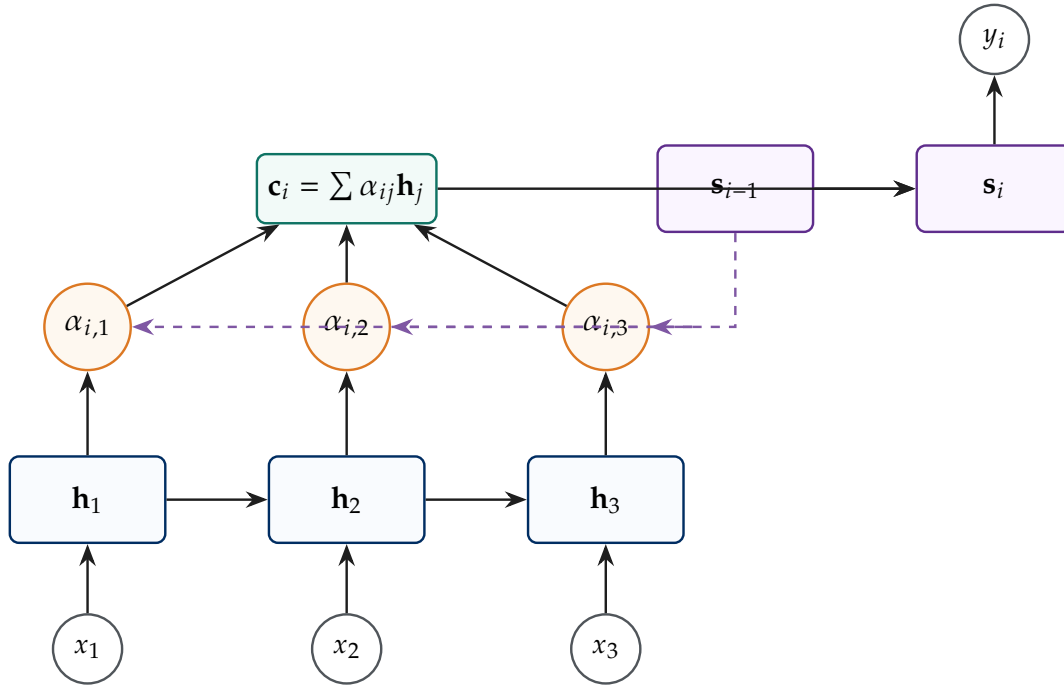


Figure 4.4: The Bahdanau Additive Attention mechanism. At decoder step i , previous decoder state $\mathbf{s}_{i-1}$ is compared against all encoder annotations $\mathbf{h}_j$ via Equation (4.27), producing attention distribution α_{ij} . The weighted sum synthesizes dynamic context vector $\mathbf{c}_i \in \mathbb{R}^{2d_h}$, directly feeding relevant source features into decoder state $\mathbf{s}_i$.

🔗 External deep dive: Historical milestone: from Schmidhuber's CEC to Sutskever's Seq2Seq

The lineage of sequence modeling represents one of the most intellectually cohesive arcs in artificial intelligence:

- In 1997, Sepp Hochreiter and Jürgen Schmidhuber published “*Long Short-Term Memory*” [21], solving the vanishing gradient problem via the Constant Error Carousel and proving that additive highways preserve error signals indefinitely.
- In 2014, Kyunghyun Cho et al. [11] introduced the Gated Recurrent Unit (GRU) and formulated RNN encoder-decoder architectures for statistical machine translation.
- Simultaneously in 2014, Ilya Sutskever, Oriol Vinyals, and Quoc V. Le [10] published “*Sequence to Sequence Learning with Neural Networks*”, demonstrating that multi-layer deep LSTMs could map sentences across languages end-to-end. Crucially, Sutskever discovered that reversing the source sentence (“ $A B C \rightarrow C B A$ ”) dramatically improved performance by shortening the initial temporal gradient distance between source and target tokens.

Bibliography

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 5998–6008, 2017.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.
- [3] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 30016–30030, 2022.
- [4] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [5] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Prentice Hall / Online Draft, 3rd draft edition, 2024. URL <https://web.stanford.edu/~jurafsky/slp3/>.
- [6] Gerasimos Potamianos and Frederick Jelinek. A study of n-gram and decision tree letter language modeling methods. *Speech Communication*, 24(2):171–192, 1998.
- [7] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- [8] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations (ICLR) Workshop Track*, 2013.
- [9] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, pages 3111–3119, 2013.
- [10] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, pages 3104–3112, 2014.
- [11] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

-
- [12] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*, 2015.
 - [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.
 - [14] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
 - [15] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
 - [16] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 27730–27744, 2022.
 - [17] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 53728–53741, 2023.
 - [18] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837, 2022.
 - [19] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
 - [20] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
 - [21] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
 - [22] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.
 - [23] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
 - [24] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314, 1989.
 - [25] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.

-
- [26] Moshe Leshno, Vladimir Ya. Lin, Allan Pinkus, and Shimon Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867, 1993.
- [27] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [28] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [29] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4):115–133, 1943.
- [30] Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.
- [31] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.
- [32] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.
- [33] Zellig S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.
- [34] John R. Firth. A synopsis of linguistic theory 1930–1955. *Studies in Linguistic Analysis*, pages 1–32, 1957.
- [35] Andrej Karpathy. Building makemore part 2: MLP language model. YouTube Video Tutorial and GitHub Repository, 2022. URL <https://github.com/karpathy/makemore>.
- [36] Grant Sanderson. Visualizing attention, embeddings and neural network trajectories. 3Blue1Brown Educational Video Series, 2024. URL <https://www.3blue1brown.com/lessons/gpt>.
- [37] Frederic Morin and Yoshua Bengio. Hierarchical probabilistic neural network language model. In *Proceedings of the Tenth International Workshop on Artificial Intelligence and Statistics (AISTATS)*, pages 246–252, 2005.
- [38] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.
- [39] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics (TACL)*, 5:135–146, 2017.
- [40] Enkk. Cosa sono gli embeddings: Geometria semantica, distanza coseno e vector databases. YouTube Educational Lecture, 2023. URL <https://www.youtube.com/watch?v=5sYkC8aVv1k>.
- [41] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.
- [42] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

-
- [43] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
 - [44] Flavio Giobergia. Word embeddings and vector space relationships demo. GitHub Gist, 2024. URL <https://gist.github.com/fgiobergia/b3a20e097f9b697d0a02fb17685cfd5a>.
 - [45] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990.
 - [46] Paul J. Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 1990.
 - [47] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994.
 - [48] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pages 1310–1318, 2013.
 - [49] Andrej Karpathy. The unreasonable effectiveness of recurrent neural networks. Blog post, 2015. URL <https://karpathy.github.io/2015/05/21/rnn-effectiveness/>.